

**MINISTRY OF EDUCATION AND TRAINING  
CAN THO UNIVERSITY  
COLLEGE OF INFORMATION AND COMMUNICATION  
TECHNOLOGY  
FACULTY OF SOFTWARE ENGINEERING**



**UNDERGRADUATE THESIS  
SOFTWARE ENGINEERING MAJOR**

**Thesis title**

**DESIGN AND IMPLEMENTATION OF A HUMAN-IN-THE-  
LOOP  
AI-ASSISTED RECRUITMENT AUTOMATION PLATFORM  
FOR CV-JD EVALUATION AND RECOMMENDATION IN  
IT**

**Student**

**Pham Huu Hung - B2203557**

**Can Tho, August 2026**

**MINISTRY OF EDUCATION AND TRAINING  
CAN THO UNIVERSITY  
COLLEGE OF INFORMATION AND COMMUNICATION  
TECHNOLOGY  
FACULTY OF SOFTWARE ENGINEERING**



**UNDERGRADUATE THESIS  
SOFTWARE ENGINEERING MAJOR**

**Thesis title**

**DESIGN AND IMPLEMENTATION OF A HUMAN-IN-THE-  
LOOP  
AI-ASSISTED RECRUITMENT AUTOMATION PLATFORM  
FOR CV-JD EVALUATION AND RECOMMENDATION IN  
IT**

**Supervisor**

Nguyen Thanh Khoa, Ph.D.

**Student**

Pham Huu Hung - B2203557

**Can Tho, August 2026**

## ACKNOWLEDGEMENTS

This thesis represents the culmination of my academic journey, sustained effort, and dedication throughout the process of research and system development. The completion of this work would not have been possible without the valuable guidance, encouragement, and support of my supervisor, lecturers, family, and friends, all of whom contributed greatly to helping me overcome the challenges encountered during the course of this thesis.

I would like to express my sincere gratitude to the lecturers and staff of the College of Information and Communication Technology, Can Tho University. Throughout my years of study, they have provided me with valuable theoretical knowledge, practical skills, and a supportive academic environment, which have formed an essential foundation for the completion of this thesis.

I would like to extend my deepest appreciation to Dr. Nguyen Thanh Khoa for his dedicated guidance, constructive feedback, and continuous support throughout this thesis. His academic and technical advice played an important role in shaping the research direction, addressing challenges during the design and implementation of the CareerFit system, and improving the overall quality of this work.

I would also like to express my heartfelt thanks to my family and friends for their constant encouragement, understanding, and support throughout my studies. Their trust and encouragement provided me with the motivation to remain focused and complete this thesis.

Although I have made every effort to conduct this work carefully and thoroughly within the available time and resources, certain limitations may inevitably remain. I sincerely welcome constructive comments, feedback, and recommendations from my supervisor, lecturers, and other researchers, as these will help me further improve my knowledge, professional competence, and research experience in the future.

Once again, I would like to express my sincere gratitude to everyone who has contributed, directly or indirectly, to the completion of this thesis.

Can Tho, August 2026

**Student**

Pham Huu Hung

## **DECLARATION OF ORIGINALITY**

I hereby declare that this thesis, titled “Design and Implementation of a Human-in-the-Loop AI-Assisted Recruitment Automation Platform for CV-JD Evaluation and Recommendation in IT,” is my original work completed under the supervision of Dr. Nguyen Thanh Khoa. The implementation, experiments, data analysis, and conclusions presented in this report are reported truthfully within the stated scope.

All ideas, publications, standards, software documentation, figures, and other materials obtained from external sources are acknowledged through citations or source notes. I accept full responsibility for the accuracy, academic integrity, and originality of the submitted work.

Can Tho, August 2026

**Student**

Pham Huu Hung

## **SUPERVISOR'S COMMENTS**

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Can Tho, August 2026  
**Supervisor**

Nguyen Thanh Khoa, Ph.D.

## TABLE OF CONTENTS

|                                                            |      |
|------------------------------------------------------------|------|
| ACKNOWLEDGEMENTS.....                                      | i    |
| DECLARATION OF ORIGINALITY .....                           | ii   |
| SUPERVISOR'S COMMENTS .....                                | iii  |
| TABLE OF CONTENTS.....                                     | iv   |
| LIST OF TABLES .....                                       | ix   |
| LIST OF FIGURES .....                                      | xi   |
| LIST OF ABBREVIATIONS .....                                | xiii |
| ABSTRACT .....                                             | xiv  |
| PART 1. INTRODUCTION .....                                 | 1    |
| 1. Problem Statement.....                                  | 1    |
| 2. Background and Related Work.....                        | 1    |
| 3. Research Objectives .....                               | 2    |
| 3.1 Overall Objective.....                                 | 2    |
| 3.2 Specific Objectives .....                              | 2    |
| 4. Research Subjects and Scope .....                       | 3    |
| 4.1 Research Subject.....                                  | 3    |
| 4.2 Research Scope.....                                    | 3    |
| 5. Research Methods and Content .....                      | 3    |
| 5.1 Research Methods.....                                  | 3    |
| 5.2 Research Content.....                                  | 4    |
| 6. Key Contributions.....                                  | 4    |
| 7. Thesis Structure .....                                  | 5    |
| PART 2. MAIN CONTENT .....                                 | 6    |
| CHAPTER 1. PROBLEM DESCRIPTION AND REQUIREMENTS.....       | 6    |
| 1.1 System Analysis Context .....                          | 6    |
| 1.2 Stakeholders and Actors .....                          | 7    |
| 1.3 Functional Requirements .....                          | 9    |
| 1.4 Non-Functional Requirements.....                       | 10   |
| 1.5 Use-Case Analysis .....                                | 12   |
| 1.5.1 Manage Career Profile.....                           | 12   |
| 1.5.2 Explore Jobs .....                                   | 16   |
| 1.5.3 Manage Job Applications .....                        | 18   |
| 1.5.4 Provide Matching Feedback .....                      | 21   |
| 1.5.5 Review Personalized Career Insights .....            | 23   |
| 1.5.6 Manage AutoFit .....                                 | 24   |
| 1.5.7 Respond Through Actionable Email.....                | 27   |
| 1.5.8 Manage Employer Profile and Job Postings.....        | 30   |
| 1.5.9 Review and Process Applicants.....                   | 32   |
| 1.5.10 Manage Talent Pool and Invitations .....            | 34   |
| 1.5.11 Review Recruitment Analytics .....                  | 36   |
| 1.5.12 Report Suspicious Recruitment Content .....         | 38   |
| 1.5.13 Administer Platform Access and Job Visibility ..... | 40   |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1.5.14 Review and Resolve Content Reports .....                   | 42 |
| 1.6 Chapter Summary .....                                         | 43 |
| CHAPTER 2. THEORETICAL BACKGROUND .....                           | 44 |
| 2.1 Recruitment Information and CV–Job Description Matching ..... | 44 |
| 2.2 Text Representation and Preprocessing .....                   | 44 |
| 2.2.1 Document Normalization .....                                | 44 |
| 2.2.2 Bag-of-Words Representation .....                           | 45 |
| 2.3 TF-IDF and Cosine Similarity .....                            | 45 |
| 2.3.1 Term Frequency and Inverse Document Frequency .....         | 45 |
| 2.3.2 Cosine Similarity .....                                     | 45 |
| 2.4 Matching, Recommendation, and Ranking .....                   | 46 |
| 2.5 Relevance Feedback and the Rocchio Method .....               | 47 |
| 2.5.1 Relevance Feedback .....                                    | 47 |
| 2.5.2 Feedback Semantics in CareerFit .....                       | 47 |
| 2.6 Evaluation Metrics for Ranked Results .....                   | 48 |
| 2.6.1 Precision@K, Recall@K, and HitRate@K .....                  | 48 |
| 2.6.2 Mean Reciprocal Rank .....                                  | 48 |
| 2.6.3 Discounted Cumulative Gain .....                            | 48 |
| 2.7 Human-in-the-Loop and Explainable Automation .....            | 49 |
| 2.7.1 Levels of Human Involvement .....                           | 49 |
| 2.7.2 Explainability and Auditability .....                       | 49 |
| 2.8 Web Architecture, Security, and Audit Foundations .....       | 50 |
| 2.8.1 Client–Server and REST .....                                | 50 |
| 2.8.2 Authentication, Authorization, and Action Tokens .....      | 50 |
| 2.8.3 Audit Logging .....                                         | 50 |
| 2.9 Related Work and Research Gap .....                           | 51 |
| 2.10 Theoretical Background Summary .....                         | 52 |
| CHAPTER 3. SYSTEM DESIGN AND IMPLEMENTATION .....                 | 53 |
| 3.1 System Architecture .....                                     | 53 |
| 3.2 Module Design .....                                           | 54 |
| 3.3 Data Design .....                                             | 55 |
| 3.3.1 Data Design Overview .....                                  | 55 |
| 3.3.2 Conceptual Data Model .....                                 | 56 |
| 3.3.3 Logical Data Model .....                                    | 57 |
| 3.3.4 Physical Data Model .....                                   | 61 |
| 3.3.5 Data Dictionary .....                                       | 65 |
| 3.4 Functional Design .....                                       | 65 |
| 3.4.1 Functional Design for Candidate .....                       | 65 |
| 3.4.1.1 Explore Jobs .....                                        | 65 |
| 3.4.1.2 Manage AutoFit .....                                      | 68 |
| 3.4.1.3 Upload and Confirm a CV .....                             | 70 |
| 3.4.2 Functional Design for Recruiter .....                       | 72 |
| 3.4.2.1 Manage Job Postings and Applicants .....                  | 72 |
| 3.4.2.2 Manage Talent Pool and Invitations .....                  | 74 |

|                                                           |     |
|-----------------------------------------------------------|-----|
| 3.4.3 Functional Design for Administrator.....            | 76  |
| 3.4.3.1 Review Administrative Audit Activity .....        | 76  |
| 3.5 Security, Failure, and Consistency Design .....       | 77  |
| 3.5.1 Authentication and Authorization.....               | 77  |
| 3.5.2 Failure Handling.....                               | 78  |
| 3.5.3 Identified Design Risks .....                       | 78  |
| 3.6 Deployment Architecture.....                          | 79  |
| 3.7 Implementation Overview .....                         | 80  |
| 3.8 Authentication and Authorization Implementation ..... | 82  |
| 3.8.1 Login and JWT Processing.....                       | 82  |
| 3.8.2 URL Rules and Ownership Checks .....                | 83  |
| 3.9 CV Ingestion and Validation.....                      | 84  |
| 3.9.1 Upload and Manual Creation.....                     | 84  |
| 3.9.2 File Validation and Extraction .....                | 85  |
| 3.10 Text Normalization and TF-IDF Implementation.....    | 87  |
| 3.10.1 Text Normalization .....                           | 87  |
| 3.10.2 Static Corpus and Vector Construction.....         | 87  |
| 3.11 Matching and Recommendation Implementation.....      | 89  |
| 3.11.1 Scoring Service.....                               | 89  |
| 3.11.2 Matching Orchestration and Persistence .....       | 90  |
| 3.11.3 Recommendation Service .....                       | 90  |
| 3.12 Feedback Learning with Rocchio .....                 | 92  |
| 3.13 Application and Recruiter Workflow.....              | 94  |
| 3.14 AutoFit and Background Processing .....              | 95  |
| 3.14.1 Async Executor and Scheduler.....                  | 95  |
| 3.14.2 Auto-Apply .....                                   | 96  |
| 3.14.3 Notification Guard and Delivery .....              | 98  |
| 3.15 Email Action and Audit Implementation.....           | 100 |
| 3.16 Persistence and API Implementation.....              | 102 |
| 3.17 Frontend Integration .....                           | 103 |
| 3.18 Deployment and Observability Implementation .....    | 104 |
| 3.19 Implementation Limitations .....                     | 105 |
| 3.20 Chapter Summary .....                                | 106 |
| CHAPTER 4. TESTING AND EVALUATION.....                    | 107 |
| 4.1 Evaluation Objectives.....                            | 107 |
| 4.2 Evaluation Environment.....                           | 107 |
| 4.3 Dataset and Evaluation Design.....                    | 108 |
| 4.3.1 Controlled Algorithm Dataset.....                   | 108 |
| 4.3.2 Local Runtime Data.....                             | 108 |
| 4.4 Backend Automated Testing.....                        | 108 |
| 4.5 Controlled Algorithm Results.....                     | 109 |
| 4.6 Concurrency Observation in Benchmark Runs .....       | 110 |
| 4.7 Frontend Build Evaluation.....                        | 110 |
| 4.8 Browser End-to-End Evaluation.....                    | 110 |



|                                                         |     |
|---------------------------------------------------------|-----|
| 4.9 Security-Oriented API Checks .....                  | 111 |
| 4.10 Runtime Health, Monitoring, and Local Latency..... | 112 |
| 4.11 Evaluation Summary by Research Question .....      | 113 |
| 4.12 Threats to Validity.....                           | 113 |
| 4.12.1 Construct Validity .....                         | 113 |
| 4.12.2 Internal Validity .....                          | 113 |
| 4.12.3 External Validity .....                          | 114 |
| 4.12.4 Reproducibility .....                            | 114 |
| 4.13 Chapter Summary .....                              | 114 |
| PART 3. CONCLUSION .....                                | 115 |
| 1. Summary of Results.....                              | 115 |
| 2. Achievement of Thesis Objectives .....               | 115 |
| 2.1 Role-Based Recruitment Platform.....                | 115 |
| 2.2 CV and Job-Description Processing .....             | 115 |
| 2.3 Matching and Recommendation.....                    | 115 |
| 2.4 Feedback Learning .....                             | 115 |
| 2.5 Policy-Driven Automation and Email Action.....      | 116 |
| 2.6 Auditability and Evaluation .....                   | 116 |
| 3. Discussion.....                                      | 117 |
| 3.1 Value of an Interpretable Baseline.....             | 117 |
| 3.2 Human-in-the-Loop as System Structure .....         | 117 |
| 3.3 Meaning of the Rocchio Improvement.....             | 117 |
| 3.4 Test Results versus Background Errors .....         | 117 |
| 3.5 Product Positioning.....                            | 117 |
| 4. Limitations .....                                    | 117 |
| 4.1 Data and Algorithm Limitations .....                | 117 |
| 4.2 Evaluation Limitations .....                        | 118 |
| 4.3 Security and Privacy Limitations .....              | 118 |
| 4.4 Reliability and Maintainability Limitations .....   | 118 |
| 5. Future Work .....                                    | 118 |
| 6. Closing Remarks.....                                 | 118 |
| REFERENCES .....                                        | 119 |
| APPENDICES .....                                        | 121 |
| Appendix A. Requirement-Test Traceability Matrix .....  | 121 |
| Appendix B. Selected API Contracts.....                 | 122 |
| Appendix C. Full Data Dictionary.....                   | 123 |
| C.1 Physical Table Catalogue .....                      | 123 |
| C.2 Field-Level Data Dictionary .....                   | 127 |
| C.2.1 Identity, Candidate, CV, and Portfolio .....      | 127 |
| 1.1.1 user_account .....                                | 127 |
| 1.1.2 user_settings.....                                | 129 |
| 1.1.3 candidate .....                                   | 130 |
| 1.1.4 candidate_portfolio_link .....                    | 133 |
| 1.1.5 candidate_portfolio_project .....                 | 134 |

|                                                      |     |
|------------------------------------------------------|-----|
| 1.1.6 cv .....                                       | 136 |
| 1.1.7 skills .....                                   | 140 |
| C.2.2 Employer, Job, Matching, and Recruitment ..... | 142 |
| 1.1.8 employer_profile .....                         | 142 |
| 1.1.9 job .....                                      | 145 |
| 1.1.10 matching .....                                | 151 |
| 1.1.11 feedback .....                                | 154 |
| 1.1.12 application .....                             | 156 |
| 1.1.13 recommendation_interaction .....              | 159 |
| 1.1.14 recruiter_cv_bookmark .....                   | 161 |
| C.2.3 Automation, Email, and Notification .....      | 163 |
| 1.1.15 automation_policy .....                       | 163 |
| 1.1.16 email_action .....                            | 167 |
| 1.1.17 email_action_token .....                      | 169 |
| 1.1.18 notification_job .....                        | 172 |
| 1.1.19 notification_delivery_log .....               | 174 |
| C.2.4 Reporting, Analytics, and Audit .....          | 176 |
| 1.1.20 content_report .....                          | 176 |
| 1.1.21 analytics_event .....                         | 179 |
| 1.1.22 audit_log .....                               | 181 |
| 1.1.23 job_trend_snapshot .....                      | 183 |
| 1.1.24 job_market_snapshot .....                     | 184 |
| Appendix D. Evaluation Summary .....                 | 186 |
| Appendix E. UAT and Demonstration Script .....       | 187 |
| Preparation .....                                    | 187 |
| Execution .....                                      | 187 |
| Acceptance and cleanup .....                         | 187 |
| Appendix F. Local Deployment Instructions .....      | 188 |
| Prerequisites .....                                  | 188 |
| Startup procedure .....                              | 188 |
| Verification .....                                   | 188 |
| Shutdown and cleanup .....                           | 188 |
| Appendix G. Role-Based User Guide .....              | 189 |
| Guest .....                                          | 189 |
| Candidate .....                                      | 189 |
| Recruiter .....                                      | 189 |
| Administrator .....                                  | 189 |

## LIST OF TABLES

|                                                                                 |     |
|---------------------------------------------------------------------------------|-----|
| Table 1.1. Mapping of objectives, contributions, and evidence .....             | 4   |
| Table 1.2. Actors and responsibilities .....                                    | 7   |
| Table 1.3. Consolidated functional requirements .....                           | 9   |
| Table 1.4. Non-functional requirements and design responses .....               | 10  |
| Table 1.5. Use case - Manage Career Profile .....                               | 12  |
| Table 1.6. Use case - Explore Jobs .....                                        | 16  |
| Table 1.7. Use case - Manage Job Applications .....                             | 18  |
| Table 1.8. Use case - Provide Matching Feedback.....                            | 21  |
| Table 1.9. Use case - Review Personalized Career Insights.....                  | 23  |
| Table 1.10. Use case - Manage AutoFit.....                                      | 24  |
| Table 1.11. Use case - Respond Through Actionable Email .....                   | 27  |
| Table 1.12. Use case - Manage Employer Profile and Job Postings .....           | 30  |
| Table 1.13. Use case - Review and Process Applicants .....                      | 32  |
| Table 1.14. Use case - Manage Talent Pool and Invitations.....                  | 34  |
| Table 1.15. Use case - Review Recruitment Analytics.....                        | 36  |
| Table 1.16. Use case - Report Suspicious Recruitment Content .....              | 38  |
| Table 1.17. Use case - Administer Platform Access and Job Visibility.....       | 40  |
| Table 1.18. Use case - Review and Resolve Content Reports.....                  | 42  |
| Table 2.1. Positioning of CareerFit against major approach categories .....     | 51  |
| Table 3.1. Backend modules and responsibilities.....                            | 54  |
| Table 3.2. Key integrity constraints.....                                       | 64  |
| Table 3.3. Interface components for Explore Jobs .....                          | 66  |
| Table 3.4. Interface components for Manage AutoFit.....                         | 68  |
| Table 3.5. Interface components for CV upload.....                              | 70  |
| Table 3.6. Interface components for Job and applicant management.....           | 72  |
| Table 3.7. Interface components for Talent Pool and invitations .....           | 74  |
| Table 3.8. Interface components for administrative audit activity .....         | 76  |
| Table 3.9. Design risks and required treatment .....                            | 78  |
| Table 3.10. Main implementation technologies.....                               | 80  |
| Table 3.11. CV processing states .....                                          | 84  |
| Table 3.12. Implemented score interpretation .....                              | 89  |
| Table 3.13. Feedback handling .....                                             | 92  |
| Table 3.14. Representative API-to-service mapping .....                         | 102 |
| Table 3.15. Verified implementation limitations .....                           | 105 |
| Table 4.1. Evaluation environment.....                                          | 107 |
| Table 4.2. Backend automated test results.....                                  | 108 |
| Table 4.3. Baseline and Rocchio results on the synthetic holdout scenario ..... | 109 |
| Table 4.4. Integrated Chrome workflow results .....                             | 111 |
| Table 4.5. API authorization observations.....                                  | 111 |
| Table 4.6. Runtime observations .....                                           | 112 |
| Table 4.7. Answers supported by current evidence .....                          | 113 |
| Table Con.1. Objective assessment .....                                         | 116 |

|                                                                              |     |
|------------------------------------------------------------------------------|-----|
| Table App.A.1. Requirement–test traceability for the approved Use Cases .... | 121 |
| Table App.B.1. Selected CareerFit API contracts.....                         | 122 |
| Table App.C.1. CareerFit physical table catalogue .....                      | 123 |
| Table App.C.2. Data dictionary for user_account .....                        | 127 |
| Table App.C.3. Data dictionary for user_settings.....                        | 129 |
| Table App.C.4. Data dictionary for candidate .....                           | 130 |
| Table App.C.5. Data dictionary for candidate_portfolio_link.....             | 133 |
| Table App.C.6. Data dictionary for candidate_portfolio_project.....          | 134 |
| Table App.C.7. Data dictionary for cv.....                                   | 136 |
| Table App.C.8. Data dictionary for skills .....                              | 140 |
| Table App.C.9. Data dictionary for employer_profile.....                     | 142 |
| Table App.C.10. Data dictionary for job .....                                | 145 |
| Table App.C.11. Data dictionary for matching.....                            | 151 |
| Table App.C.12. Data dictionary for feedback .....                           | 154 |
| Table App.C.13. Data dictionary for application.....                         | 156 |
| Table App.C.14. Data dictionary for recommendation_interaction .....         | 159 |
| Table App.C.15. Data dictionary for recruiter_cv_bookmark .....              | 161 |
| Table App.C.16. Data dictionary for automation_policy .....                  | 163 |
| Table App.C.17. Data dictionary for email_action.....                        | 167 |
| Table App.C.18. Data dictionary for email_action_token.....                  | 169 |
| Table App.C.19. Data dictionary for notification_job.....                    | 172 |
| Table App.C.20. Data dictionary for notification_delivery_log.....           | 174 |
| Table App.C.21. Data dictionary for content_report .....                     | 176 |
| Table App.C.22. Data dictionary for analytics_event.....                     | 179 |
| Table App.C.23. Data dictionary for audit_log .....                          | 181 |
| Table App.C.24. Data dictionary for job_trend_snapshot.....                  | 183 |
| Table App.C.25. Data dictionary for job_market_snapshot .....                | 184 |

## LIST OF FIGURES

|                                                                                         |     |
|-----------------------------------------------------------------------------------------|-----|
| Figure 1.1. CareerFit system context.....                                               | 6   |
| Figure 1.2. Overall CareerFit use-case diagram.....                                     | 8   |
| Figure 1.3. Candidate use-case diagram.....                                             | 12  |
| Figure 1.4. Recruiter use-case diagram .....                                            | 30  |
| Figure 1.5. Shared Candidate-Recruiter reporting use-case diagram.....                  | 38  |
| Figure 1.6. Administrator use-case diagram.....                                         | 40  |
| Figure 3.1. CareerFit container and component architecture .....                        | 54  |
| Figure 3.2. CareerFit conceptual data model.....                                        | 56  |
| Figure 3.3. CareerFit logical data model - identity and Career Profile .....            | 57  |
| Figure 3.4. CareerFit logical data model - recruitment and Matching .....               | 58  |
| Figure 3.5. CareerFit logical data model - automation and communication .....           | 59  |
| Figure 3.6. CareerFit logical data model - governance and analytics .....               | 60  |
| Figure 3.7. CareerFit physical data model - identity and Career Profile tables .....    | 61  |
| Figure 3.8. CareerFit physical data model - recruitment and Matching tables .....       | 62  |
| Figure 3.9. CareerFit physical data model - automation and communication tables.....    | 63  |
| Figure 3.10. CareerFit physical data model - governance and analytics tables .....      | 64  |
| Figure 3.11. Annotated interface for Explore Jobs.....                                  | 66  |
| Figure 3.12. Sequence diagram for Explore Jobs.....                                     | 67  |
| Figure 3.13. Annotated interface for Manage AutoFit .....                               | 68  |
| Figure 3.14. Sequence diagram for Manage AutoFit .....                                  | 69  |
| Figure 3.15. Annotated interface for CV upload .....                                    | 70  |
| Figure 3.16. Sequence diagram for CV upload, review, and confirmation .....             | 71  |
| Figure 3.17. Annotated interface for Job and applicant management .....                 | 72  |
| Figure 3.18. Sequence diagram for Job publication and applicant review.....             | 73  |
| Figure 3.19. Annotated interface for Talent Pool and invitations.....                   | 74  |
| Figure 3.20. Sequence diagram for Talent Pool bookmarking and invitation....            | 75  |
| Figure 3.21. Annotated interface for administrative audit activity.....                 | 76  |
| Figure 3.22. Sequence diagram for reviewing administrative audit activity.....          | 77  |
| Figure 3.23. Local and containerized deployment topology .....                          | 80  |
| Figure 3.24. Backend module structure and request path .....                            | 82  |
| Figure 3.25. JWT authentication, role authorization, and ownership-check sequence ..... | 84  |
| Figure 3.26. CV ingestion, review, confirmation, and matching flowchart .....           | 86  |
| Figure 3.27. Seed-corpus initialization and TF-IDF construction flowchart .....         | 88  |
| Figure 3.28. CV-Job matching and Potential assessment flowchart .....                   | 91  |
| Figure 3.29. Rocchio feedback-learning and recomputation flowchart .....                | 93  |
| Figure 3.30. Application and invitation UML state machine .....                         | 95  |
| Figure 3.31. AutoFit eligibility and application decision flowchart.....                | 97  |
| Figure 3.32. Notification policy evaluation and delivery outcome flowchart....          | 99  |
| Figure 3.33. Actionable-email confirmation and redemption flowchart .....               | 101 |
| Figure 3.34. Frontend request and API-response sequence.....                            | 104 |

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| Figure 4.1. Baseline and Rocchio benchmark metrics at $K = 5$ ..... | 110 |
|---------------------------------------------------------------------|-----|

## LIST OF ABBREVIATIONS

| Abbreviation | Full Term                                 | Meaning / Explanation                                                |
|--------------|-------------------------------------------|----------------------------------------------------------------------|
| API          | Application Programming Interface         | A defined interface through which software components communicate    |
| CV           | Curriculum Vitae                          | A document describing a candidate's qualifications and experience    |
| E2E          | End-to-End                                | Testing of a complete workflow across integrated components          |
| HITL         | Human-in-the-Loop                         | Automation in which humans retain oversight or approval authority    |
| JD           | Job Description                           | A description of a job's responsibilities and requirements           |
| JWT          | JSON Web Token                            | A compact token format used for authentication and authorization     |
| MRR          | Mean Reciprocal Rank                      | A ranking metric based on the position of the first relevant item    |
| nDCG         | Normalized Discounted Cumulative Gain     | A ranking metric that accounts for relevance and position            |
| NLP          | Natural Language Processing               | Methods for processing and analyzing human language                  |
| OCR          | Optical Character Recognition             | Extraction of machine-readable text from images or scanned documents |
| RBAC         | Role-Based Access Control                 | Authorization based on assigned user roles                           |
| REST         | Representational State Transfer           | An architectural style for networked APIs                            |
| TF-IDF       | Term Frequency–Inverse Document Frequency | A weighting method for representing text terms                       |
| UAT          | User Acceptance Testing                   | Validation of whether the system satisfies intended user workflows   |

## ABSTRACT

**Background:** Recruitment in the information technology sector requires candidates and recruiters to compare many types of information from curricula vitae and job descriptions. Most job portals support posting, searching, and manual applications, while automated screening tools may not clearly explain how a score is calculated or how an automated action is controlled.

**Objectives:** This thesis designs and implements CareerFit IT AutoPilot, a web-based recruitment platform that combines a job portal, CV-job description matching, personalized job recommendations, policy-based automation, and Human-in-the-Loop interaction in one auditable workflow.

**Methods:** Recruitment text is normalized and represented with Term Frequency-Inverse Document Frequency vectors. Cosine similarity ranks CV-job pairs and candidate-job recommendations, while Rocchio feedback updates learned representations from explicit feedback. A separate skill-transfer model explains Potential candidates without changing the direct cosine score. AutoFit checks consent and policy limits before an action. The platform also has a report moderation flow for suspicious Jobs and CVs. CareerFit uses a Spring Boot backend, a React and TypeScript frontend, and PostgreSQL with Flyway migrations.

**Results:** On August 12, 2026, the refreshed backend suite passed 141 of 141 tests across 35 suites. TypeScript checking, ESLint, the production frontend build, and the bundle check also passed. All 49 Chrome workflow, contract, and resilience tests passed against the integrated application, including Job/CV reporting and administrator moderation cases. In the controlled synthetic benchmark,  $nDCG@5$  increased from 0.037737 to 0.837737 after Rocchio feedback. These results support a tested academic prototype, but they do not prove production readiness or real-world hiring effectiveness.

**Keywords:** *recruitment automation; CV-job matching; recommendation system; Human-in-the-Loop; Rocchio feedback; explainable automation.*



## **PART 1. INTRODUCTION**

### **1. Problem Statement**

IT recruitment requires candidates and recruiters to compare skills, experience, location, and working conditions across many Jobs and CVs. As these records grow, manual comparison becomes slow and inconsistent [10].

Most Job portals support posting, search, and application but provide limited explanation of fit. CareerFit therefore separates CV–JD Matching, which compares one CV and Job, from profile-based Recommendation, which reflects broader Candidate preferences.

A Matching score alone should not trigger a recruitment action. CareerFit combines consent, policy conditions, existing state, and review points in a Human-in-the-Loop workflow.

This thesis develops CareerFit IT AutoPilot as an IT-focused decision-support platform with explainable Matching, feedback learning, controlled AutoFit, actionable email, Recruiter discovery, moderation, and auditability. It supports rather than replaces recruitment judgment.

### **2. Background and Related Work**

Vietnamese recruitment portals provide the practical baseline for online Job discovery, CV support, and employer access. ITviec and TopDev emphasize technology recruitment, while CareerViet and TopCV provide broader Job-search and recruitment services [15]–[18]. CareerFit reduces fragmentation by combining Candidate Job, CV, and Application workflows with Recruiter Job, applicant, and Talent Pool workflows in one role-based system. Email provides an additional action channel.

TF-IDF and cosine similarity have less semantic capacity than dense models, but their terms and weights are inspectable. They provide a reproducible academic baseline for explaining scores, limitations, and feedback effects.

AutoFit separates relevance from action. It checks Candidate consent and policy state, an eligible default CV, ACTIVE Job status, threshold, duplicates, and a per-run limit; notification delivery applies separate quota, cooldown, timezone, quiet-hour, and preference rules.

Related work covers lexical retrieval, learned resume–Job representations, explainable recommendation, and human-centered governance [6]–[10], [14]. CareerFit addresses an integration gap by connecting transparent scoring, feedback learning, policy-controlled actions, and audit records in one academic prototype.

### **3. Research Objectives**

#### **3.1 Overall Objective**

The overall objective of this thesis is to design, implement, and evaluate a web-based Human-in-the-Loop recruitment automation platform for the IT domain. The platform is intended to support CV–job description evaluation, personalized job recommendation, feedback-based adaptation, and policy-driven actions while preserving user control, explainability, security, and auditability.

#### **3.2 Specific Objectives**

To achieve the overall objective, the thesis defines the following specific objectives:

- Design a role-based web platform for guests, candidates, recruiters, and administrators, with protected workflows and a content-reporting path for suspicious Jobs and CVs.
- Provide candidate profile and CV management, including upload, manual drafts, OCR-assisted extraction, section review, user correction, confirmation, validation, and quality feedback before matching.
- Implement separate CV–JD matching and candidate-profile recommendation workflows using a shared text-normalization, TF-IDF, and cosine-similarity pipeline.
- Present normalized scores, relevance labels, matching reasons, validation signals, and a separate Potential explanation based on transferable skills, skill families, shared foundations, and seniority compatibility.
- Integrate explicit user feedback and apply the Rocchio relevance-feedback method to update learned representations and recompute rankings reproducibly.
  - Implement AutoFit policies that convert eligible Matching results into controlled Applications using enablement, pause state, a processed default CV, ACTIVE Job state, score threshold, duplicate checks, and a bounded per-run limit.
- Support expiring email-action links with hashed token storage, a non-mutating confirmation page, POST execution, action-state tracking, and audit logging.
- Evaluate the algorithmic behavior and system workflows using reproducible experiments, automated tests, scripted acceptance scenarios, and security and reliability checks appropriate to the project scope.

## **4. Research Subjects and Scope**

### **4.1 Research Subject**

The research subject is the IT recruitment workflow supported by a web platform, including CV and Job-description text, lexical ranking and recommendation, explicit feedback signals, policy-controlled automation, and auditable user and system actions.

### **4.2 Research Scope**

The functional scope includes public Job discovery; Candidate, Recruiter, and Administrator workspaces; reviewed CV/Profile and employer/Job management; Matching and Recommendation; Applications, Talent Pool and invitations; feedback; AutoFit; notifications; analytics; reporting, moderation, and audit.

The data-processing scope includes extraction from supported CV documents, image preprocessing and OCR fallback, OCR cleanup, section review, bilingual-oriented text normalization, TF-IDF vectorization, cosine-similarity scoring, a versioned skill-transfer Potential assessment, relevance labeling, and Rocchio feedback updates. PostgreSQL is the main data store, and Flyway manages database changes. The React and TypeScript frontend calls REST APIs provided by the Spring Boot backend.

The prototype excludes interview scheduling, offer, payroll, and employee lifecycle management; external Job-board application submission; unrestricted large-language-model decisions; distributed microservices; large message brokers; and semantic embedding models. These may be future work but are not presented as implemented features.

The evaluation scope is limited. Controlled or synthetic datasets can show whether Rocchio changes rankings in the expected direction, but they cannot prove recruitment quality in a real production setting. The source and purpose of scraped, seeded, or synthetic data must be stated. Claims about usability, performance, security, or production readiness are limited to the test environment, participants, procedures, and evidence available during the final evaluation.

## **5. Research Methods and Content**

### **5.1 Research Methods**

This thesis follows an applied software-engineering research approach. The work combines requirement analysis, architecture and data design, incremental implementation, a controlled algorithm experiment, automated backend and browser testing, runtime observation, and threats-to-validity analysis.

Implementation claims are checked against source code, migrations, API contracts, logs, test reports, and generated evaluation artifacts.

## 5.2 Research Content

The research models CareerFit as a Perception-Decision-Action-Learning-Audit workflow. It first identifies recruitment roles, requirements, and control boundaries; then designs and implements the data, matching, recommendation, feedback, automation, email action, and audit modules. Finally, the study evaluates algorithm behavior, software correctness, browser workflows, authorization, runtime health, and local latency within the stated experimental scope.

## 6. Key Contributions

The principal contribution of this thesis is the design and implementation of an integrated, controlled recruitment workflow rather than the invention of a new machine-learning model. The contributions are summarized as follows:

- An integrated recruitment platform that combines public job discovery, reviewed CV processing, CV–JD matching, personalized recommendation, application management, Recruiter Jobs, Talent Pool, and a report-moderation workflow.
- A transparent scoring design that keeps the TF-IDF/cosine direct-match score separate from the versioned Potential assessment and explains transferable skills, shared foundations, and seniority checks.
- A feedback-learning workflow based on the Rocchio method, designed to update learned representations from explicit feedback and to support reproducible recomputation rather than uncontrolled cumulative modification.
- A policy-driven AutoFit layer that separates relevance scoring from business actions and applies Human-in-the-Loop controls, user consent, operational constraints, and previous interaction states.
- An actionable email mechanism with expiring, hashed tokens, a non-mutating GET confirmation page, POST redemption, and action-state tracking.
- A traceable evaluation structure that distinguishes algorithm experiments from functional, integration, acceptance, performance, reliability, and security evidence, while explicitly documenting threats to validity.

*Table 1.1. Mapping of objectives, contributions, and evidence*

| Thesis objective                 | Implemented contribution                                    | Evaluation evidence               |
|----------------------------------|-------------------------------------------------------------|-----------------------------------|
| Analyze IT recruitment workflows | Role-specific requirements and Human-in-the-Loop boundaries | Use cases and traceability matrix |

| Thesis objective               | Implemented contribution                                                                        | Evaluation evidence                         |
|--------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------|
| Implement CV–JD matching       | TF-IDF, cosine scoring, explanations, and persisted Matchings                                   | Algorithm benchmark and backend tests       |
| Learn from feedback            | Rocchio updates followed by scheduled recomputation                                             | Baseline-versus-learned ranking metrics     |
| Support controlled automation  | AutoFit threshold, default-CV, active-Job, duplicate, per-run, notification, and audit controls | Security tests and P0 E2E flows             |
| Deliver an integrated platform | Spring Boot, React/TypeScript, PostgreSQL, and Flyway                                           | Build, runtime health, and browser evidence |

## 7. Thesis Structure

The report contains an Introduction, four chapters, and a Conclusion:

Chapter 1 defines the problem, stakeholders, requirements, and use cases.

Chapter 2 presents the theoretical background.

Chapter 3 presents the system design and implementation.

Chapter 4 presents the evaluation method, evidence, and threats to validity.

The Conclusion discusses achievements, limitations, and future work.

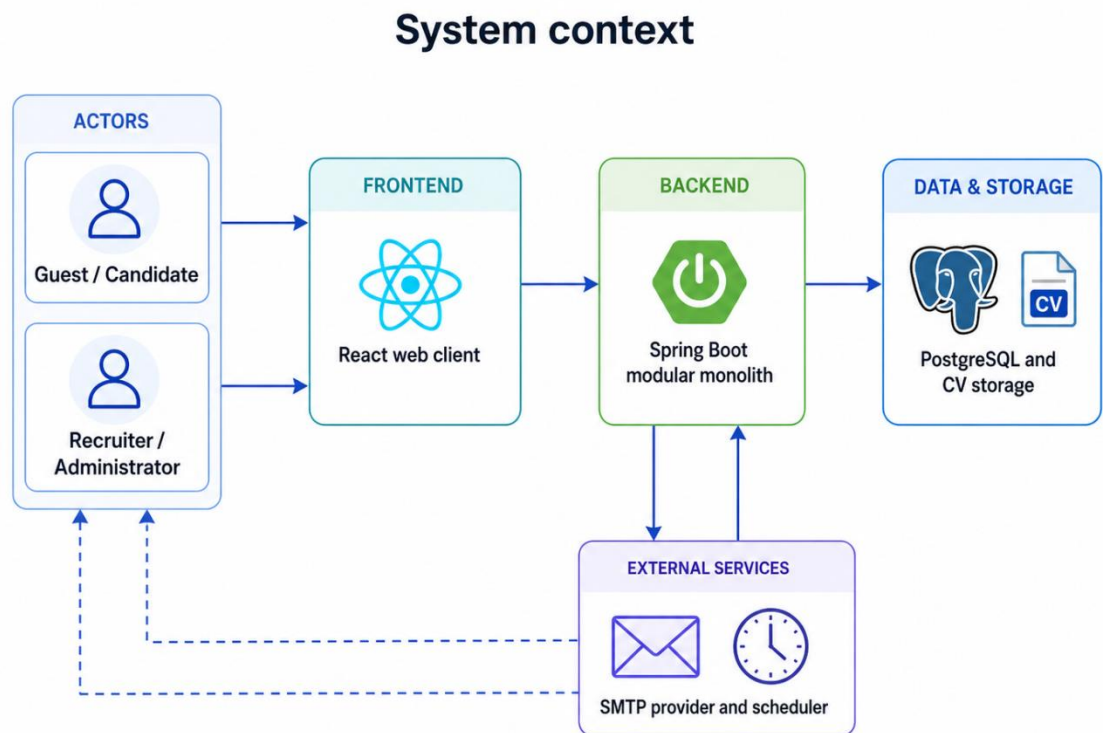
## PART 2. MAIN CONTENT

### CHAPTER 1. PROBLEM DESCRIPTION AND REQUIREMENTS

#### 1.1 System Analysis Context

CareerFit is designed as a role-based system for IT recruitment. Its boundary includes the React frontend, Spring Boot backend, PostgreSQL database, CV storage, background scheduler, and optional email provider. External job boards, interview scheduling, offers, payroll, and unrestricted automated hiring remain outside the implemented scope.

The central design distinction is between evidence, business state, and action. A matching record contains evidence about the similarity between one CV and one job. An application record represents a candidate's participation in a recruitment process. An automation policy represents consent and operational preferences. These concepts must remain separate: a high similarity score must not silently become an application, and an application status must not be interpreted as a relevance label.



*Figure 1.1. CareerFit system context*

## 1.2 Stakeholders and Actors

CareerFit supports four interactive user contexts: Guest, Candidate, Recruiter, and Administrator. Guest represents an unauthenticated frontend context, while Candidate, Recruiter, and Administrator are persisted account roles. Background scheduling and mail delivery are supporting system components rather than interactive use-case actors.

*Table 1.2. Actors and responsibilities*

| Actor         | Primary responsibilities                                                                                                                                                                | Access boundary                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Guest         | Explore public Jobs and view public Job and employer details                                                                                                                            | Public Job operations only; authentication is required for Candidate-specific actions |
| Candidate     | Manage Career Profile; explore Jobs; manage Applications; review Matching, recommendations, and analytics; provide Feedback; manage AutoFit; report an ACTIVE Job; use actionable email | Candidate-owned data and Candidate/report operations                                  |
| Recruiter     | Manage Employer Profile and Jobs; review applicants; manage Talent Pool and invitations; review recruitment analytics; report a visible CV                                              | Owned Jobs and CVs visible through an owned Job                                       |
| Administrator | Manage User access and Job visibility; review and resolve reported Job/CV cases                                                                                                         | Administrative operations only                                                        |

Stakeholder needs sometimes conflict. Candidates want relevant opportunities, privacy, and control over automatic applications. Recruiters want efficient prioritization without losing access to applicants who do not rank highly.

Administrators need visibility and recoverability without unrestricted exposure of CV content. The design therefore combines role authorization, ownership checks, policy settings, validation, explicit statuses, and audit records rather than relying on a single ranking score.

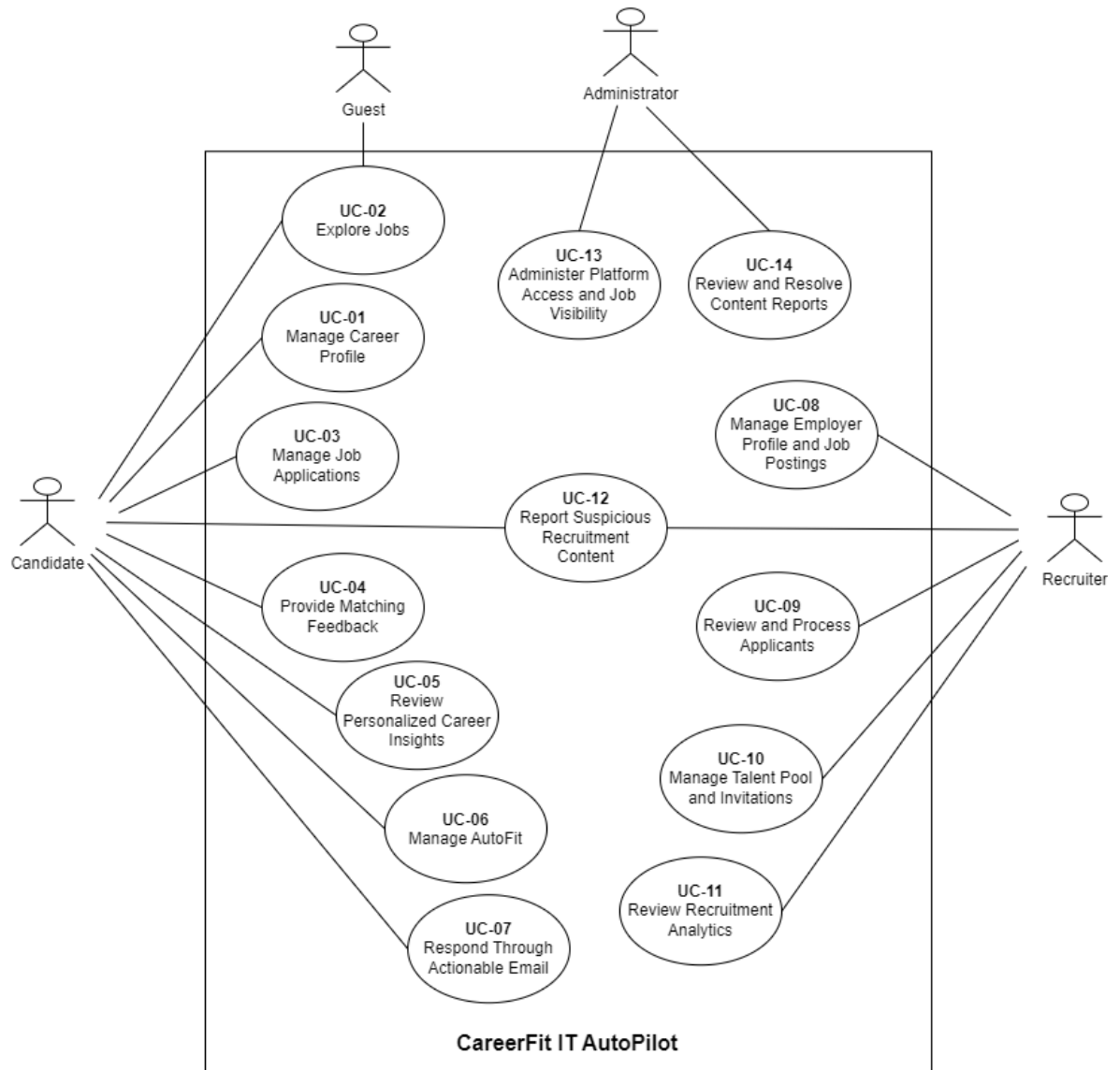


Figure 1.2. Overall CareerFit use-case diagram



### 1.3 Functional Requirements

The detailed Software Requirements Specification contains individual requirement identifiers. For the thesis, they are consolidated into functional groups that map to implemented modules and observable workflows.

*Table 1.3. Consolidated functional requirements*

| <b>Group</b>                            | <b>Required capabilities</b>                                                                                              | <b>Main actor</b>                   |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Authentication and account              | Registration, login, account lookup, role enforcement, and supporting settings                                            | Candidate, Recruiter, Administrator |
| Job exploration                         | Public search, filters, sorting, pagination, urgent Jobs, Job/employer details, and similar Jobs                          | Guest, Candidate                    |
| Career Profile                          | Candidate Profile and Portfolio editing; CV draft, upload, review, confirmation, default selection, and eligible deletion | Candidate                           |
| Matching, recommendation, and analytics | Separate CV–Job Matching and personalized Job Recommendation views; Candidate and Recruiter analytics                     | Candidate, Recruiter                |
| Employer and Job management             | Employer Profile; Job create, draft, publish, update, supported status, delete, and export                                | Recruiter                           |
| Applications and Talent Pool            | Apply, history, withdrawal, invitation response, applicant decisions, Candidate discovery, bookmarks, and invitations     | Candidate, Recruiter                |

| Group                               | Required capabilities                                                                                           | Main actor                             |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Feedback and AutoFit                | Matching Feedback;<br>AutoFit enablement,<br>threshold, pause/resume,<br>manual run, and automatic<br>execution | Candidate                              |
| Supporting notification<br>settings | Email-type preferences,<br>quota, cooldown, quiet<br>hours, digest, and related<br>delivery controls            | Candidate, Recruiter                   |
| Content reporting and<br>moderation | Candidate Job report;<br>Recruiter visible-CV<br>report; Administrator<br>queue, dismiss, and ban               | Candidate, Recruiter,<br>Administrator |
| Actionable email                    | Confirm-then-execute<br>Matching Feedback and<br>Recruiter invitation<br>response                               | Candidate                              |

The implementation exposes these groups through REST-oriented endpoints. Examples include `/api/jobs/search`, `/api/cv/upload`, `/api/cv/{cvId}/review`, `/api/matches/me/cards`, `/api/recommendations/jobs`, `/api/applications`, `/api/recruiter/jobs/{jobId}/applicants`, `/api/automation/policy`, `/api/reports/jobs/{jobId}`, `/api/reports/cvs/{cvId}`, `/api/admin/reports`, and `/api/admin/audit-logs`. Chapter 3 explains the main behavior.

## 1.4 Non-Functional Requirements

*Table 1.4. Non-functional requirements and design responses*

| Quality attribute | Requirement                                                          | Design response                                                                                                                                             |
|-------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Security          | Protected resources must require authenticated and authorized access | Stateless Spring Security filter chain, JWT processing, role rules, ownership checks, BCrypt password hashing, CORS allow-list, CSP, and HSTS configuration |

| Quality attribute | Requirement                                                                                | Design response                                                                                                                                     |
|-------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Privacy           | CV and account data must not be public or unnecessarily logged                             | Candidate-scoped APIs, local protected storage boundary, selective DTOs, and audit metadata rather than full document content                       |
| Reliability       | Repeated or failed operations should not create inconsistent domain state                  | Database constraints, transactions, unique keys, optimistic version columns, idempotency checks, explicit statuses, and retry-aware background work |
| Performance       | Public queries and ranking views should remain responsive for the academic dataset         | Database indexes, pagination, stored vectors, background scoring, and bounded result sets                                                           |
| Maintainability   | Domain changes should remain localized and database evolution reproducible                 | Modular package structure, controller–service–repository separation, DTOs, configuration properties, Flyway migrations, and automated tests         |
| Usability         | Users must understand loading, errors, empty results, score context, and available actions | Role-specific pages, normalized labels and reasons, validation signals, and explicit action controls                                                |
| Auditability      | Significant automated or administrative events should be reconstructable                   | audit_log, notification delivery records, email-action state, timestamps, actor/source metadata, and administrative views                           |

| Quality attribute | Requirement                                                                          | Design response                                                                                                 |
|-------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Explainability    | The system should expose evidence without presenting a score as a hiring probability | Lexical term/skill reasons, separate labels and application states, policy settings, and documented limitations |

Security and auditability are defense-in-depth properties. A role rule at the URL layer does not replace ownership verification in a service, and an audit row does not make an unsafe action safe. Similarly, a database index is a performance design decision, not proof of latency under load. The evaluation chapter must test each critical claim in the environment in which it is reported.

## 1.5 Use-Case Analysis

The following fourteen use cases represent the main role-based goals available through the current frontend, actionable email pages, and protected backend workflows. Related screens are grouped into one user goal instead of treating every button or passive notification as a separate use case.

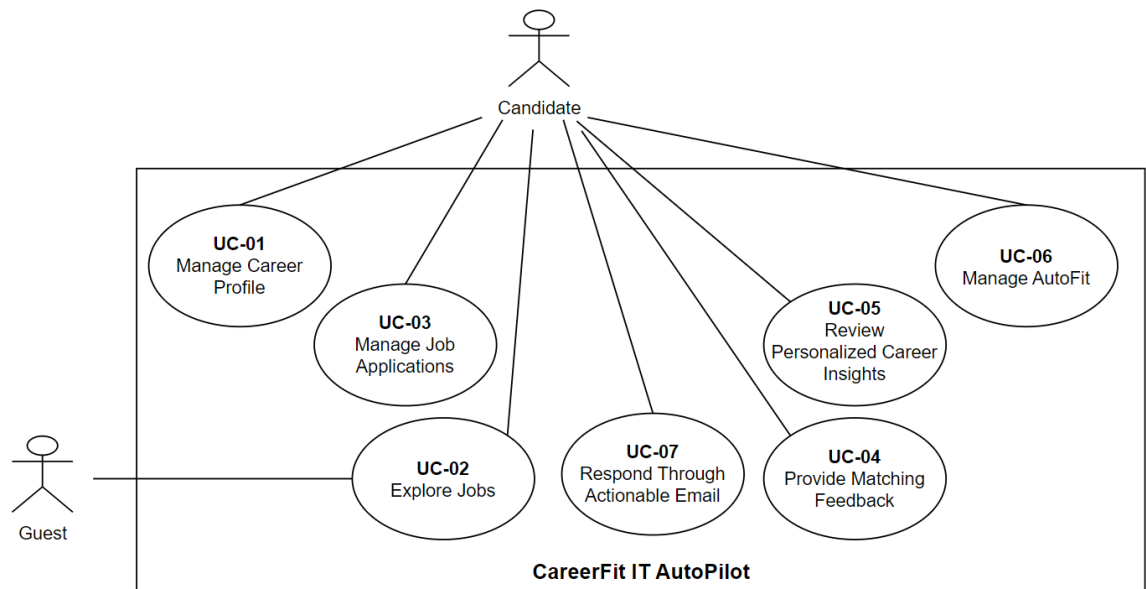


Figure 1.3. Candidate use-case diagram

### 1.5.1 Manage Career Profile

Table 1.5. Use case - Manage Career Profile

| Field            | Description           |
|------------------|-----------------------|
| Use Case ID      | UC-01                 |
| Use Case Name    | Manage Career Profile |
| Primary Actor(s) | Candidate             |

| <b>Field</b>              | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Secondary Actor(s)</b> | None                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Description</b>        | This use case allows a Candidate to maintain their Profile, Portfolio, and CV collection, including CV review and confirmation for later matching.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Preconditions</b>      | P1. The Candidate account is active and authenticated.<br>P2. The Candidate may manage only their own Profile, Portfolio, and CVs.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Trigger</b>            | The Candidate opens Career Profile and selects a Profile, Portfolio, or CV action.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Main Flow</b>          | <ol style="list-style-type: none"> <li>1. The Candidate opens Career Profile.</li> <li>2. The System displays the Profile, Portfolio, and owned CVs with their statuses.</li> <li>3. The Candidate selects Upload CV and submits a supported file.</li> <li>4. The System validates the file and extracts reviewable CV sections.</li> <li>5. The System marks the CV REVIEW_REQUIRED and displays extracted content and warnings.</li> <li>6. The Candidate reviews and corrects the content.</li> <li>7. The Candidate confirms the CV.</li> <li>8. The System validates and accepts it for matching processing.</li> <li>9. After successful processing, the System marks the CV SCORING_DONE and makes it default if no default CV exists.</li> <li>10. The System displays the updated CV collection; Matching results are viewed through UC-05.</li> </ol> |

| Field                           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Alternative Flows</b></p> | <p>A1 – Create or save a manual CV (from Step 3)</p> <ol style="list-style-type: none"> <li>1. The Candidate enters CV information manually.</li> <li>2. The System saves unfinished content as DRAFT or completed content as REVIEW_REQUIRED.</li> <li>3. A REVIEW_REQUIRED CV continues at Step 5.</li> </ol> <p>A2 – Update Profile or Portfolio (from Step 3)</p> <ol style="list-style-type: none"> <li>1. The Candidate changes editable Profile information or adds, updates, or removes a Portfolio item.</li> <li>2. The System validates, saves, and displays the change.</li> </ol> <p>A3 – Resume an unconfirmed CV (from Step 3)</p> <ol style="list-style-type: none"> <li>1. The Candidate opens an owned DRAFT or REVIEW_REQUIRED CV and updates it.</li> <li>2. Confirmation continues at Step 7; otherwise, the CV remains unconfirmed.</li> </ol> <p>A4 – Select a default CV (from Step 3)</p> <ol style="list-style-type: none"> <li>1. The Candidate selects an owned SCORING_DONE CV.</li> <li>2. The System makes it the only default CV and updates the collection.</li> </ol> <p>A5 – Delete an eligible CV (from Step 3)</p> <ol style="list-style-type: none"> <li>1. The Candidate selects an owned non-default CV and confirms deletion.</li> <li>2. The System verifies eligibility and removes the CV.</li> </ol> |

| Field           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Exception Flows | <p>E1 – File or extraction is invalid (from Step 4)</p> <ol style="list-style-type: none"> <li>1. The System rejects an unsupported file or marks the CV FAILED when extraction cannot complete.</li> <li>2. No invalid Matching result is presented.</li> </ol> <p>E2 – Confirmation or processing fails (from Step 8)</p> <ol style="list-style-type: none"> <li>1. The System displays validation issues or marks processing as FAILED.</li> <li>2. The CV remains unconfirmed or failed and may be corrected when allowed.</li> </ol> <p>E3 – CV is unavailable or unauthorized</p> <ol style="list-style-type: none"> <li>1. The System denies the operation and changes no CV information.</li> </ol> <p>E4 – Default or deletion request is ineligible (from A4 or A5)</p> <ol style="list-style-type: none"> <li>1. The System rejects a non-SCORING_DONE default request or deletion of a default or BANNED CV, or a CV with pending reports.</li> <li>2. Existing CV states remain unchanged.</li> </ol> |
| Postconditions  | <p>Success Postconditions</p> <p>S1. Valid Profile or Portfolio changes are stored.</p> <p>S2. A CV remains DRAFT/REVIEW_REQUIRED or reaches SCORING_DONE according to its progress.</p> <p>S3. The Candidate has at most one default CV.</p> <p>S4. A CV may reach SCORING_DONE even when no eligible Job produces a Matching result.</p> <p>Minimal Guarantees</p> <p>F1. Invalid operations do not replace valid career information.</p> <p>F2. The Candidate cannot change another Candidate's data or view invalid Matching results.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

| Field             | Description                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------|
| Related Use Cases | UC-03 – Manage Job Applications<br>UC-05 – Review Personalized Career Insights<br>UC-06 – Manage AutoFit |
| Priority          | High                                                                                                     |

### 1.5.2 Explore Jobs

Table 1.6. Use case - Explore Jobs

| Field              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID        | UC-02                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Use Case Name      | Explore Jobs                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Primary Actor(s)   | Candidate                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Secondary Actor(s) | Guest – alternative unauthenticated context                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Description        | This use case allows a Candidate or Guest to search and review publicly available Jobs; protected Candidate actions still require authentication.                                                                                                                                                                                                                                                                                                                                                                    |
| Preconditions      | P1. Public Job exploration requires no authentication.<br>P2. Protected actions require an active, authenticated Candidate account.                                                                                                                                                                                                                                                                                                                                                                                  |
| Trigger            | The Candidate or Guest opens Job exploration or submits supported criteria.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Main Flow          | <ol style="list-style-type: none"> <li>1. The Candidate opens Job exploration.</li> <li>2. The System displays public ACTIVE Jobs and supported search, filter, sorting, and pagination controls.</li> <li>3. The Candidate enters criteria.</li> <li>4. The System displays the matching Job summaries.</li> <li>5. The Candidate selects a Job.</li> <li>6. The System verifies public availability and displays Job and employer details.</li> <li>7. The System displays similar Jobs when available.</li> </ol> |



| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Alternative Flows</b> | <p>A1 – Guest explores Jobs (from Step 1)<br/> 1. A Guest searches, filters, and views public Job details.<br/> 2. The System requests authentication before a protected action.</p> <p>A2 – Change the result view (from Step 3 or Step 4)<br/> 1. The Candidate selects urgent Jobs, clears criteria, changes sorting, or changes page.<br/> 2. The System displays the corresponding ACTIVE Job set.</p> <p>A3 – No Jobs match (from Step 4)<br/> 1. The System displays an empty state and allows the criteria to be changed.</p> <p>A4 – Open a similar Job (from Step 7)<br/> 1. The Candidate selects a similar Job, and the flow returns to Step 6.</p> |
| <b>Exception Flows</b>   | <p>E1 – Selected Job is unavailable (from Step 6)<br/> 1. The System does not expose a missing or non-public Job and returns the user to available results.</p> <p>E2 – Protected action lacks authentication<br/> 1. The System does not perform the action and requests Candidate authentication.</p>                                                                                                                                                                                                                                                                                                                                                         |
| <b>Postconditions</b>    | <p>Success Postconditions<br/> S1. Current public Job information is displayed according to the selected criteria.<br/> S2. Exploration alone creates no Application.</p> <p>Minimal Guarantees<br/> F1. Non-public Job details and protected actions are not exposed without authorization.<br/> F2. Exploration changes no Job or Candidate data.</p>                                                                                                                                                                                                                                                                                                         |

| Field             | Description                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Related Use Cases | UC-03 – Manage Job Applications<br>UC-05 – Review Personalized Career Insights<br>UC-12 – Report Suspicious Recruitment Content |
| Priority          | High                                                                                                                            |

### 1.5.3 Manage Job Applications

Table 1.7. Use case - Manage Job Applications

| Field              | Description                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID        | UC-03                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Use Case Name      | Manage Job Applications                                                                                                                                                                                                                                                                                                                                                                                                         |
| Primary Actor(s)   | Candidate                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Secondary Actor(s) | None                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Description        | This use case allows a Candidate to apply for Jobs, review Application history, withdraw eligible Applications, and respond to Recruiter invitations.                                                                                                                                                                                                                                                                           |
| Preconditions      | P1. The Candidate account is active and authenticated.<br>P2. The Candidate may manage only their own Applications.<br>P3. A new Application requires an ACTIVE Job and an owned selected or default CV.                                                                                                                                                                                                                        |
| Trigger            | The Candidate applies for a Job or opens Application management.                                                                                                                                                                                                                                                                                                                                                                |
| Main Flow          | 1. The Candidate opens an ACTIVE Job and selects Apply.<br>2. The System verifies the Candidate and Job.<br>3. The System checks for an existing Candidate–Job Application.<br>4. The System identifies the selected owned CV or owned default CV.<br>5. The System creates a PENDING Application.<br>6. The System confirms submission.<br>7. The Application appears in Candidate history and the Recruiter's applicant list. |

| Field                           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Alternative Flows</b></p> | <p>A1 – Select the CV (from Step 4)</p> <ol style="list-style-type: none"> <li>1. The System uses an explicitly selected owned CV or, when none is supplied, the owned default CV.</li> <li>2. The flow continues at Step 5.</li> </ol> <p>A2 – Review Application history (alternative from the Trigger)</p> <ol style="list-style-type: none"> <li>1. The System displays owned Applications and statuses for filtering and review.</li> </ol> <p>A3 – Withdraw an eligible Application (from A2)</p> <ol style="list-style-type: none"> <li>1. The Candidate requests withdrawal of an Application not APPROVED or REJECTED.</li> <li>2. The System verifies eligibility and changes it to NOT_INTERESTED.</li> </ol> <p>A4 – Respond to an invitation (alternative from the Trigger)</p> <ol style="list-style-type: none"> <li>1. The Candidate selects an INVITED Application.</li> <li>2. The System changes it to PENDING when accepted or NOT_INTERESTED when declined.</li> </ol> |

| Field             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Exception Flows   | <p>E1 – Job is unavailable (from Step 2)<br/>1. The System rejects a missing or non-ACTIVE Job; no Application is created.</p> <p>E2 – Application already exists (from Step 3)<br/>1. The System rejects the duplicate request and keeps the existing Application.</p> <p>E3 – No eligible owned CV is available (from Step 4)<br/>1. The System rejects a missing, unauthorized, or unavailable CV; no Application is created.</p> <p>E4 – Withdrawal or invitation response is invalid (from A3 or A4)<br/>1. The Application is unavailable, unauthorized, or in an ineligible status.<br/>2. The System rejects the request and preserves its status.</p> |
| Postconditions    | <p>Success Postconditions<br/>S1. A submitted or accepted Application is PENDING and visible to both sides.<br/>S2. A withdrawn or declined Application is NOT_INTERESTED.</p> <p>Minimal Guarantees<br/>F1. Failed requests create no duplicate or unauthorized Application.<br/>F2. A Candidate cannot change another Candidate's Application.</p>                                                                                                                                                                                                                                                                                                           |
| Related Use Cases | <p>UC-01 – Manage Career Profile<br/>UC-02 – Explore Jobs<br/>UC-06 – Manage AutoFit<br/>UC-07 – Respond Through Actionable Email<br/>UC-09 – Review and Process Applicants</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Priority          | High                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

### 1.5.4 Provide Matching Feedback

Table 1.8. Use case - Provide Matching Feedback

| Field              | Description                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID        | UC-04                                                                                                                                                                                                                                                                                                                                                                                |
| Use Case Name      | Provide Matching Feedback                                                                                                                                                                                                                                                                                                                                                            |
| Primary Actor(s)   | Candidate                                                                                                                                                                                                                                                                                                                                                                            |
| Secondary Actor(s) | None                                                                                                                                                                                                                                                                                                                                                                                 |
| Description        | This use case allows a Candidate to evaluate an owned CV–Job Matching result so supported Feedback can influence later personalized rankings.                                                                                                                                                                                                                                        |
| Preconditions      | P1. The Candidate account is active and authenticated.<br>P2. The Matching belongs to a CV owned by the Candidate.                                                                                                                                                                                                                                                                   |
| Trigger            | The Candidate selects a Feedback option for a Matching result.                                                                                                                                                                                                                                                                                                                       |
| Main Flow          | 1. The Candidate opens a Matching result.<br>2. The System displays the Job, Matching information, and supported Feedback options.<br>3. The Candidate selects a Feedback type.<br>4. The System verifies ownership and validates the type.<br>5. The System records the Feedback and confirms receipt.<br>6. When applicable, the Feedback is used for later personalized rankings. |
| Alternative Flows  | A1 – Change previous Feedback (from Step 3)<br>1. The Candidate selects another supported type, and the System replaces the previous Feedback.<br><br>A2 – Mark the Job NOT_INTERESTED (from Step 3)<br>1. The System records the choice without using it as a preference-learning signal.                                                                                           |

| <b>Field</b>             | <b>Description</b>                                                                                                                                                                                                                                                                                                             |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Exception Flows</b>   | <p>E1 – Matching is unavailable or unauthorized (from Step 4)<br/> 1. The System rejects the request and changes no Feedback.</p> <p>E2 – Feedback type is unsupported (from Step 4)<br/> 1. The System rejects the value and preserves existing Feedback.</p>                                                                 |
| <b>Postconditions</b>    | <p>Success Postconditions<br/> S1. The Matching stores the Candidate's latest supported Feedback.<br/> S2. Eligible Feedback may influence later rankings;<br/> NOT_INTERESTED is stored without preference learning.</p> <p>Minimal Guarantees<br/> F1. Invalid or unauthorized requests do not change existing Feedback.</p> |
| <b>Related Use Cases</b> | <p>UC-05 – Review Personalized Career Insights<br/> UC-07 – Respond Through Actionable Email</p>                                                                                                                                                                                                                               |
| <b>Priority</b>          | High                                                                                                                                                                                                                                                                                                                           |

### 1.5.5 Review Personalized Career Insights

Table 1.9. Use case - Review Personalized Career Insights

| Field              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID        | UC-05                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Use Case Name      | Review Personalized Career Insights                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Primary Actor(s)   | Candidate                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Secondary Actor(s) | None                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Description        | This use case allows a Candidate to review CV–Job Matching results, independent profile/preference-based Job recommendations, and available career analytics.                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Preconditions      | P1. The Candidate account is active and authenticated.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Trigger            | The Candidate opens personalized career insights and selects a view.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Main Flow          | <ol style="list-style-type: none"> <li>1. The Candidate opens personalized career insights.</li> <li>2. The System displays the available Matching, recommendation, and analytics views.</li> <li>3. The Candidate opens CV–Job Matching results.</li> <li>4. The System displays available results for the Candidate's eligible CVs.</li> <li>5. The Candidate opens Job recommendations.</li> <li>6. The System independently displays recommendations based on supported Candidate profile or preference information.</li> <li>7. The Candidate reviews the available insights.</li> </ol> |
| Alternative Flows  | <p>A1 – Review Candidate analytics (from Step 2)</p> <ol style="list-style-type: none"> <li>1. The Candidate selects analytics, and the System displays available summaries and trends.</li> </ol> <p>A2 – Limited data in a selected view (from Step 4, Step 6, or A1)</p> <ol style="list-style-type: none"> <li>1. The System displays available information and an empty or limited-data state for the missing section.</li> <li>2. Other insight views remain available.</li> </ol>                                                                                                      |
| Exception Flows    | <p>E1 – Matching information is unavailable or unauthorized (from Step 4)</p> <ol style="list-style-type: none"> <li>1. The System withholds the unavailable result and changes no Candidate data.</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                 |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Postconditions</b>    | <p>Success Postconditions</p> <p>S1. Available Matching, recommendation, or analytics information is displayed in its corresponding view.</p> <p>S2. Reviewing insights changes no Application or Job state.</p> <p>Minimal Guarantees</p> <p>F1. Missing data in one view does not incorrectly hide other available insights.</p> <p>F2. Unauthorized Matching information is not exposed.</p> |
| <b>Related Use Cases</b> | <p>UC-01 – Manage Career Profile</p> <p>UC-02 – Explore Jobs</p> <p>UC-04 – Provide Matching Feedback</p> <p>UC-06 – Manage AutoFit</p>                                                                                                                                                                                                                                                         |
| <b>Priority</b>          | High                                                                                                                                                                                                                                                                                                                                                                                            |

### 1.5.6 Manage AutoFit

Table 1.10. Use case - Manage AutoFit

| Field                     | Description                                                                                                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-06                                                                                                                                                        |
| <b>Use Case Name</b>      | Manage AutoFit                                                                                                                                               |
| <b>Primary Actor(s)</b>   | Candidate                                                                                                                                                    |
| <b>Secondary Actor(s)</b> | None                                                                                                                                                         |
| <b>Description</b>        | This use case allows a Candidate to configure AutoFit and request a controlled run; the System may also execute it automatically on its configured schedule. |
| <b>Preconditions</b>      | P1. The Candidate account is active and authenticated.                                                                                                       |
| <b>Trigger</b>            | The Candidate changes AutoFit configuration or requests a run; alternatively, the system-configured AutoFit schedule initiates execution.                    |



| Field            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Main Flow</b> | <ol style="list-style-type: none"> <li>1. The Candidate opens AutoFit settings.</li> <li>2. The System displays enablement, pause state, and threshold.</li> <li>3. The Candidate changes the configuration or requests a manual run.</li> <li>4. The System validates and saves a threshold from 50 to 100.</li> <li>5. For execution, the System verifies an owned default CV in SCORING_DONE status.</li> <li>6. The System selects Matchings for ACTIVE Jobs at or above the threshold.</li> <li>7. The System skips existing Candidate–Job Applications.</li> <li>8. The System creates at most three AUTO_APPLIED Applications and displays the run summary.</li> </ol> |

| Field                           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Alternative Flows</b></p> | <p>A1 – Automatic execution (alternative from the Trigger)<br/> 1. The configured schedule initiates execution for an enabled, unpaused policy.<br/> 2. The flow continues at Step 5 without treating the internal scheduler as an actor.</p> <p>A2 – Pause or resume AutoFit (from Step 3)<br/> 1. The Candidate changes the pause state, and the System saves it.</p> <p>A3 – No eligible Matching (from Step 6)<br/> 1. The System creates no Application and displays a zero-result summary.</p> <p>A4 – Existing Application is skipped (from Step 7)<br/> 1. The System skips the duplicate and continues with other eligible Matchings.</p> <p>A5 – No eligible default CV (from Step 5)<br/> 1. The System creates no Application and informs the Candidate.</p> <p>A6 – Save configuration without running AutoFit (from Step 4)<br/> 1. The Candidate does not request a manual run.<br/> 2. The System retains the configuration for future manual or automatic execution, and the Use Case ends.</p> |
| <p><b>Exception Flows</b></p>   | <p>E1 – Threshold is outside 50–100 (from Step 4)<br/> 1. The System rejects the value and preserves the previous valid configuration.</p> <p>E2 – Policy is unavailable or unauthorized<br/> 1. The System denies the operation and creates no Application.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Postconditions</b>    | <p>Success Postconditions</p> <p>S1. Valid AutoFit configuration is stored.</p> <p>S2. An execution creates zero to three non-duplicate AUTO_APPLIED Applications for eligible ACTIVE Jobs.</p> <p>S3. Notification quota, cooldown, quiet hours, and preferences are not AutoApply eligibility rules.</p> <p>Minimal Guarantees</p> <p>F1. Invalid configuration does not replace the last valid configuration.</p> <p>F2. Missing eligibility creates no Application.</p> |
| <b>Related Use Cases</b> | <p>UC-01 – Manage Career Profile</p> <p>UC-03 – Manage Job Applications</p> <p>UC-05 – Review Personalized Career Insights</p>                                                                                                                                                                                                                                                                                                                                              |
| <b>Priority</b>          | High                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

### 1.5.7 Respond Through Actionable Email

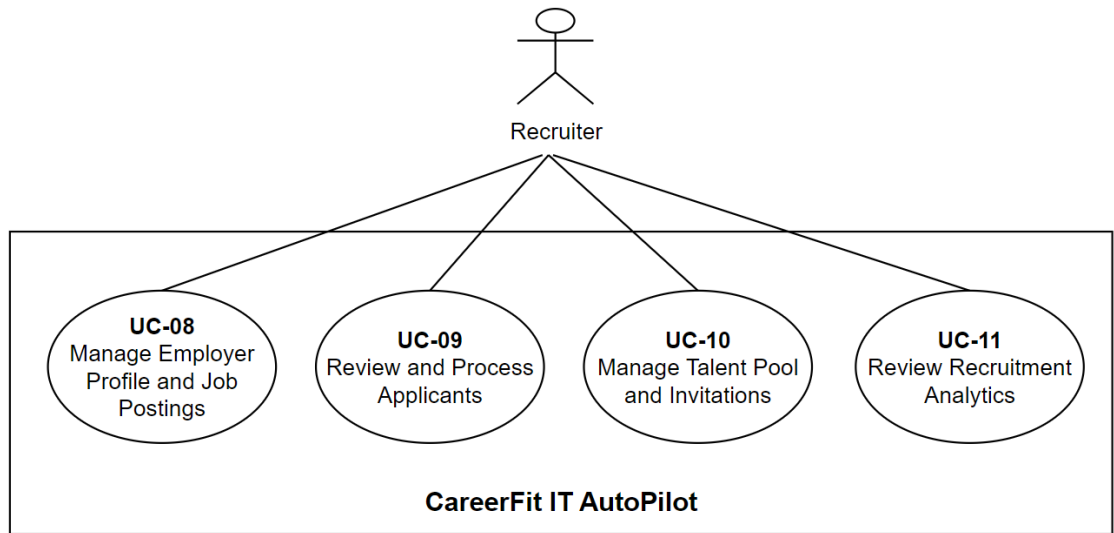
*Table 1.11. Use case - Respond Through Actionable Email*

| Field                     | Description                                                                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-07                                                                                                                                              |
| <b>Use Case Name</b>      | Respond Through Actionable Email                                                                                                                   |
| <b>Primary Actor(s)</b>   | Candidate                                                                                                                                          |
| <b>Secondary Actor(s)</b> | None                                                                                                                                               |
| <b>Description</b>        | This use case allows a Candidate to confirm and complete supported Matching Feedback or Recruiter-invitation actions from an actionable email.     |
| <b>Preconditions</b>      | <p>P1. The Candidate has received a CareerFit actionable email.</p> <p>P2. The email contains a link associated with a CareerFit email action.</p> |
| <b>Trigger</b>            | The Candidate opens the actionable link.                                                                                                           |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Main Flow</b>         | <ol style="list-style-type: none"> <li>1. The Candidate opens the link.</li> <li>2. The System identifies the email action and verifies its current state and expiry.</li> <li>3. The System displays a non-mutating confirmation page.</li> <li>4. The Candidate reviews and confirms the action.</li> <li>5. The System validates the action and referenced record again.</li> <li>6. The System performs the supported action.</li> <li>7. The System marks the email action REDEEMED.</li> <li>8. The System displays the result.</li> </ol>                                                                                   |
| <b>Alternative Flows</b> | <p>A1 – Submit Matching Feedback (from Step 6)</p> <ol style="list-style-type: none"> <li>1. The System submits the corresponding GOOD_MATCH, POTENTIAL, BAD_MATCH, or NOT_INTERESTED Feedback.</li> </ol> <p>A2 – Respond to a Recruiter invitation (from Step 6)</p> <ol style="list-style-type: none"> <li>1. For an INVITED Application, acceptance changes it to PENDING and decline changes it to NOT_INTERESTED.</li> </ol> <p>A3 – Leave without confirming (from Step 4)</p> <ol style="list-style-type: none"> <li>1. No business action occurs; the email action remains pending until redemption or expiry.</li> </ol> |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Exception Flows</b>   | <p>E1 – Action link is invalid (from Step 2)<br/> 1. The System displays an invalid-link message and changes no business data.</p> <p>E2 – Action was already redeemed (from Step 2 or Step 5)<br/> 1. The System informs the Candidate that it was processed previously and does not execute it again.</p> <p>E3 – Action has expired (from Step 2 or Step 5)<br/> 1. The System rejects the action without changing the referenced state.<br/> 2. When expiry is detected during confirmation, the System records EXPIRED.</p> <p>E4 – Referenced Matching or invitation is unavailable (from Step 5)<br/> 1. The System rejects the action and preserves existing Feedback or Application state.</p> |
| <b>Postconditions</b>    | <p>Success Postconditions<br/> S1. The supported action is applied to the referenced Matching or Application.<br/> S2. The email action is REDEEMED and cannot be executed again.</p> <p>Minimal Guarantees<br/> F1. Opening confirmation changes no business data.<br/> F2. Invalid, expired, redeemed, or failed actions do not change the referenced state.</p>                                                                                                                                                                                                                                                                                                                                      |
| <b>Related Use Cases</b> | UC-03 – Manage Job Applications<br>UC-04 – Provide Matching Feedback                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Priority</b>          | Medium                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Other lifecycle emails are treated as notifications rather than separate use cases because they inform the recipient without adding another token-based user goal or a separate state-changing workflow.



*Figure 1.4. Recruiter use-case diagram*

### 1.5.8 Manage Employer Profile and Job Postings

*Table 1.12. Use case - Manage Employer Profile and Job Postings*

| Field              | Description                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID        | UC-08                                                                                                                         |
| Use Case Name      | Manage Employer Profile and Job Postings                                                                                      |
| Primary Actor(s)   | Recruiter                                                                                                                     |
| Secondary Actor(s) | None                                                                                                                          |
| Description        | This use case allows a Recruiter to maintain Employer Profile information and manage the lifecycle of owned Job postings.     |
| Preconditions      | P1. The Recruiter account is active and authenticated.<br>P2. Existing-Job actions apply only to Jobs owned by the Recruiter. |
| Trigger            | The Recruiter opens Employer Profile or Job management.                                                                       |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Main Flow</b>         | <ol style="list-style-type: none"> <li>1. The Recruiter opens Job management.</li> <li>2. The System displays owned Jobs and available actions.</li> <li>3. The Recruiter selects Create Job.</li> <li>4. The System displays the Job form.</li> <li>5. The Recruiter enters Job information and submits it for publication.</li> <li>6. The System validates the information and Recruiter ownership.</li> <li>7. The System creates the Job as ACTIVE.</li> <li>8. The System displays the published Job in Job management.</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Alternative Flows</b> | <p>A1 – Update Employer Profile (alternative from the Trigger)</p> <ol style="list-style-type: none"> <li>1. The Recruiter changes editable employer information, and the System validates and saves it.</li> </ol> <p>A2 – Save as draft (from Step 5)</p> <ol style="list-style-type: none"> <li>1. The Recruiter saves the Job without publication, and the System stores it as DRAFT.</li> </ol> <p>A3 – Publish an existing draft (from Step 2)</p> <ol style="list-style-type: none"> <li>1. The Recruiter selects an owned DRAFT Job, completes valid information, and publishes it as ACTIVE.</li> </ol> <p>A4 – Update or close an existing Job (from Step 2)</p> <ol style="list-style-type: none"> <li>1. The Recruiter edits an owned Job or closes it through an allowed transition.</li> <li>2. The System validates, saves, and displays the result.</li> </ol> <p>A5 – Delete an eligible Job (from Step 2)</p> <ol style="list-style-type: none"> <li>1. The Recruiter confirms deletion of an owned Job with no blocking Applications or pending reports.</li> <li>2. The System deletes it and updates the list.</li> </ol> |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Exception Flows</b>   | <p>E1 – Job is unavailable or unauthorized<br/>1. The System denies access and changes no Job.</p> <p>E2 – Job information is invalid (from Step 6 or A3)<br/>1. The System displays validation issues and does not publish the Job.</p> <p>E3 – Job action is not allowed (from A4 or A5)<br/>1. The System rejects an invalid transition or deletion blocked by Applications or pending reports.<br/>2. The Job remains unchanged.</p> |
| <b>Postconditions</b>    | <p>Success Postconditions<br/>S1. Valid Employer Profile changes are stored.<br/>S2. A published Job is ACTIVE; a draft or closed Job retains its selected valid state.<br/>S3. An eligible deleted Job is unavailable.</p> <p>Minimal Guarantees<br/>F1. A Recruiter cannot modify another Recruiter's Job.<br/>F2. Invalid or blocked actions do not publish, transition, or delete the Job.</p>                                       |
| <b>Related Use Cases</b> | <p>UC-02 – Explore Jobs<br/>UC-09 – Review and Process Applicants<br/>UC-10 – Manage Talent Pool and Invitations</p>                                                                                                                                                                                                                                                                                                                     |
| <b>Priority</b>          | High                                                                                                                                                                                                                                                                                                                                                                                                                                     |

### 1.5.9 Review and Process Applicants

*Table 1.13. Use case - Review and Process Applicants*

| Field                     | Description                                                                                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-09                                                                                                                 |
| <b>Use Case Name</b>      | Review and Process Applicants                                                                                         |
| <b>Primary Actor(s)</b>   | Recruiter                                                                                                             |
| <b>Secondary Actor(s)</b> | None                                                                                                                  |
| <b>Description</b>        | This use case allows a Recruiter to review Applications for an owned Job and record a supported recruitment decision. |
| <b>Preconditions</b>      | <p>P1. The Recruiter account is active and authenticated.<br/>P2. The selected Job belongs to the Recruiter.</p>      |



| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trigger</b>           | The Recruiter opens the applicant workspace for an owned Job.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Main Flow</b>         | <ol style="list-style-type: none"> <li>1. The Recruiter selects an owned Job.</li> <li>2. The System displays its Applications and supported filters.</li> <li>3. The Recruiter selects an applicant.</li> <li>4. The System displays available Candidate, CV, Portfolio, Application, Matching, and Potential information.</li> <li>5. The Recruiter reviews the information and selects Approve or Reject.</li> <li>6. The System verifies ownership, CV status, and the requested decision.</li> <li>7. The System updates the Application and displays the result.</li> </ol> |
| <b>Alternative Flows</b> | <p>A1 – Review without a decision (from Step 5)</p> <ol style="list-style-type: none"> <li>1. The Recruiter closes the detail, and the Application remains unchanged.</li> </ol> <p>A2 – No applicants match (from Step 2)</p> <ol style="list-style-type: none"> <li>1. The System displays an empty state and allows filters or the selected Job to change.</li> </ol>                                                                                                                                                                                                          |
| <b>Exception Flows</b>   | <p>E1 – Job or Application is unavailable or unauthorized</p> <ol style="list-style-type: none"> <li>1. The System denies access or rejects the decision without changing status.</li> </ol> <p>E2 – Associated CV is BANNED (from Step 6)</p> <ol style="list-style-type: none"> <li>1. The System rejects the decision and preserves the Application status.</li> </ol> <p>E3 – Requested status is unsupported (from Step 6)</p> <ol style="list-style-type: none"> <li>1. The System rejects the update and preserves the existing status.</li> </ol>                         |

| Field                    | Description                                                                                                                                                                                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Postconditions</b>    | <p>Success Postconditions</p> <p>S1. The Application reflects the supported Recruiter decision and is visible to the Candidate.</p> <p>Minimal Guarantees</p> <p>F1. Review alone changes nothing.</p> <p>F2. Unauthorized, BANNED-CV, or unsupported decisions do not change status.</p> |
| <b>Related Use Cases</b> | <p>UC-03 – Manage Job Applications</p> <p>UC-08 – Manage Employer Profile and Job Postings</p> <p>UC-10 – Manage Talent Pool and Invitations</p> <p>UC-12 – Report Suspicious Recruitment Content</p>                                                                                     |
| <b>Priority</b>          | High                                                                                                                                                                                                                                                                                      |

#### 1.5.10 Manage Talent Pool and Invitations

*Table 1.14. Use case - Manage Talent Pool and Invitations*

| Field                     | Description                                                                                                                                  |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-10                                                                                                                                        |
| <b>Use Case Name</b>      | Manage Talent Pool and Invitations                                                                                                           |
| <b>Primary Actor(s)</b>   | Recruiter                                                                                                                                    |
| <b>Secondary Actor(s)</b> | None                                                                                                                                         |
| <b>Description</b>        | This use case allows a Recruiter to discover suitable Candidates for an owned Job, manage Job-specific CV bookmarks, and manage invitations. |
| <b>Preconditions</b>      | <p>P1. The Recruiter account is active and authenticated.</p> <p>P2. The selected Job belongs to the Recruiter.</p>                          |
| <b>Trigger</b>            | The Recruiter opens the Talent Pool for an owned Job.                                                                                        |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Main Flow</b>         | <ol style="list-style-type: none"> <li>1. The Recruiter selects an owned Job.</li> <li>2. The System displays Candidates with available Matchings, separated into High and Potential groups.</li> <li>3. The Recruiter filters and selects a Candidate.</li> <li>4. The System displays the available CV, score, label, and explanation.</li> <li>5. The Recruiter selects Invite.</li> <li>6. The System verifies the ACTIVE Job, Candidate default CV, and existing Application state.</li> <li>7. The System creates an INVITED Application.</li> <li>8. The System displays the Candidate in the invited group.</li> </ol>                                                                                                                                                        |
| <b>Alternative Flows</b> | <p>A1 – Add or remove a bookmark (from Step 4)</p> <ol style="list-style-type: none"> <li>1. The System updates the Job-specific bookmark and bookmark view.</li> </ol> <p>A2 – Withdraw a pending invitation (from Step 8)</p> <ol style="list-style-type: none"> <li>1. The Recruiter confirms withdrawal of an INVITED Application.</li> <li>2. The System removes it and updates the invited group.</li> </ol> <p>A3 – An Application already exists (from Step 6)</p> <ol style="list-style-type: none"> <li>1. The System displays the existing state instead of creating a duplicate.</li> </ol> <p>A4 – Talent Pool is empty (from Step 2)</p> <ol style="list-style-type: none"> <li>1. The System displays an empty state; no bookmark or invitation is created.</li> </ol> |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Exception Flows</b>   | <p>E1 – Job is unavailable, unauthorized, or not ACTIVE (from Step 6)<br/>1. The System rejects the invitation and creates nothing.</p> <p>E2 – Candidate has no default CV (from Step 6)<br/>1. The System rejects the invitation and creates nothing.</p> <p>E3 – Invitation cannot be withdrawn (from A2)<br/>1. The Application is unavailable, unauthorized, or no longer INVITED.<br/>2. The System rejects withdrawal and preserves it.</p> |
| <b>Postconditions</b>    | <p>Success Postconditions<br/>S1. Bookmarks reflect the Recruiter's Job-specific selections.<br/>S2. A new invitation is INVITED; a withdrawn pending invitation is removed.<br/>S3. Existing Applications are not duplicated.</p> <p>Minimal Guarantees<br/>F1. Invalid Job, CV, ownership, or withdrawal state changes nothing.</p>                                                                                                              |
| <b>Related Use Cases</b> | <p>UC-03 – Manage Job Applications<br/>UC-08 – Manage Employer Profile and Job Postings<br/>UC-09 – Review and Process Applicants<br/>UC-12 – Report Suspicious Recruitment Content</p>                                                                                                                                                                                                                                                            |
| <b>Priority</b>          | Medium                                                                                                                                                                                                                                                                                                                                                                                                                                             |

### 1.5.11 Review Recruitment Analytics

Table 1.15. Use case - Review Recruitment Analytics

| Field                     | Description                                                                                                                      |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-11                                                                                                                            |
| <b>Use Case Name</b>      | Review Recruitment Analytics                                                                                                     |
| <b>Primary Actor(s)</b>   | Recruiter                                                                                                                        |
| <b>Secondary Actor(s)</b> | None                                                                                                                             |
| <b>Description</b>        | This use case allows a Recruiter to review available analytics for owned Jobs, Applications, recruitment trends, and skill gaps. |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Preconditions</b>     | P1. The Recruiter account is active and authenticated.<br>P2. Job-specific analytics apply only to an owned Job.                                                                                                                                                                                                                                                                                                                 |
| <b>Trigger</b>           | The Recruiter opens recruitment analytics.                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Main Flow</b>         | <ol style="list-style-type: none"> <li>1. The Recruiter opens recruitment analytics and selects a period.</li> <li>2. The System displays Recruiter-level Job and Application statistics.</li> <li>3. The Recruiter selects an owned Job.</li> <li>4. The System verifies ownership.</li> <li>5. The System displays the Job funnel, skill gaps, and available trends.</li> <li>6. The Recruiter reviews the results.</li> </ol> |
| <b>Alternative Flows</b> | <p>A1 – Review aggregate analytics (from Step 3)</p> <ol style="list-style-type: none"> <li>1. No Job is selected, so the System continues to display Recruiter-level analytics.</li> </ol> <p>A2 – Limited analytics data (from Step 5)</p> <ol style="list-style-type: none"> <li>1. The System displays available values and a limited-data state for missing sections.</li> </ol>                                            |
| <b>Exception Flows</b>   | <p>E1 – Job is unavailable or unauthorized (from Step 4)</p> <ol style="list-style-type: none"> <li>1. The System denies Job-specific analytics while keeping the Recruiter overview available.</li> </ol>                                                                                                                                                                                                                       |
| <b>Postconditions</b>    | <p>Success Postconditions</p> <p>S1. Available aggregate or Job-specific analytics are displayed.</p> <p>Minimal Guarantees</p> <p>F1. Another Recruiter's analytics are not exposed, and review changes no operational data.</p>                                                                                                                                                                                                |
| <b>Related Use Cases</b> | <p>UC-08 – Manage Employer Profile and Job Postings</p> <p>UC-09 – Review and Process Applicants</p> <p>UC-10 – Manage Talent Pool and Invitations</p>                                                                                                                                                                                                                                                                           |
| <b>Priority</b>          | Medium                                                                                                                                                                                                                                                                                                                                                                                                                           |

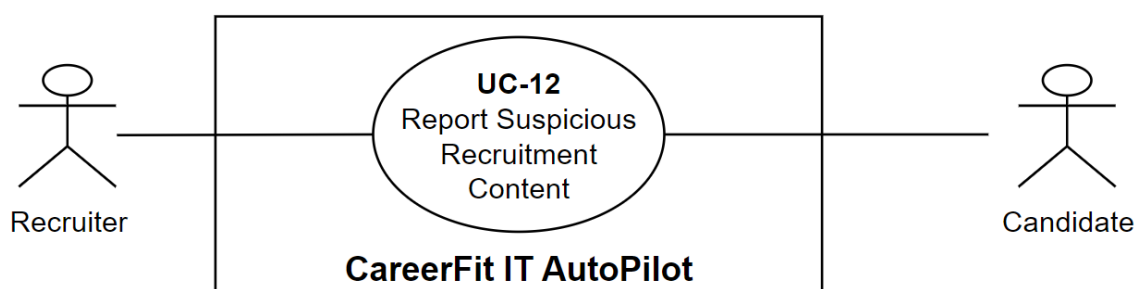


Figure 1.5. Shared Candidate-Recruiter reporting use-case diagram

### 1.5.12 Report Suspicious Recruitment Content

Table 1.16. Use case - Report Suspicious Recruitment Content

| Field              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use Case ID        | UC-12                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Use Case Name      | Report Suspicious Recruitment Content                                                                                                                                                                                                                                                                                                                                                                                                       |
| Primary Actor(s)   | Candidate, Recruiter                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Secondary Actor(s) | None                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Description        | This use case allows a Candidate to report a suspicious Job and a Recruiter to report a suspicious visible CV for Administrator review.                                                                                                                                                                                                                                                                                                     |
| Preconditions      | P1. The reporting account is active and authenticated.<br>P2. A Candidate reports an ACTIVE Job.<br>P3. A Recruiter reports a CV visible through an Application or Matching for an owned Job.                                                                                                                                                                                                                                               |
| Trigger            | A Candidate or Recruiter selects Report for eligible recruitment content.                                                                                                                                                                                                                                                                                                                                                                   |
| Main Flow          | <ol style="list-style-type: none"> <li>1. The Candidate opens an ACTIVE Job and selects Report.</li> <li>2. The System displays supported reasons and a comment field.</li> <li>3. The Candidate selects a reason and submits.</li> <li>4. The System verifies actor, target, reason, and duplicate state.</li> <li>5. The System creates a PENDING report.</li> <li>6. The System confirms submission for Administrator review.</li> </ol> |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Alternative Flows</b> | <p>A1 – Recruiter reports a CV (alternative from the Trigger)</p> <ol style="list-style-type: none"> <li>1. The Recruiter opens a CV visible through an owned Job and submits a reason, comment, and Job.</li> <li>2. The System verifies ownership and visibility, creates a PENDING report, and confirms submission.</li> </ol>                                                                                                                                                                                                                                                                               |
| <b>Exception Flows</b>   | <p>E1 – Target is unavailable or ineligible (from Step 4 or A1)</p> <ol style="list-style-type: none"> <li>1. The System rejects a missing/non-ACTIVE Job or a CV outside the Recruiter's visible owned-Job context.</li> <li>2. No report is created.</li> </ol> <p>E2 – A pending report already exists (from Step 4 or A1)</p> <ol style="list-style-type: none"> <li>1. The System rejects the duplicate and preserves the existing pending report.</li> </ol> <p>E3 – Reason is unsupported</p> <ol style="list-style-type: none"> <li>1. The System rejects the request and creates no report.</li> </ol> |
| <b>Postconditions</b>    | <p>Success Postconditions</p> <p>S1. A valid report is PENDING and available to Administrator moderation.</p> <p>S2. The target's pending-report count reflects the report.</p> <p>Minimal Guarantees</p> <p>F1. Invalid, unauthorized, or duplicate requests create no additional report.</p>                                                                                                                                                                                                                                                                                                                  |
| <b>Related Use Cases</b> | <p>UC-02 – Explore Jobs</p> <p>UC-09 – Review and Process Applicants</p> <p>UC-10 – Manage Talent Pool and Invitations</p> <p>UC-14 – Review and Resolve Content Reports</p>                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Priority</b>          | Medium                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

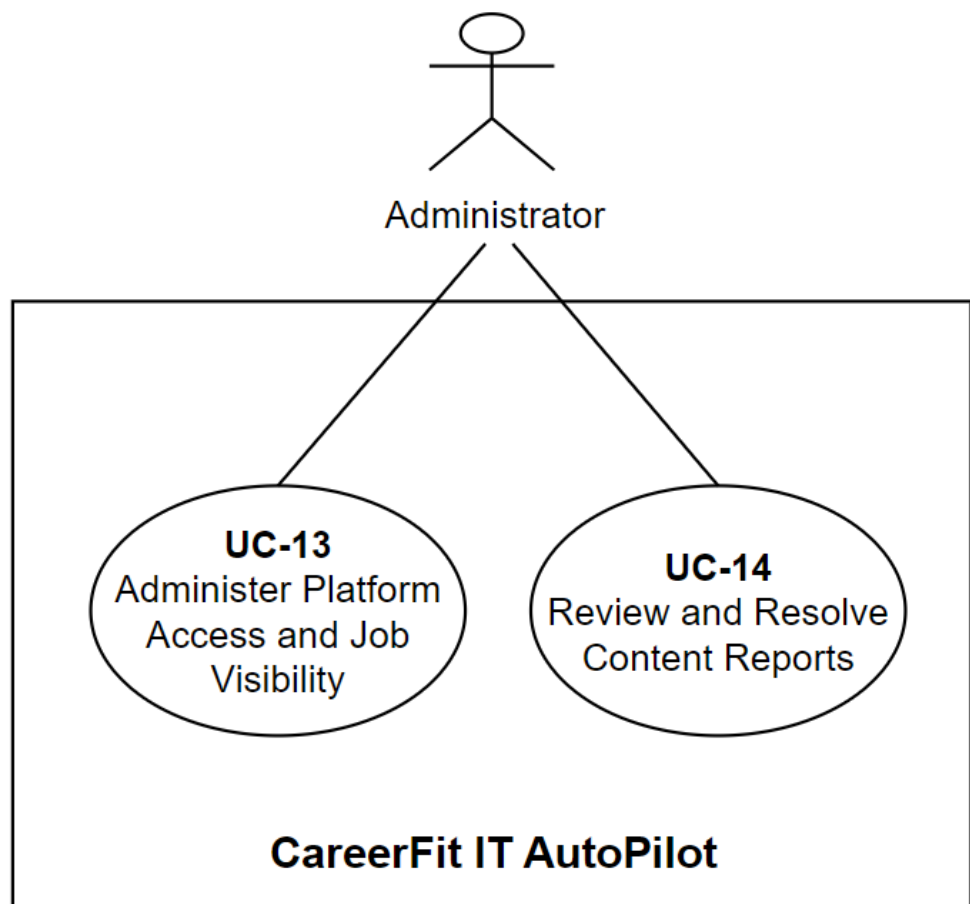


Figure 1.6. Administrator use-case diagram

### 1.5.13 Administer Platform Access and Job Visibility

Table 1.17. Use case - Administer Platform Access and Job Visibility

| Field              | Description                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------|
| Use Case ID        | UC-13                                                                                           |
| Use Case Name      | Administer Platform Access and Job Visibility                                                   |
| Primary Actor(s)   | Administrator                                                                                   |
| Secondary Actor(s) | None                                                                                            |
| Description        | This use case allows an Administrator to control User-account access and public Job visibility. |
| Preconditions      | P1. The Administrator account is active and authenticated with the Administrator role.          |
| Trigger            | The Administrator opens User or Job administration and selects an action.                       |



| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Main Flow</b>         | <ol style="list-style-type: none"> <li>1. The Administrator opens User administration and selects a User.</li> <li>2. The System displays account information and active state.</li> <li>3. The Administrator selects Suspend.</li> <li>4. The System verifies that the target is not the current Administrator.</li> <li>5. The System sets the account inactive.</li> <li>6. The System confirms suspension.</li> </ol>                                                                                                                                                                                                                                                                                                            |
| <b>Alternative Flows</b> | <p>A1 – Activate a User (from Step 3)</p> <ol style="list-style-type: none"> <li>1. The Administrator selects an inactive User, and the System activates the account.</li> </ol> <p>A2 – Hide an ACTIVE Job (alternative from the Trigger)</p> <ol style="list-style-type: none"> <li>1. The Administrator selects an ACTIVE Job, and the System changes it to HIDDEN_BY_ADMIN.</li> </ol> <p>A3 – Restore a hidden Job (from A2)</p> <ol style="list-style-type: none"> <li>1. The Administrator selects a HIDDEN_BY_ADMIN Job, and the System restores it to ACTIVE.</li> </ol> <p>A4 – No records match</p> <ol style="list-style-type: none"> <li>1. The System displays an empty state and allows filters to change.</li> </ol> |
| <b>Exception Flows</b>   | <p>E1 – Administrator attempts self-suspension (from Step 4)</p> <ol style="list-style-type: none"> <li>1. The System rejects the request; the Administrator remains active.</li> </ol> <p>E2 – Target is unavailable</p> <ol style="list-style-type: none"> <li>1. The System rejects the action and changes no state.</li> </ol> <p>E3 – Job transition is not allowed (from A2 or A3)</p> <ol style="list-style-type: none"> <li>1. The System rejects Hide for a non-ACTIVE Job or Restore for a Job not HIDDEN_BY_ADMIN.</li> <li>2. The Job remains unchanged.</li> </ol>                                                                                                                                                      |

| Field                    | Description                                                                                                                                                                                                                                                                                                                |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Postconditions</b>    | <p>Success Postconditions</p> <p>S1. A suspended User is inactive; an activated User is active.</p> <p>S2. A hidden Job is HIDDEN_BY_ADMIN; a restored Job is ACTIVE.</p> <p>Minimal Guarantees</p> <p>F1. The Administrator cannot suspend their own account.</p> <p>F2. Invalid requests do not change target state.</p> |
| <b>Related Use Cases</b> | <p>UC-08 – Manage Employer Profile and Job Postings</p> <p>UC-14 – Review and Resolve Content Reports</p>                                                                                                                                                                                                                  |
| <b>Priority</b>          | Medium                                                                                                                                                                                                                                                                                                                     |

#### 1.5.14 Review and Resolve Content Reports

Table 1.18. Use case - Review and Resolve Content Reports

| Field                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>        | UC-14                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Use Case Name</b>      | Review and Resolve Content Reports                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Primary Actor(s)</b>   | Administrator                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Secondary Actor(s)</b> | None                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Description</b>        | This use case allows an Administrator to review reports grouped by target and resolve them by dismissal or by banning the reported Job or CV.                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Preconditions</b>      | P1. The Administrator account is active and authenticated with the Administrator role.                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Trigger</b>            | The Administrator opens the content-report queue.                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Main Flow</b>          | <ol style="list-style-type: none"> <li>1. The System displays pending reports grouped by Job or CV target.</li> <li>2. The Administrator selects a case.</li> <li>3. The System displays the target, reasons, comments, count, and status.</li> <li>4. The Administrator reviews the evidence and selects Dismiss.</li> <li>5. The System verifies that pending reports remain.</li> <li>6. The System marks them DISMISSED and clears the target count.</li> <li>7. The System displays the resolved case.</li> </ol> |

| Field                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Alternative Flows</b> | <p>A1 – Ban the reported target (from Step 4)</p> <ol style="list-style-type: none"> <li>1. The Administrator selects Ban and may add a note.</li> <li>2. The System sets the Job or CV to BANNED; for a CV, it removes the default designation.</li> <li>3. The System marks pending reports ACTIONED, clears the count, and displays the case.</li> </ol> <p>A2 – Report queue is empty (from Step 1)</p> <ol style="list-style-type: none"> <li>1. The System displays an empty queue; no moderation action occurs.</li> </ol> |
| <b>Exception Flows</b>   | <p>E1 – Report target is unavailable (from Step 3)</p> <ol style="list-style-type: none"> <li>1. The System cannot resolve the case and records no decision.</li> </ol> <p>E2 – No pending reports remain (from Step 5)</p> <ol style="list-style-type: none"> <li>1. The System rejects repeated resolution and preserves existing states.</li> </ol>                                                                                                                                                                            |
| <b>Postconditions</b>    | <p>Success Postconditions</p> <p>S1. Dismissed reports are DISMISSED; reports for a banned target are ACTIONED.</p> <p>S2. A banned target is BANNED and has no pending-report count.</p> <p>Minimal Guarantees</p> <p>F1. Review alone changes no target state.</p> <p>F2. Failed or repeated resolution does not partially change the case.</p>                                                                                                                                                                                 |
| <b>Related Use Cases</b> | <p>UC-12 – Report Suspicious Recruitment Content</p> <p>UC-13 – Administer Platform Access and Job Visibility</p>                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Priority</b>          | Medium                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

## 1.6 Chapter Summary

This chapter defined CareerFit's actors, functional and non-functional requirements, and fourteen approved role-oriented use cases. The specifications distinguish actor goals from internal processing and keep CV–Job Matching, Job Recommendation, Applications, AutoFit eligibility, notification delivery, reporting, moderation, and actionable email behavior traceable to verified implementation.

## **CHAPTER 2. THEORETICAL BACKGROUND**

### **2.1 Recruitment Information and CV–Job Description Matching**

Recruitment matching is an information-retrieval and decision-support task that combines structured conditions with inconsistent free text. Exact filters can enforce location, experience, or employment conditions, but they cannot by themselves rank overall fit between a CV and a Job.

The same information supports different goals: Candidates rank Jobs, while Recruiters review applicants or discover Potential Candidates. These workflows may share representations and similarity functions, but a Recommendation is not an Application and a high score is not a hiring decision. Scores only support prioritization.

Technical terms, titles, and skill families vary across companies and seniority levels, while public CV corpora are limited by privacy. The Djinni Recruitment Dataset shows the scale and value of anonymized recruitment data [6]. CareerFit uses a narrower IT scope, explicit fields, and normalized text so its behavior can be inspected.

### **2.2 Text Representation and Preprocessing**

#### **2.2.1 Document Normalization**

Before vectorization, text is extracted and normalized into deterministic tokens [2]. Recruitment preprocessing must preserve technical names, versions, and abbreviations because excessive filtering can remove important evidence.

Vietnamese diacritics and multi-syllable expressions coexist with English technical terms. Structured attributes such as location, language, seniority, and salary mode remain available for validation or filtering instead of being forced into one text vector.

Missing OCR text, malformed files, very short descriptions, and duplicated boilerplate can distort scores. CareerFit therefore validates content before scoring: blocking errors stop unusable input, while warnings allow review and correction.

### 2.2.2 Bag-of-Words Representation

In a bag-of-words representation, a document is represented by the terms it contains and their weights; word order is not modeled directly. If the vocabulary contains  $|V|$  terms, each document is mapped to a vector in  $\mathbb{R}^{|V|}$ . This representation is sparse because a particular CV or JD contains only a small portion of the complete vocabulary. Its main advantages are computational simplicity, reproducibility, and the ability to inspect which terms contribute to similarity. Its main limitation is lexical dependence: synonyms and related skills do not match unless normalization or an explicit mapping connects them.

## 2.3 TF-IDF and Cosine Similarity

### 2.3.1 Term Frequency and Inverse Document Frequency

The vector-space model represents documents and queries as weighted term vectors and ranks documents according to their geometric relationship [1]. TF-IDF is a standard weighting family for this model [2]. Term frequency (TF) reflects how strongly a term occurs in a particular document. Inverse document frequency (IDF) reduces the influence of terms that occur in many documents and increases the relative influence of terms that distinguish a smaller subset of the corpus.

Let  $tf(t,d)$  denote the frequency of term  $t$  in document  $d$ , let  $N$  be the number of documents in the reference corpus, and let  $df(t)$  be the number of documents containing  $t$ . A basic formulation is:

$$tfidf(t,d) = tf(t,d) \times \log(N / df(t)).$$

Implementations often use logarithmic TF scaling, smoothed IDF, or vector normalization to handle repeated terms and unseen values. The exact formulation changes numeric scores, so a thesis must document the implemented formula rather than referring to TF-IDF as if it were a single universal function. A controlled or static reference corpus can improve score stability because the IDF of a term does not change whenever a user adds one document; however, it may become less representative as technologies and hiring terminology evolve.

### 2.3.2 Cosine Similarity

Cosine similarity compares the direction of two vectors rather than their unnormalized magnitude. For vectors  $x$  and  $y$ , it is defined as:

$$\cos(x,y) = (x \cdot y) / (\|x\|_2 \|y\|_2).$$

For non-negative TF-IDF vectors, cosine similarity usually lies between 0 and 1. A larger value indicates a greater proportion of shared weighted terms. Length normalization is valuable for CV–JD comparison because a long CV should not receive a higher score merely because it contains more words. TF-IDF-weighted

cosine distance has also been used as an efficient and reproducible document-alignment method [3].

Cosine similarity remains a lexical measure. If a JD uses “container orchestration” while a CV only uses “Kubernetes,” the vectors may not overlap unless the vocabulary or normalization layer establishes that relationship. Conversely, shared generic terms can increase similarity without proving that mandatory requirements are satisfied. CareerFit therefore treats similarity as one component of a broader workflow that also includes structured fields, validation, labels, reasons, user feedback, and policy conditions.

## 2.4 Matching, Recommendation, and Ranking

Matching and recommendation can be formalized as ranking tasks. Given a query representation  $q$  and a candidate set  $D$ , the system computes a relevance score  $s(q,d)$  for each  $d \in D$  and orders the results by decreasing score. For candidate-to-job matching,  $q$  may be a CV vector and  $D$  the active job vectors. For recruiter-side discovery,  $q$  may be the selected JD and  $D$  the available candidate or CV vectors. For profile-based recommendation,  $q$  represents the candidate’s desired role, skills, location, seniority, and other preferences rather than one uploaded CV.

This separation matters because a CV describes past experience, while a preference profile describes what the candidate wants. A candidate may have Java experience but look for a Go position, so CV-based matching and preference-based recommendation can produce different but valid lists. Keeping the workflows separate also makes evaluation clearer because the labels, candidate sets, and user goals of one task should not be assumed to fit the other task.

A raw similarity value is not automatically a calibrated probability of hiring success. Mapping it to a percentage or label is a presentation and business-rule decision, not a statistical guarantee. Thresholds such as Low, Medium, High, or Potential should be documented, tested at boundary values, and kept distinct from application status. Stable secondary sorting is also necessary when multiple items have equal scores so that repeated requests do not appear inconsistent.

Recent resume–job matching research increasingly uses dense encoders and contrastive learning. ConFit v2, for example, improves resume–job representations through hypothetical resumes and hard-negative mining [7]. Such approaches can capture semantic relations beyond exact terms, but they require training data, model governance, and an evaluation design appropriate to the domain. CareerFit intentionally begins with a transparent lexical baseline whose calculations can be reproduced and explained, while treating semantic or hybrid retrieval as future work rather than an implemented result.

## 2.5 Relevance Feedback and the Rocchio Method

### 2.5.1 Relevance Feedback

Relevance feedback uses judgments about retrieved items to modify a query or profile representation. Instead of assuming that the original text completely expresses the user's information need, the system observes which results are marked relevant or non-relevant. Explicit feedback is especially useful in recruitment because the importance of related skills may be known to the candidate or recruiter but omitted from the original CV, profile, or JD.

The classic Rocchio method updates a query vector by combining the original query with the centroids of relevant and non-relevant document vectors [4]. Let  $q_0$  be the original vector,  $D_r$  the set of relevant feedback documents, and  $D_{nr}$  the set of non-relevant documents. A common form is:

$$q_m = \alpha q_0 + (\beta / |D_r|) \sum (d \in D_r) d - (\gamma / |D_{nr}|) \sum (d \in D_{nr}) d.$$

The parameters  $\alpha$ ,  $\beta$ , and  $\gamma$  control retention of the original representation, attraction toward relevant examples, and movement away from non-relevant examples. They are hyperparameters rather than universal constants. Positive and negative feedback may also be weighted differently because an explicit rejection can be ambiguous: a user may reject a job because of location, timing, or salary even when its technical content is relevant.

### 2.5.2 Feedback Semantics in CareerFit

CareerFit distinguishes feedback such as GOOD\_MATCH, POTENTIAL, BAD\_MATCH, and NOT\_INTERESTED. These labels should not be collapsed without a documented mapping. GOOD\_MATCH provides a strong positive relevance signal. POTENTIAL may represent partial or transferable fit rather than direct relevance. BAD\_MATCH is a negative judgment about the match. NOT\_INTERESTED can reflect personal intent and may be weaker evidence about technical similarity. The implementation chapter must state exactly how each event affects the learned vector and whether role or channel changes its weight.

A reliable update process keeps the original base vector and rebuilds the learned vector from the current feedback history. Updating the previously learned vector again and again can cause cumulative drift: the same recomputation may change the output even when there is no new feedback. In CareerFit, the same base vector, feedback set, and parameters should always produce the same learned vector. This repeatable behavior is important for retries, scheduled jobs, debugging, and audits.

Feedback learning should be evaluated by comparing rankings before and after a defined feedback event in a controlled setup. The experiment records the baseline

ranking, adds feedback, rebuilds the representation, and measures the effect on separate holdout items. If the same positive item is used both as feedback and as the only proof of improvement, the result is circular. Chapter 4 therefore separates feedback examples from holdout evaluation and clearly labels synthetic scenarios.

## 2.6 Evaluation Metrics for Ranked Results

Ranking evaluation focuses on the ordered results users inspect. It therefore measures relevance at cutoff K, the position of useful items, and graded relevance when available [2].

### 2.6.1 Precision@K, Recall@K, and HitRate@K

Precision@K measures how much of the visible top-K list is relevant, Recall@K measures how much known relevant content it retrieves, and HitRate@K records whether the list contains at least one relevant result [2].

### 2.6.2 Mean Reciprocal Rank

Reciprocal rank is  $1/r$  for the first relevant result at rank  $r$ , or zero when none is retrieved. MRR averages this value across queries, emphasizing the first useful result rather than additional relevant items [2].

### 2.6.3 Discounted Cumulative Gain

Discounted Cumulative Gain (DCG) supports graded relevance and discounts useful items at lower ranks. Normalized DCG divides the observed score by the ideal ordering, allowing comparison across queries [5]. One common formulation is:

$$DCG@K = \sum_{i=1..K} (2^{rel_i} - 1) / \log_2(i + 1), \quad nDCG@K = DCG@K / IDCG@K.$$

Metric values require documented labels, candidate sets, cutoff K, and calculation rules. Synthetic improvement demonstrates only the designed scenario, so Chapter 4 reports the data source, holdout logic, repeated-run behavior, and validity limits with the scores.



## **2.7 Human-in-the-Loop and Explainable Automation**

### **2.7.1 Levels of Human Involvement**

Human-in-the-Loop is not a single interface feature. Human involvement can occur during data validation, model configuration, review of recommendations, approval of actions, exception handling, and post-action feedback. NIST describes human–AI configurations as ranging from manual decision making through systems that provide an additional opinion or defer to an expert, to more autonomous operation [9]. The appropriate configuration depends on the consequences and failure modes of the task.

Recruitment decisions can strongly affect people, so a similarity engine should help users set priorities instead of acting as the final decision-maker. CareerFit separates input processing, decision support, actions, learning, and audits. The matching engine provides evidence; the AutoFit policy checks whether an action is allowed; users can review or confirm important actions; feedback changes later rankings; and audit records support investigation. This design does not remove bias, but it makes the control points and system state clearer.

### **2.7.2 Explainability and Auditability**

Explainability describes why a result or action was produced. Auditability asks whether the inputs, rules, actor, channel, time, and outcome can be checked later. A list of shared skills can explain lexical similarity, but it does not prove that the whole model or decision is fair. In the same way, an audit log records what happened but does not prove that the policy was correct.

Research on algorithmic hiring warns that claims about reducing bias and improving performance must be checked against real practices, data, and organizational context [10]. Explanations can help users understand a result, but they should be based on the features that the system actually uses. Research on explainable recommendation also separates scoring from the user-facing reasons shown with a result [14]. CareerFit therefore uses match reasons and policy outcomes that can be checked instead of unsupported natural-language claims about candidate suitability.

The NIST AI Risk Management Framework groups AI risk work into governance, mapping, measurement, and management [9]. CareerFit is not a complete implementation of this framework. However, its design follows several practical ideas: state the scope and limitations, keep people responsible for important actions, measure behavior with defined evidence, and keep records for later review.

## **2.8 Web Architecture, Security, and Audit Foundations**

### **2.8.1 Client–Server and REST**

CareerFit separates presentation and navigation in the web client from authentication, authorization, validation, transactions, persistence, and protected business rules in the backend. Browser state is therefore not treated as authority for protected actions.

The backend exposes a REST-oriented HTTP API with stateless interaction and resource representations [11]. Chapter 3 documents the actual endpoints and state transitions rather than assuming that JSON over HTTP satisfies every REST constraint.

### **2.8.2 Authentication, Authorization, and Action Tokens**

CareerFit combines authenticated identity, role authorization, and resource ownership checks. Sharing the Candidate role, for example, must not permit one account to access another Candidate's private CV.

JWT carries signed claims such as subject, role, and expiry [12]. CareerFit must still validate the expected algorithm and claims, reload account state, and apply business authorization. Access tokens and purpose-specific email-action tokens are not interchangeable credentials.

An actionable link can be forwarded or opened by a mail scanner. CareerFit therefore uses a non-mutating GET confirmation followed by deliberate POST execution. Only the SHA-256 token hash is stored, and expiry and redeemed-state checks prevent normal reuse. Production still requires rate limiting, origin controls, secret rotation, and delivery monitoring.

### **2.8.3 Audit Logging**

CareerFit audit records identify the actor, action, target, source channel, result, timestamp, and relevant policy or score context. Sensitive CV content and credentials are excluded unless a documented need exists. This supports investigation without treating ordinary debug logs as business evidence [13].

Traceability still depends on database permissions, retention, integrity, and access control. Operational debug logs and domain audit records may therefore use different schemas and retention rules; Chapter 3 documents the implemented CareerFit mechanisms.

## 2.9 Related Work and Research Gap

Prior work can be grouped into lexical retrieval, learned resume–job representation, explainable recommendation, and human-centered governance. The classical vector-space and relevance-feedback literature provides transparent mathematical foundations [1], [4]. Contemporary resume–job matching methods such as ConFit v2 use trained dense representations and hard-negative strategies to improve semantic matching [7]. Real-world observational research comparing human and LLM resume ratings found only minor correlation between the two, indicating that automated and human judgments should not be treated as interchangeable [8]. Explainable recommendation research seeks to accompany scores with understandable, preferably grounded reasons [14]. Risk-management and algorithmic-hiring studies further support explicit human oversight, documented controls, and careful treatment of bias claims [9], [10].

A CareerViet industry article describes AI Matching as comparing Candidate CV information with Job requirements, presenting a matching score, and supporting suitable-Job and CV-improvement guidance [19]. This article was a practical source of inspiration for CareerFit, but it is treated as an industry example rather than academic evidence of algorithmic effectiveness.

CareerFit does not attempt to outperform these research systems on a shared benchmark. Its engineering gap is different: connecting an inspectable lexical baseline and explicit relevance feedback to an end-to-end recruitment workflow with role-based interfaces, policy gating, actionable email, and auditable state changes. The resulting contribution is system integration and controlled automation within an IT-focused academic prototype.

*Table 2.1. Positioning of CareerFit against major approach categories*

| <b>Approach</b>       | <b>Strength</b>                            | <b>Limitation<br/>relative to<br/>CareerFit scope</b> | <b>CareerFit position</b>                                                    |
|-----------------------|--------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------|
| Lexical TF-IDF/cosine | Transparent,<br>efficient,<br>reproducible | Limited semantic<br>equivalence                       | Implemented<br>baseline with<br>normalization,<br>reasons, and<br>validation |

| <b>Approach</b>                   | <b>Strength</b>                                | <b>Limitation relative to CareerFit scope</b>                | <b>CareerFit position</b>                                          |
|-----------------------------------|------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------|
| Dense resume–job encoders         | Captures semantic relations                    | Requires training data, governance, and benchmark validation | Future hybrid/semantic extension; not claimed as implemented       |
| General job portal                | Strong search, publication, and application UX | May not expose scoring, feedback, or policy internals        | Combines portal workflows with inspectable matching and automation |
| Fully automated decision pipeline | Reduces manual steps                           | Higher control, accountability, and error-propagation risk   | Policy-gated HITL actions with confirmation and audit              |
| Explainable recommender           | Provides user-facing reasons                   | Explanation may be unfaithful if not grounded                | Uses reasons tied to processed fields and lexical evidence         |

The comparison shows that CareerFit deliberately accepts the semantic limitations of a lexical baseline in exchange for inspectability and implementation control. Its research value must therefore be assessed through reproducible behavior, workflow integration, and clearly stated limitations rather than through unsupported claims of general AI superiority.

## 2.10 Theoretical Background Summary

This chapter presented the main ideas used by CareerFit. TF-IDF and cosine similarity provide an inspectable lexical baseline, while Rocchio uses explicit feedback to adjust ranking. Ranking metrics measure ordered results, and Human-in-the-Loop policies keep similarity evidence separate from recruitment actions. Chapter 1 turns these ideas into system requirements, and Chapter 3 applies them in the system design and implementation.

## CHAPTER 3. SYSTEM DESIGN AND IMPLEMENTATION

This chapter applies the foundations from Chapter 2 to the CareerFit solution. It first presents the architecture, modules, data model, functional design, security and consistency design, and deployment topology. It then explains how these decisions are implemented in the backend, frontend, database, automation, email-action, reporting, and monitoring workflows.

### 3.1 System Architecture

CareerFit is implemented as a modular monolith. The frontend is a separate React application, while the backend is one Spring Boot deployable application organized into domain packages. This is not a microservice architecture: modules share one process, one primary PostgreSQL database, common security and exception handling, and direct in-process service calls. The modular monolith reduces deployment and distributed-transaction complexity while preserving boundaries that can support later extraction if scale or ownership requires it.

The presentation layer contains React pages, reusable components, route guards, API mapping, and query state. It calls the backend over HTTP and must not be trusted to enforce protected business rules. The API layer contains controllers and DTO validation. The service layer owns use-case orchestration, transactions, scoring, policy decisions, and ownership checks. Repositories map domain entities to PostgreSQL through Spring Data JPA. Cross-cutting configuration covers security, CORS, async execution, exception responses, scheduling, API documentation, and health/metrics endpoints.

The backend stores relational business state in PostgreSQL. JSONB is used for naturally variable collections or snapshots such as skills, extracted terms, vectors, reasons, benefits, policy metadata, and analytic distributions. CV binaries are stored through a storage service in a configured local path or Docker volume; the database stores file metadata and the processing result. SMTP is optional, and a no-operation mail implementation allows local workflows when outbound email is disabled.

## CareerFit Container and Component Architecture

UML component view with external dependencies

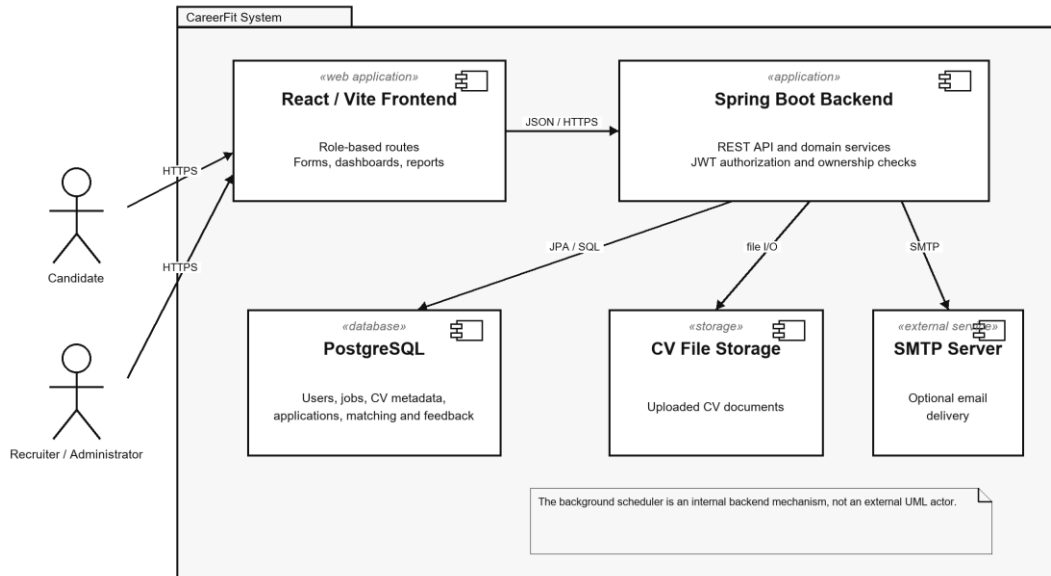


Figure 3.1. CareerFit container and component architecture

## 3.2 Module Design

Table 3.1. Backend modules and responsibilities

| Module                   | Main responsibility                                                                              | Representative services                                                         |
|--------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Auth and Security        | Password login, JWT validation, account resolution, role and route rules                         | AuthService, JwtService, security filters                                       |
| Candidate, CV, and Skill | Profile, CV draft/review, storage, OCR, validation, and skill suggestions                        | CandidateProfileService, CvIngestionService, PdfExtractionService, SkillService |
| Job and Employer         | Public catalogue, company onboarding, Job quality, draft/publish, urgent/deadline, and lifecycle | JobService, EmployerService, QualityValidationService                           |

| Module                      | Main responsibility                                                                         | Representative services                                                      |
|-----------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Matching and Recommendation | TF-IDF/cosine, reasons, Potential, batch matching, filters, ranking, and Job recommendation | ScoringService, SkillTransferService, MatchingService, RecommendationService |
| Application and Talent      | Apply/withdraw, applicants, invitations, status changes, Talent Pool, and bookmarks         | ApplicationService, RecruiterTalentService                                   |
| Feedback and Automation     | Rocchio update, recomputation, account policy, auto-apply, reminders, and scheduler         | FeedbackService, RocchioService, AutoApplyService, AutomationScheduler       |
| Notification                | Policy guard, delivery log, SMTP/no-op mail, and signed email actions                       | NotificationPolicyGuard, NotificationEmailService, EmailActionService        |
| Content Report              | Job/CV reports, access checks, queue grouping, ban/dismiss, counters, and audit             | ContentReportService, ContentReportController, AdminReportController         |
| Analytics, Admin, and Audit | Live analytics, account/Job administration, monitoring, and audit persistence               | Analytics services, administrative services, audit/common components         |

The modules cooperate through service calls rather than direct controller-to-repository shortcuts for core workflows. Transaction boundaries belong at service operations that change a consistent group of records. Read-only queries may use dedicated query services or projections to avoid loading unrelated state. DTOs prevent persistence entities from becoming an accidental public API contract.

### 3.3 Data Design

#### 3.3.1 Data Design Overview

CareerFit uses PostgreSQL as its transactional data store. Flyway migrations V1-V25 are the authoritative schema history, while Hibernate ddl-auto=validate checks that the application mappings match the migrated schema. The final schema contains 24 active application tables and 293 columns; the earlier email\_token table

was removed by V16. JSONB is retained for variable collections and snapshots such as skills, vectors, reasons, review sections, notification payloads, and analytic distributions.

The models below separate three levels of abstraction. The conceptual model shows business information without implementation detail. The logical models show normalized entities and cardinalities. The physical models use the final PostgreSQL table and column names. This separation prevents a high-level domain diagram from being mistaken for the deployed schema.

### 3.3.2 Conceptual Data Model

The conceptual model groups CareerFit information into identity, Career Profile, recruitment, Feedback, Automation, communication, and governance. A Candidate owns Profile and CV information, while a Recruiter owns an employer profile and Jobs. Matching links a CV and Job as decision-support evidence; Application records a separate recruitment action, so a score is never treated as an Application or final decision.

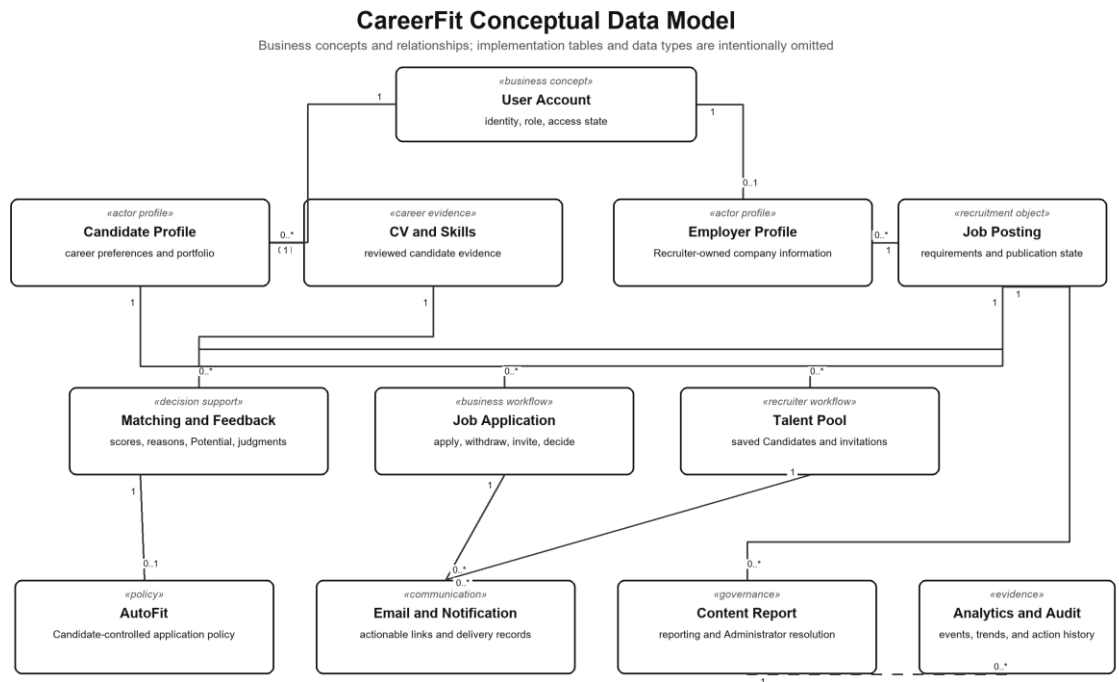


Figure 3.2. CareerFit conceptual data model



### 3.3.3 Logical Data Model

The logical model resolves these concepts into normalized entities and explicit cardinalities. Four diagrams keep the main ownership and lifecycle relationships readable: identity and Career Profile; recruitment and Matching; Automation and communication; and governance and analytics. Unique and foreign-key relationships protect one-account policies, one Candidate–Job Application, one CV–Job Matching, and traceable Feedback, invitation, Report, notification, and audit records.

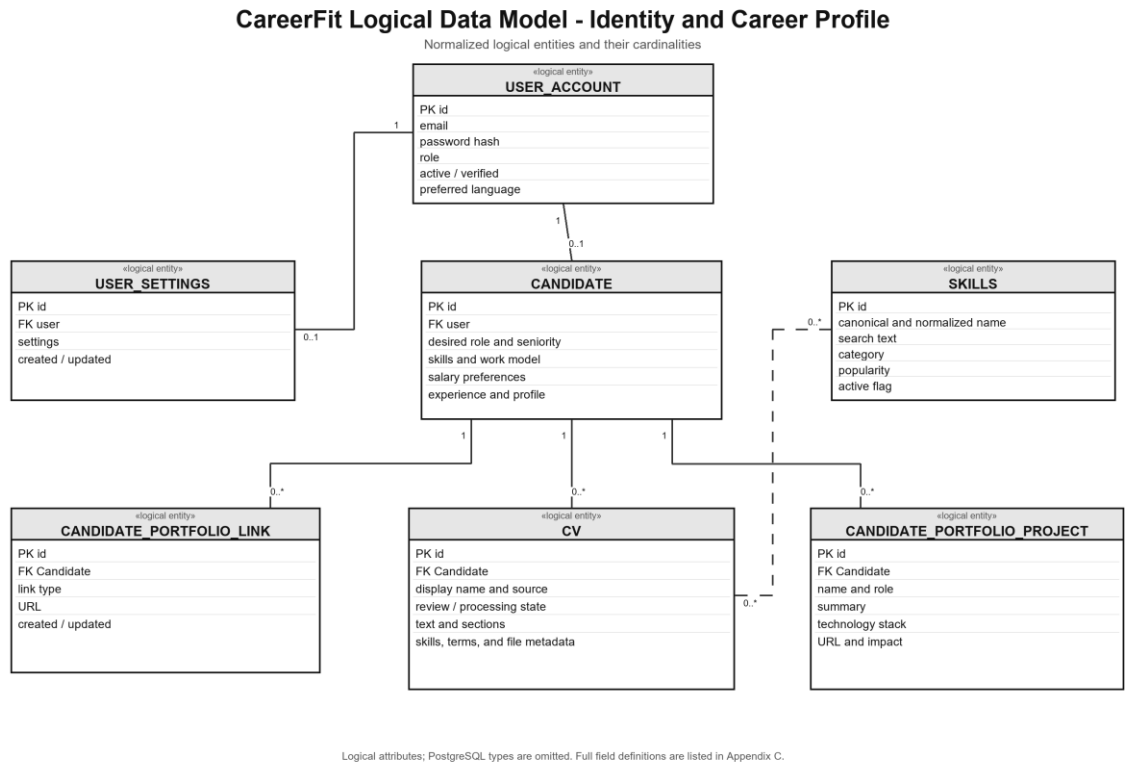


Figure 3.3. CareerFit logical data model - identity and Career Profile

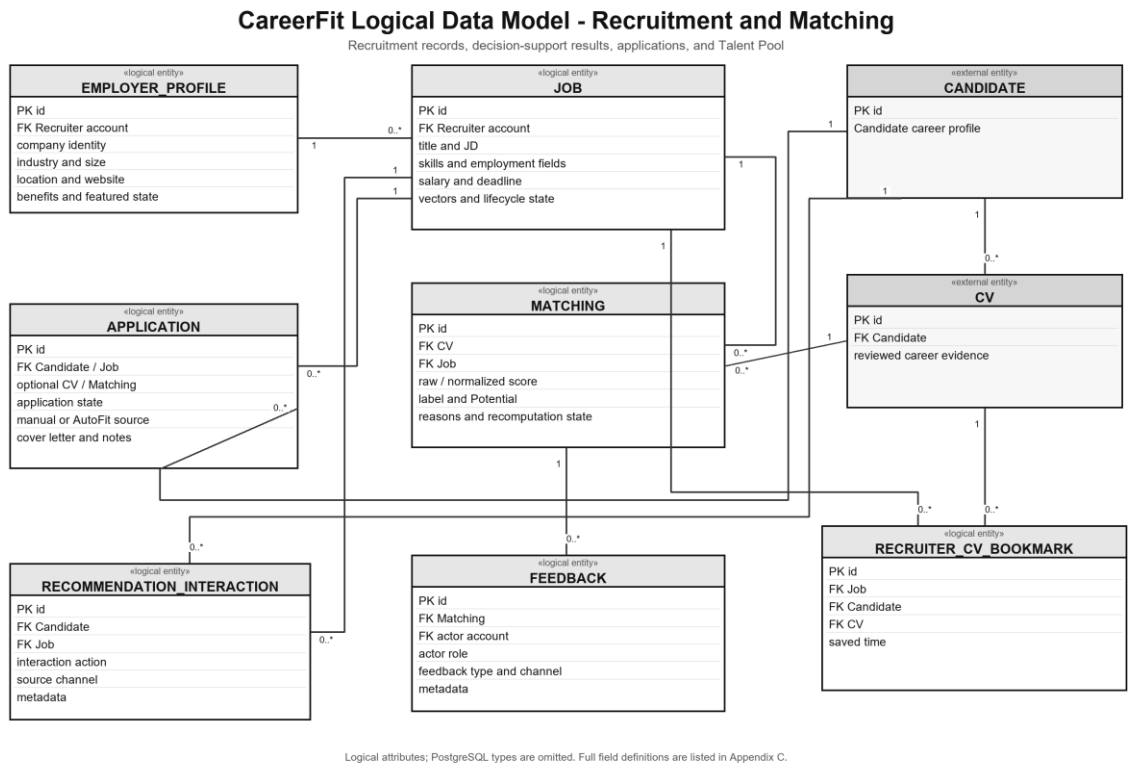


Figure 3.4. CareerFit logical data model - recruitment and Matching

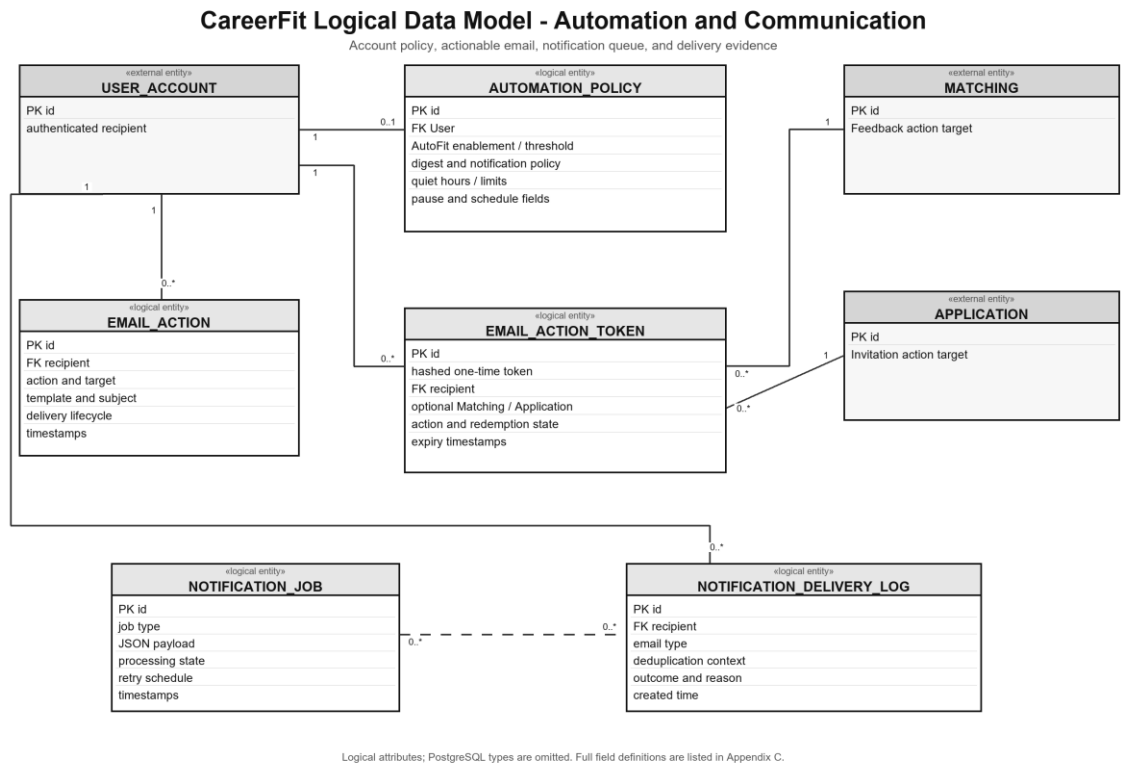


Figure 3.5. CareerFit logical data model - automation and communication

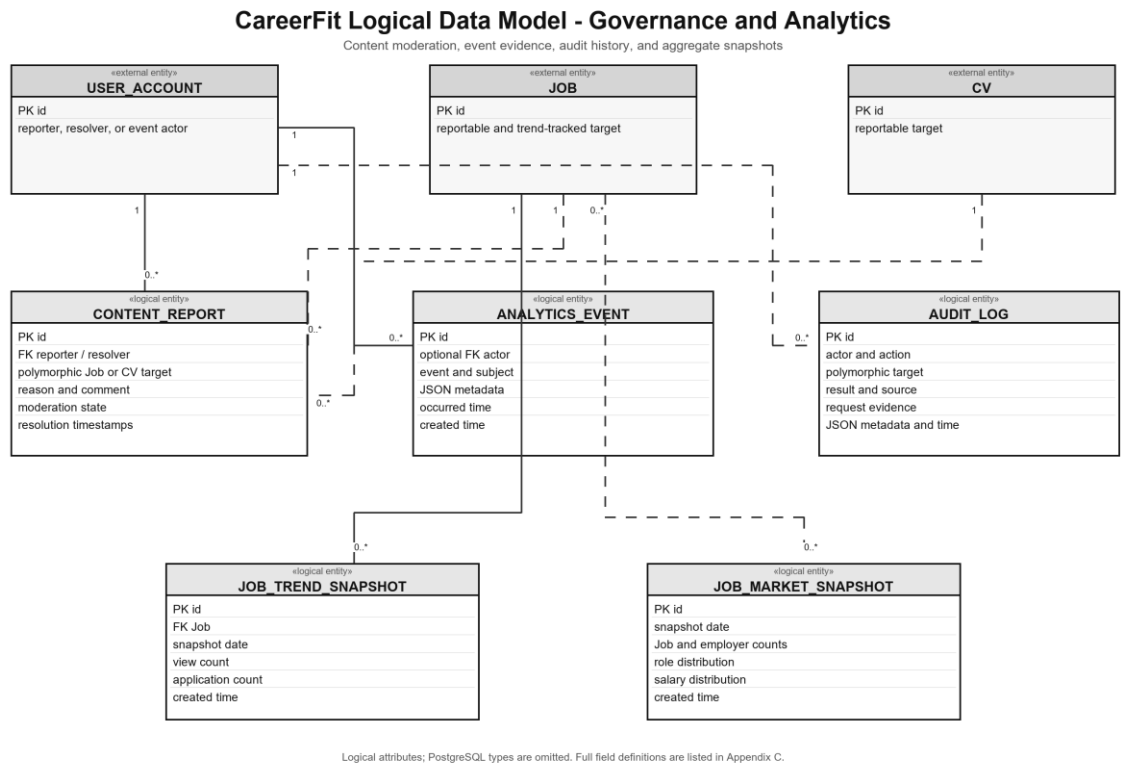


Figure 3.6. CareerFit logical data model - governance and analytics

### 3.3.4 Physical Data Model

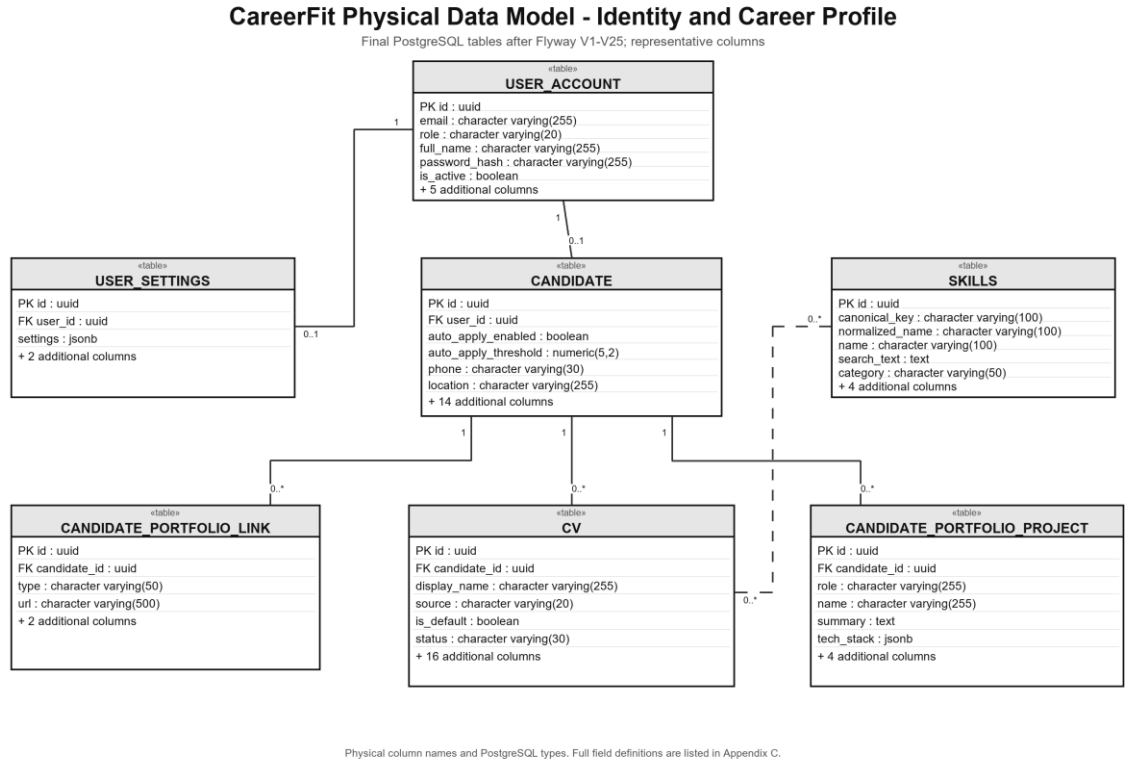
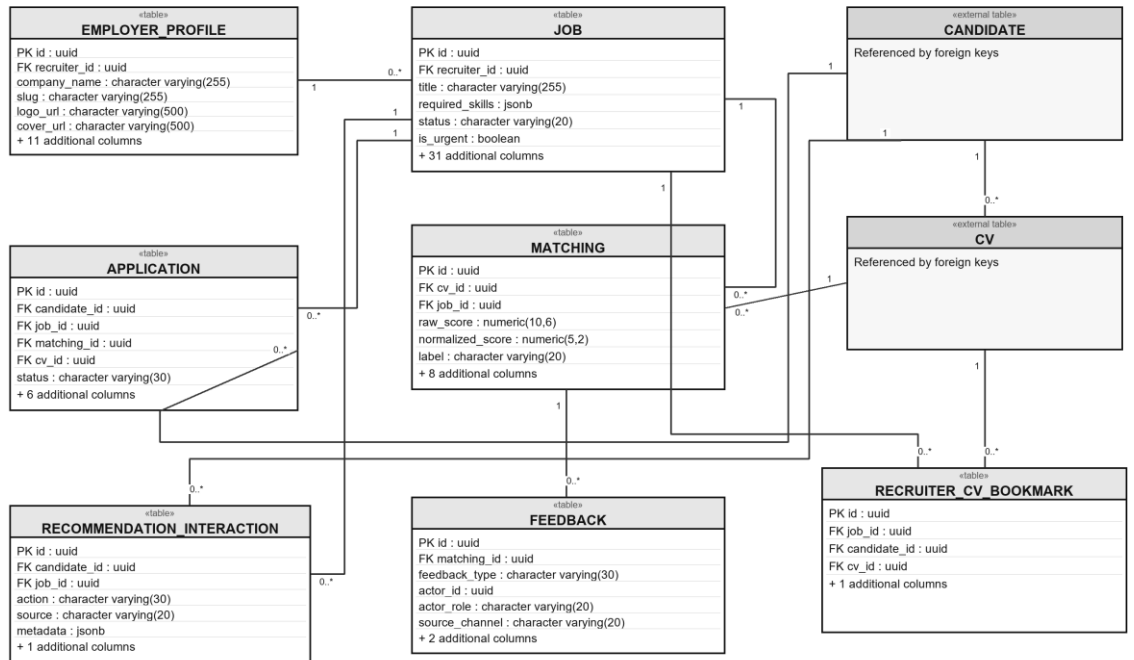


Figure 3.7. CareerFit physical data model - identity and Career Profile tables

## CareerFit Physical Data Model - Recruitment and Matching

Final PostgreSQL tables after Flyway V1-V25; representative columns

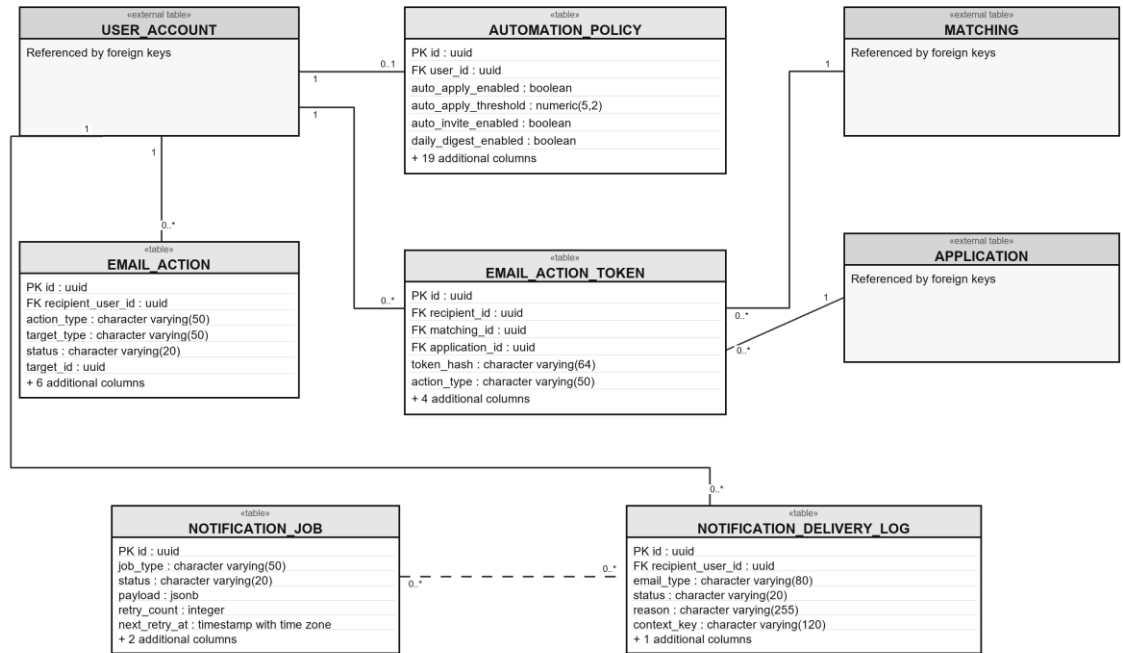


Physical column names and PostgreSQL types. Full field definitions are listed in Appendix C.

Figure 3.8. CareerFit physical data model - recruitment and Matching tables

## CareerFit Physical Data Model - Automation and Communication

Final PostgreSQL tables after Flyway V1-V25; representative columns

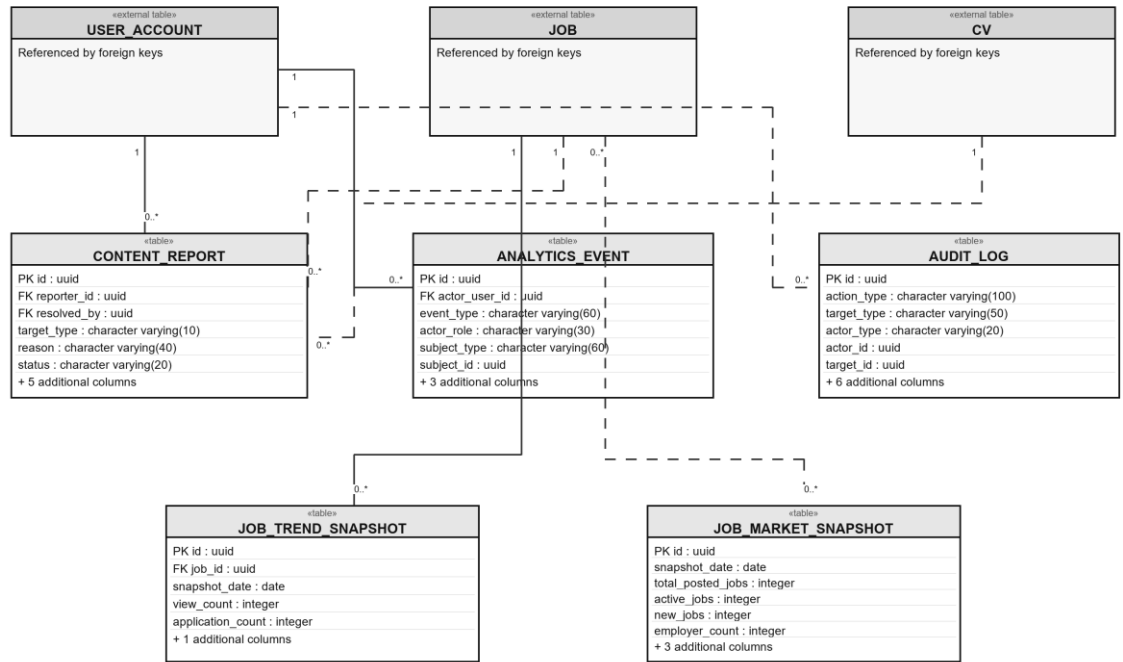


Physical column names and PostgreSQL types. Full field definitions are listed in Appendix C.

Figure 3.9. CareerFit physical data model - automation and communication tables

### CareerFit Physical Data Model - Governance and Analytics

Final PostgreSQL tables after Flyway V1-V25; representative columns



Physical column names and PostgreSQL types. Full field definitions are listed in Appendix C.

Figure 3.10. CareerFit physical data model - governance and analytics tables

Table 3.2. Key integrity constraints

| Constraint                                                                      | Purpose                                                           |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Unique normalized user email                                                    | Prevent duplicate identities that differ only by case             |
| One role profile and one policy per account                                     | Preserve ownership and account-level settings                     |
| Partial unique default CV per Candidate                                         | Ensure at most one active default CV                              |
| Unique Matching per CV and Job                                                  | Prevent duplicate direct scores for the same pair                 |
| Unique Application per Candidate and Job                                        | Prevent duplicate applications or invitations                     |
| Unique bookmark and feedback keys                                               | Keep one bookmark/judgment for each defined actor-target relation |
| Unique pending report per reporter and target                                   | Prevent duplicate unresolved abuse reports from the same account  |
| Check constraints on roles, states, labels, report values, salary, and channels | Reject invalid domain values at the database boundary             |
| Indexes for Jobs, ownership, ranking, audit, and report queue                   | Support catalogues, reminders, matching, and moderation queries   |



The constraints above are enforced at the database boundary in addition to service-level validation. Indexes support active Job search, ownership queries, Matching order, application history, feedback lookup, token redemption, notification policy checks, audit chronology, and the pending content-report queue.

### **3.3.5 Data Dictionary**

Appendix C contains the physical table catalogue and field-level data dictionary. Each entry is derived from the running PostgreSQL schema produced by Flyway V1-V25 and records the column name, PostgreSQL type, nullability, key role, default or check constraint, and business meaning. The dictionary documents persisted columns only; DTO-only fields, transient properties, repositories, and internal service variables are excluded.

## **3.4 Functional Design**

This section follows the role-oriented Functional Design structure used by the thesis template. Each selected function is presented through its purpose, an annotated implementation screenshot, a compact interface-component table, and a sequence diagram. Data tables are not repeated here because Section 3.3 and Appendix C already provide the complete data models and data dictionary.

### **3.4.1 Functional Design for Candidate**

#### ***3.4.1.1 Explore Jobs***

**Purpose:** This function allows a Candidate or Guest to search the active Job catalogue, apply visible filters, open a selected Job, and review the information needed before deciding whether to apply. Candidate-only guidance may also be shown when relevant Matching context is available.

## Interface:

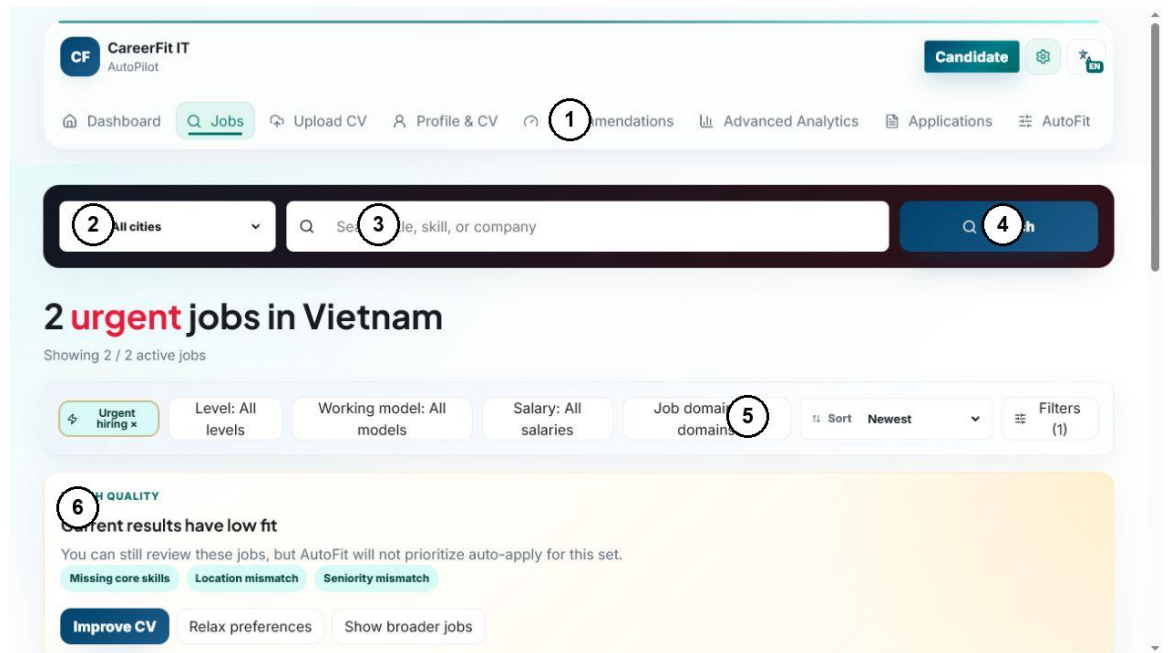


Figure 3.11. Annotated interface for Explore Jobs

## Interface Components:

Table 3.3. Interface components for Explore Jobs

| No. | Control Type      | Description                                           |
|-----|-------------------|-------------------------------------------------------|
| 1   | Navigation        | Open Candidate pages.                                 |
| 2   | Select            | Choose the Job location scope.                        |
| 3   | Input             | Enter a Job title, skill, or company keyword.         |
| 4   | Button            | Submit the current search criteria.                   |
| 5   | Filter / Sort     | Refine the results and sort order.                    |
| 6   | Information panel | Show result-quality guidance or a limited-data state. |

## Processing Logic:

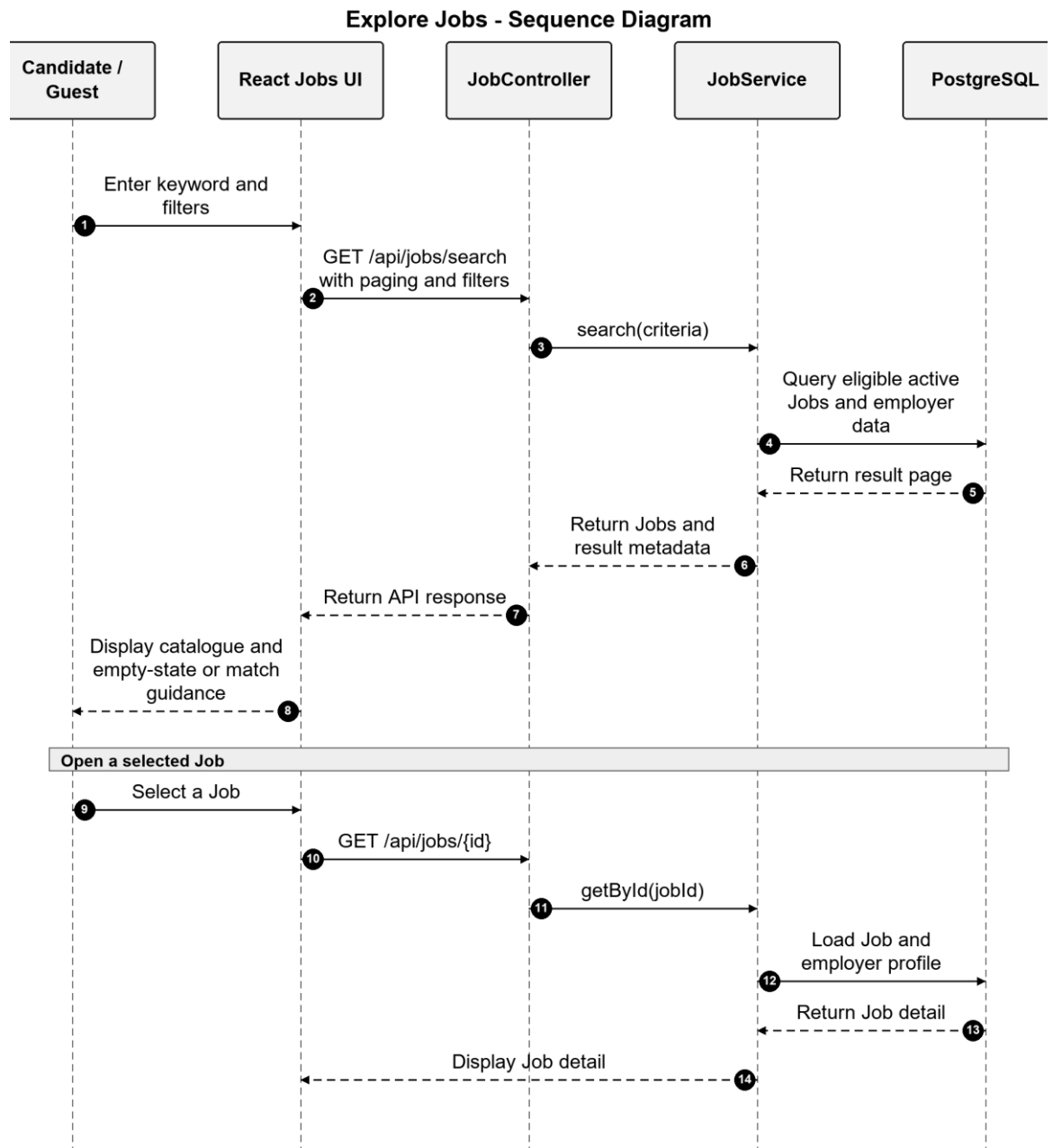


Figure 3.12. Sequence diagram for Explore Jobs

### 3.4.1.2 Manage AutoFit

**Purpose:** This function allows a Candidate to review and save AutoFit configuration and, when requested, start a manual AutoFit run. The saved configuration is also available to the system-configured automatic execution.

**Interface:**

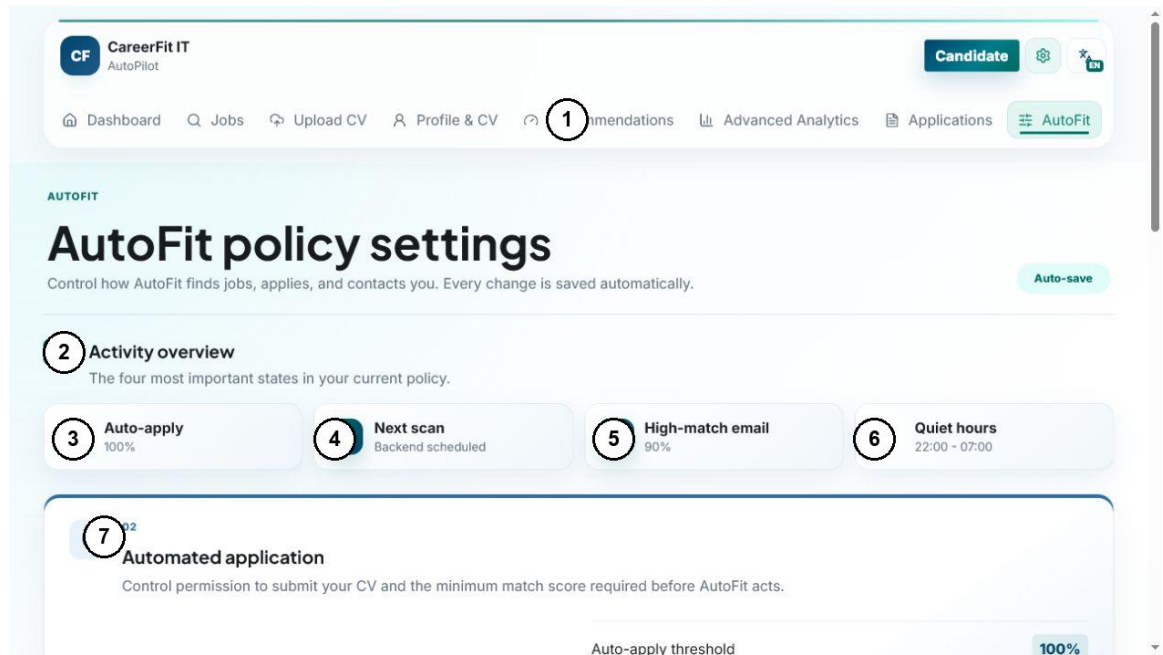


Figure 3.13. Annotated interface for Manage AutoFit

**Interface Components:**

Table 3.4. Interface components for Manage AutoFit

| No. | Control Type          | Description                                                                   |
|-----|-----------------------|-------------------------------------------------------------------------------|
| 1   | Navigation            | Open the Candidate workspace and AutoFit page.                                |
| 2   | Summary section       | Present the most important current AutoFit states.                            |
| 3   | Status card           | Show whether automatic application is enabled and its threshold.              |
| 4   | Status card           | Show the next system-configured AutoFit scan.                                 |
| 5   | Status card           | Show the high-match email state and threshold.                                |
| 6   | Status card           | Show the configured quiet-hours window.                                       |
| 7   | Configuration section | Change automatic-application consent and the minimum accepted Matching score. |

## Processing Logic:

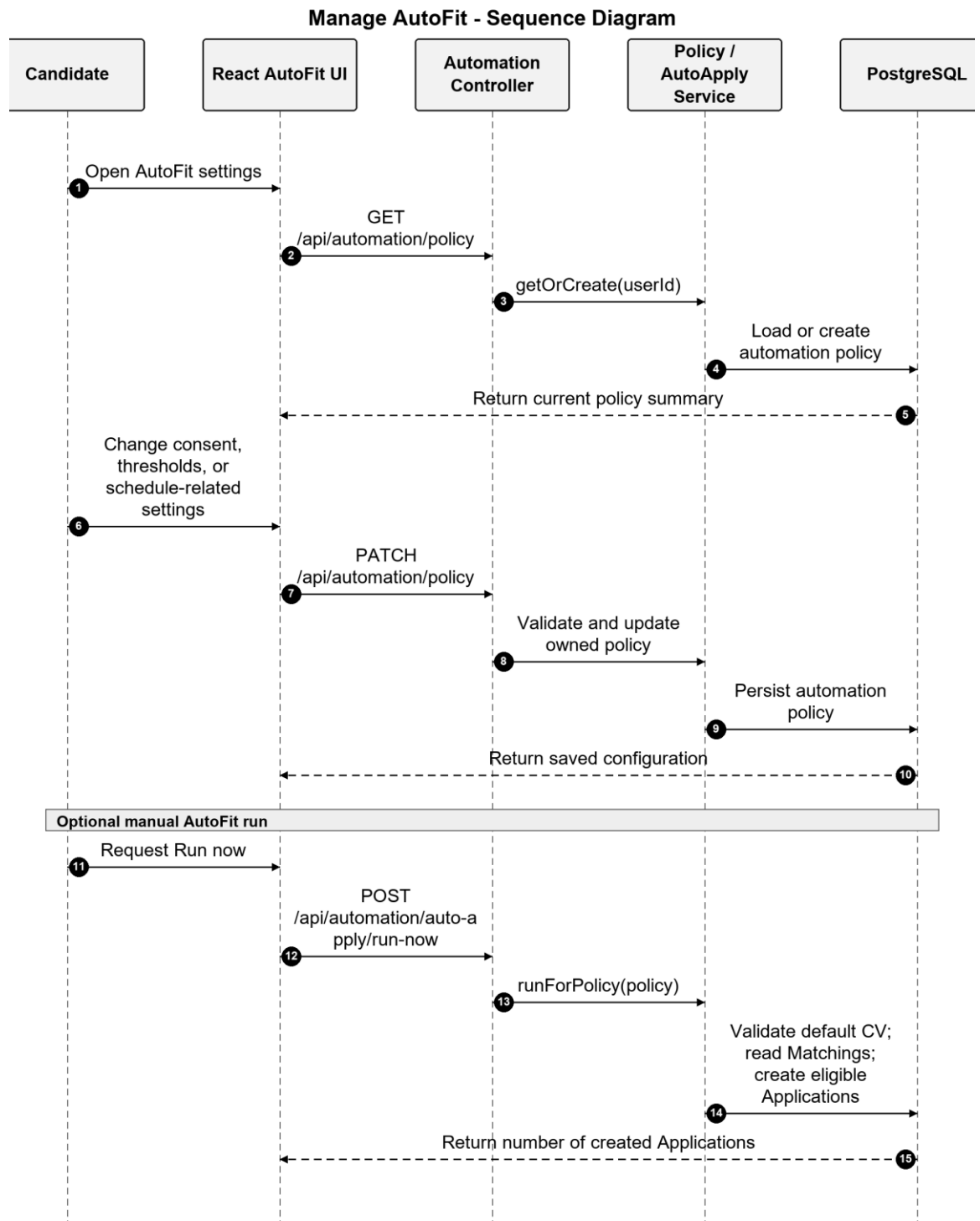


Figure 3.14. Sequence diagram for Manage AutoFit

### 3.4.1.3 Upload and Confirm a CV

**Purpose:** This function allows a Candidate to upload a supported CV document or use manual entry, review the extracted content, correct it when needed, and confirm the CV before scoring and Matching continue.

**Interface:**

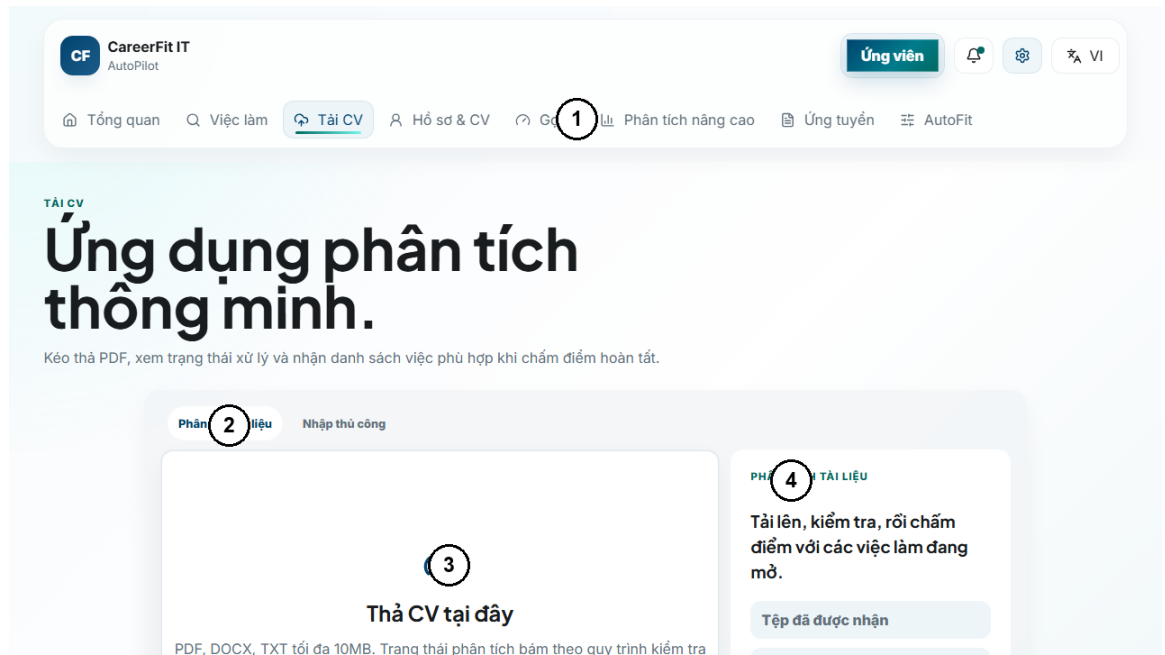


Figure 3.15. Annotated interface for CV upload

**Interface Components:**

Table 3.5. Interface components for CV upload

| No. | Control Type | Description                                                       |
|-----|--------------|-------------------------------------------------------------------|
| 1   | Navigation   | Open Candidate pages and the CV upload entry point.               |
| 2   | Tab          | Choose document analysis or manual CV entry.                      |
| 3   | Upload area  | Select or drop a supported CV file for processing.                |
| 4   | Status panel | Show file acceptance, processing, review, or failure information. |

## Processing Logic:

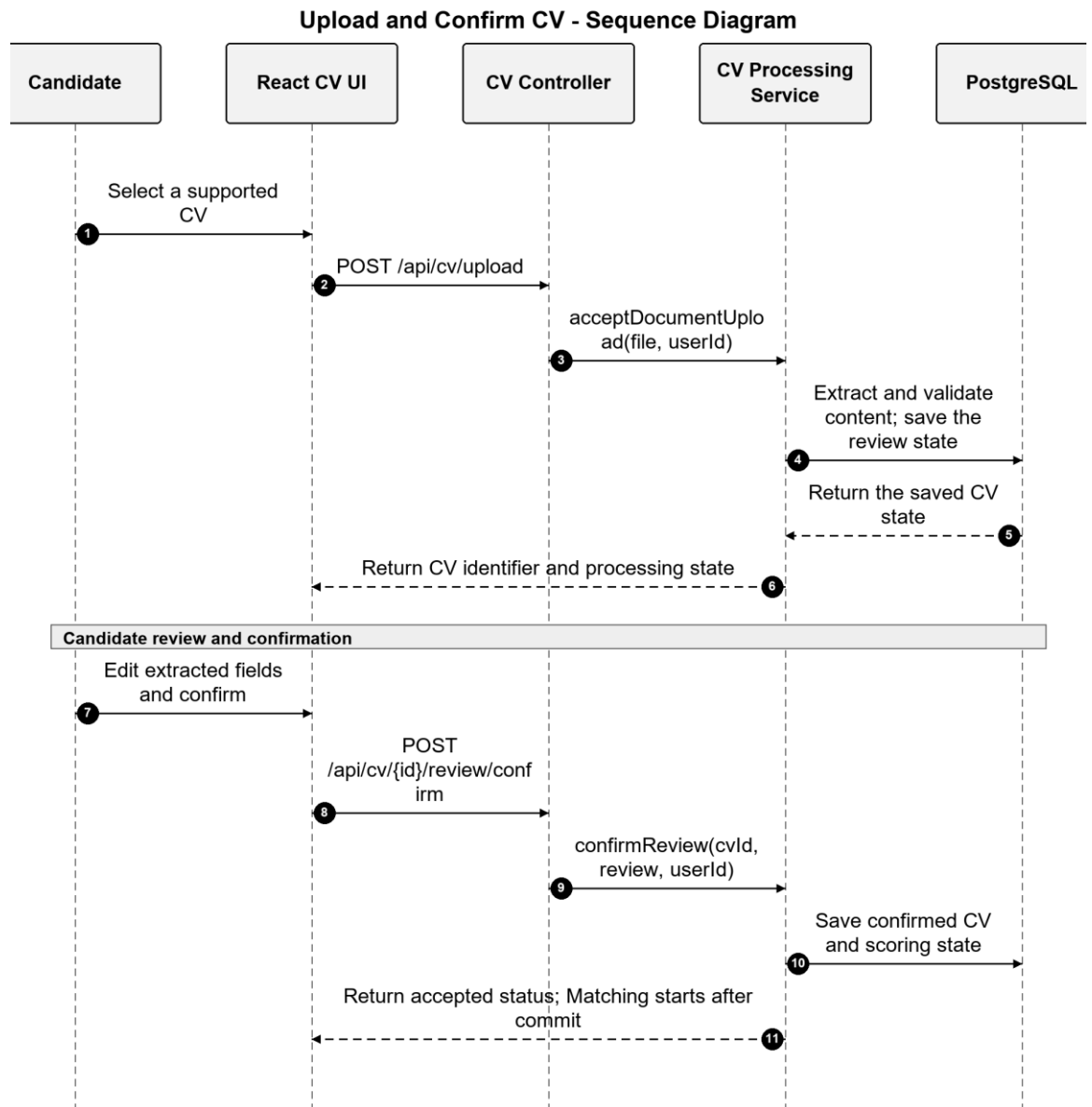


Figure 3.16. Sequence diagram for CV upload, review, and confirmation

### 3.4.2 Functional Design for Recruiter

#### 3.4.2.1 Manage Job Postings and Applicants

**Purpose:** This function supports the Recruiter's main Job workspace. It provides Job search and status filtering, draft or publication actions, selected-Job details, and access to the Applications submitted for an owned Job.

**Interface:**

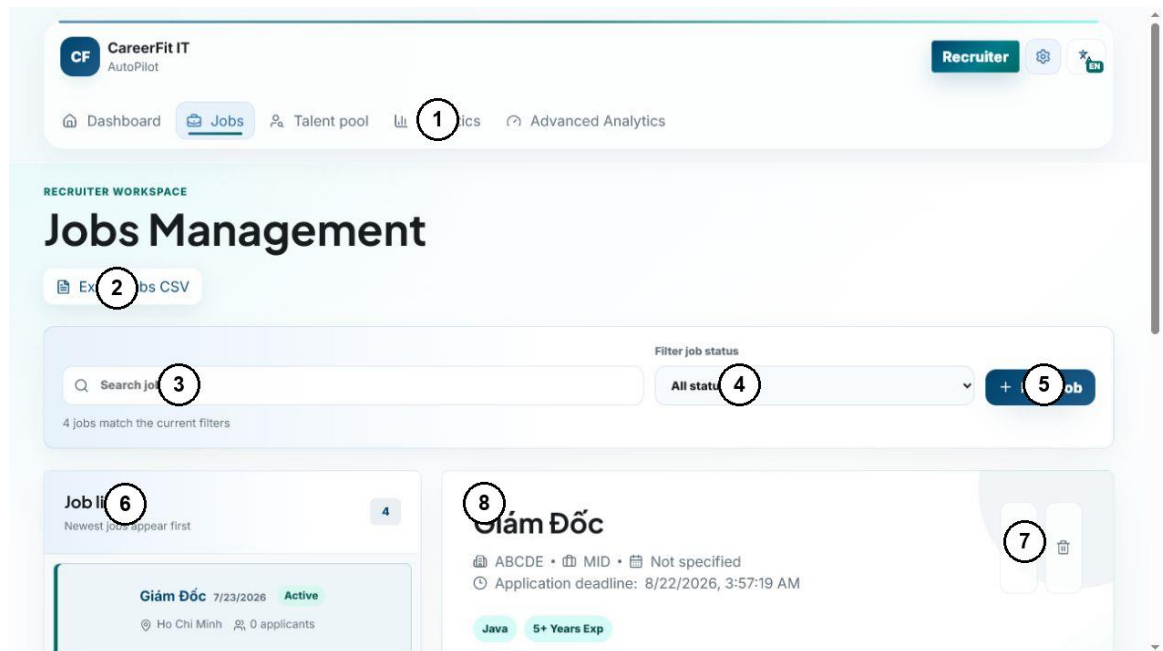


Figure 3.17. Annotated interface for Job and applicant management

**Interface Components:**

Table 3.6. Interface components for Job and applicant management

| No. | Control Type   | Description                                      |
|-----|----------------|--------------------------------------------------|
| 1   | Navigation     | Open Recruiter pages.                            |
| 2   | Button         | Export the Recruiter's Job list as CSV.          |
| 3   | Input          | Search within the Recruiter's Jobs.              |
| 4   | Select         | Filter Jobs by lifecycle status.                 |
| 5   | Button         | Open the form for a new Job posting.             |
| 6   | Job list       | Display owned Jobs and select one Job.           |
| 7   | Action buttons | Edit or delete the selected Job when permitted.  |
| 8   | Detail panel   | Show the selected Job and open applicant review. |



## Processing Logic:

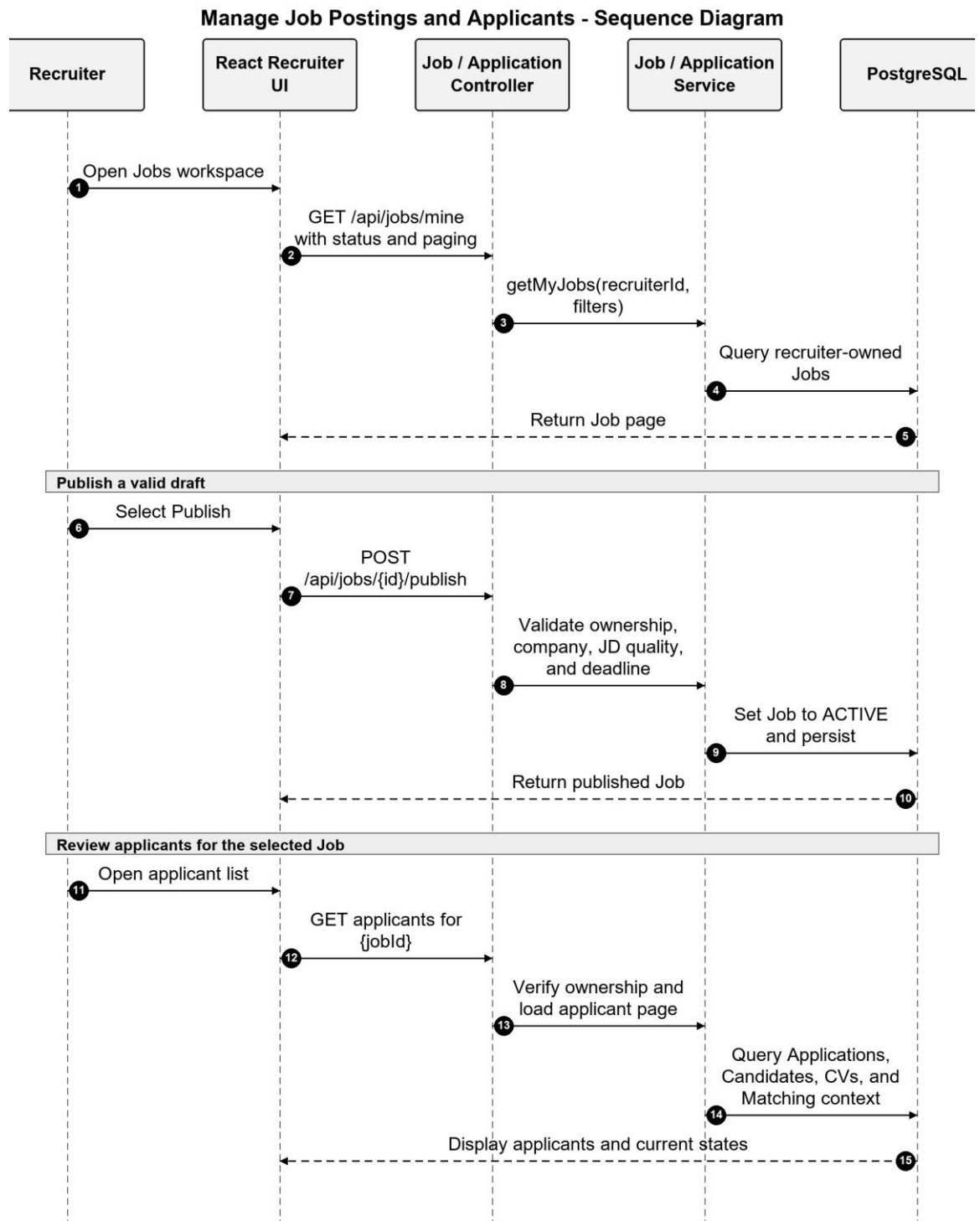


Figure 3.18. Sequence diagram for Job publication and applicant review

### 3.4.2.2 Manage Talent Pool and Invitations

**Purpose:** This function allows a Recruiter to inspect Matching and Potential CV groups for an owned Job, bookmark Candidates for later review, inspect visible CV details, and create or track recruitment invitations.

**Interface:**

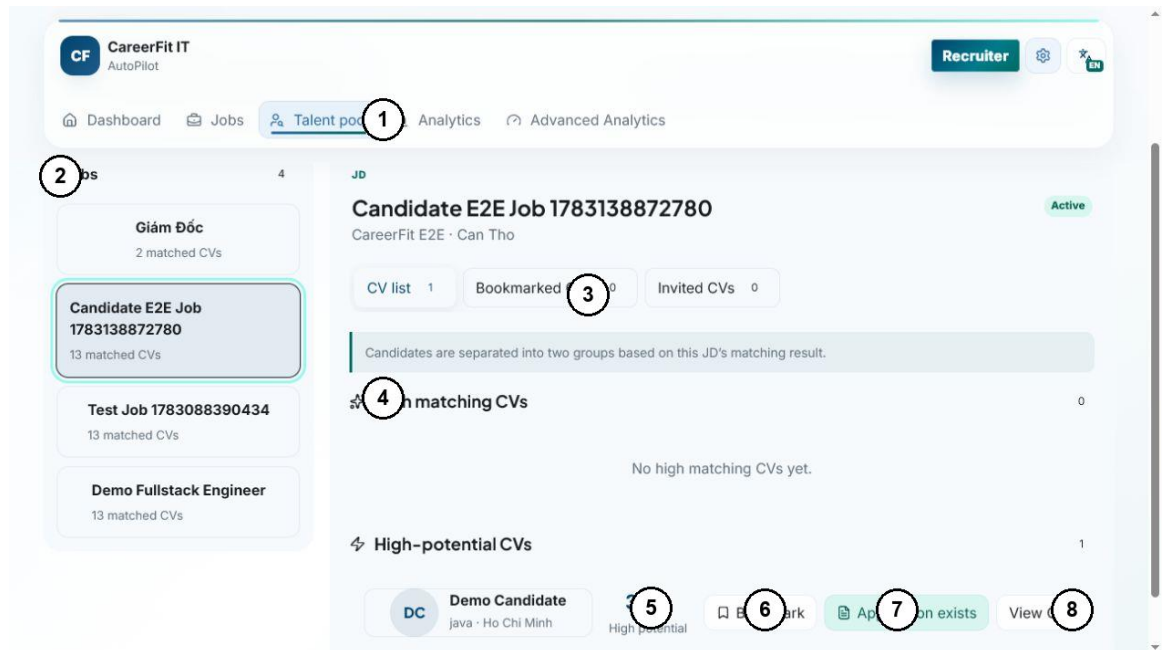


Figure 3.19. Annotated interface for Talent Pool and invitations

**Interface Components:**

Table 3.7. Interface components for Talent Pool and invitations

| No. | Control Type      | Description                                      |
|-----|-------------------|--------------------------------------------------|
| 1   | Navigation        | Open the Recruiter Talent Pool.                  |
| 2   | Job list          | Select an owned Job for Candidate results.       |
| 3   | Tabs              | Switch between all, bookmarked, and invited CVs. |
| 4   | Result groups     | Separate Matching and Potential CV groups.       |
| 5   | Candidate summary | Show the Candidate summary and score context.    |
| 6   | Button            | Save or remove the Candidate bookmark.           |
| 7   | State badge       | Show an existing Application or invitation.      |
| 8   | Button            | Open the visible CV detail.                      |

## Processing Logic:

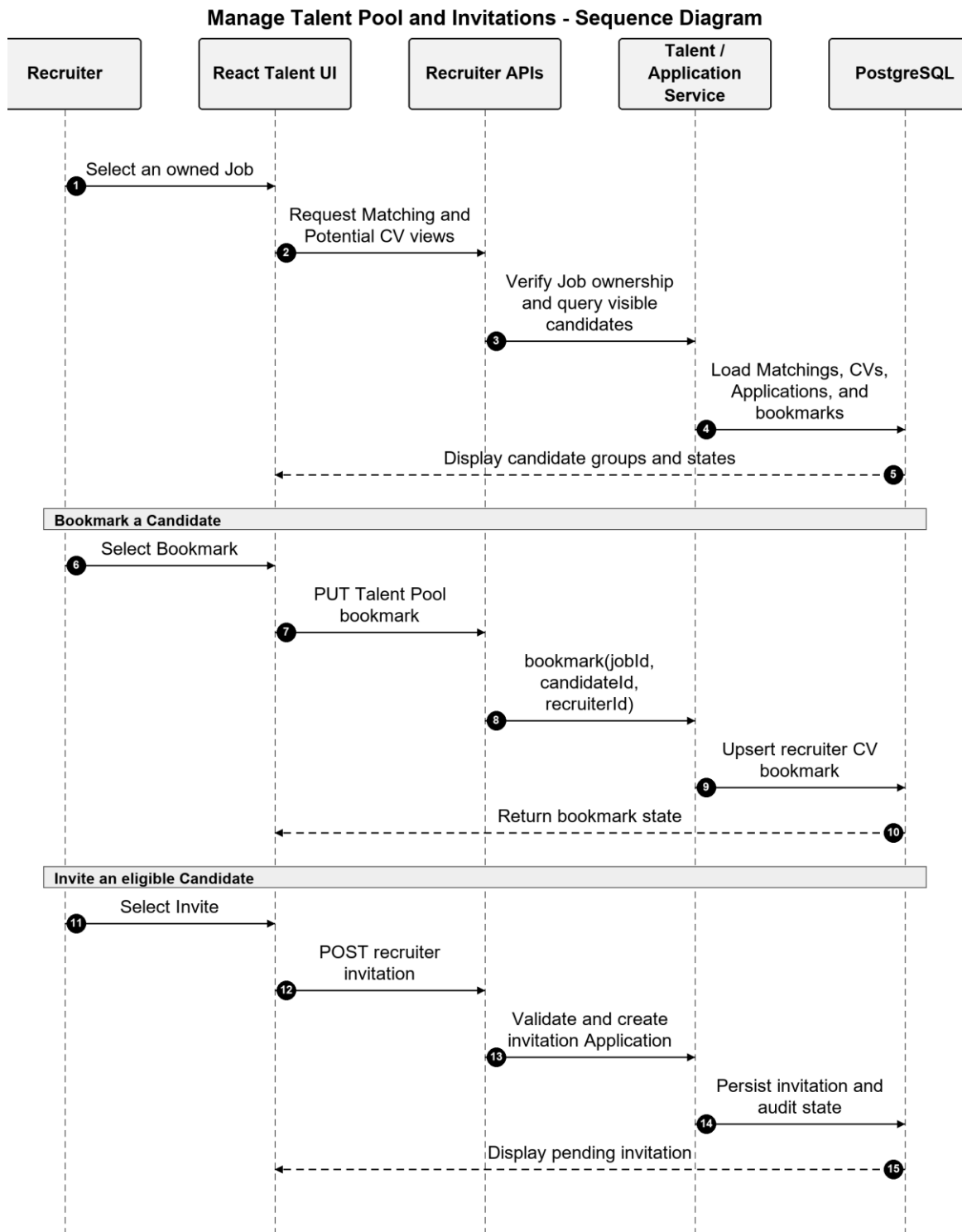


Figure 3.20. Sequence diagram for Talent Pool bookmarking and invitation

### 3.4.3 Functional Design for Administrator

#### 3.4.3.1 Review Administrative Audit Activity

**Purpose:** This supporting function allows an Administrator to review recorded administrative and business events. It helps verify who performed an action, which target was affected, and whether the recorded result succeeded.

**Interface:**

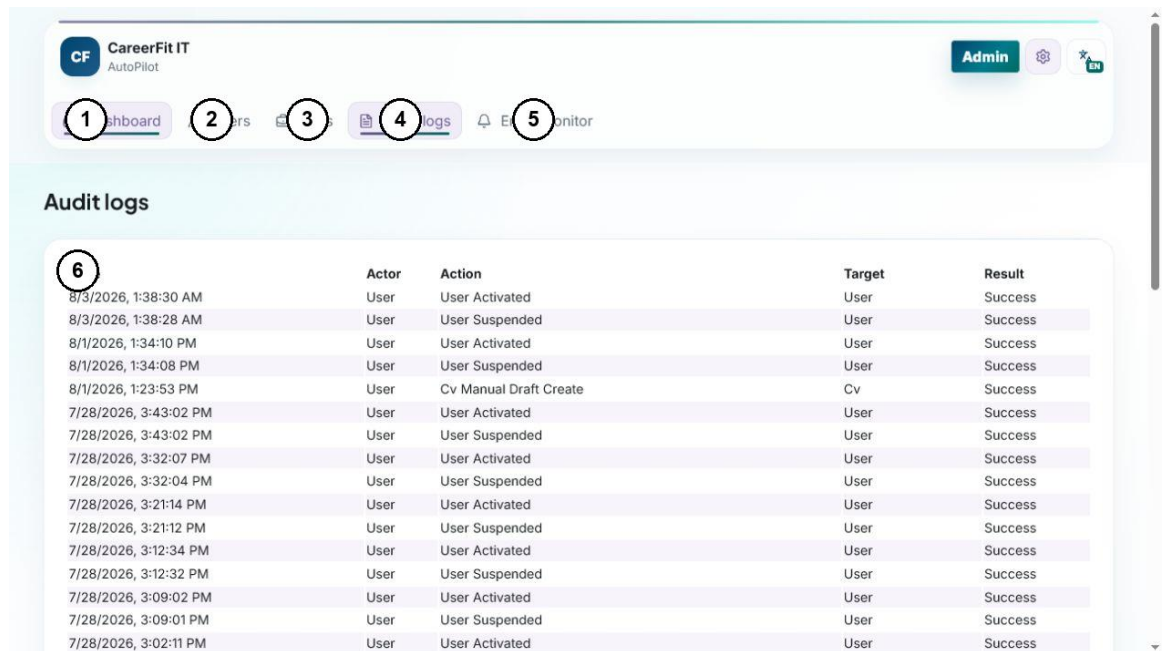


Figure 3.21. Annotated interface for administrative audit activity

**Interface Components:**

Table 3.8. Interface components for administrative audit activity

| No. | Control Type | Description                                           |
|-----|--------------|-------------------------------------------------------|
| 1   | Navigation   | Open the administrative overview.                     |
| 2   | Navigation   | Open user access administration.                      |
| 3   | Navigation   | Open Job visibility administration.                   |
| 4   | Navigation   | Open the audit-log view.                              |
| 5   | Navigation   | Open email-action monitoring.                         |
| 6   | Audit table  | Review event time, actor, action, target, and result. |

## Processing Logic:

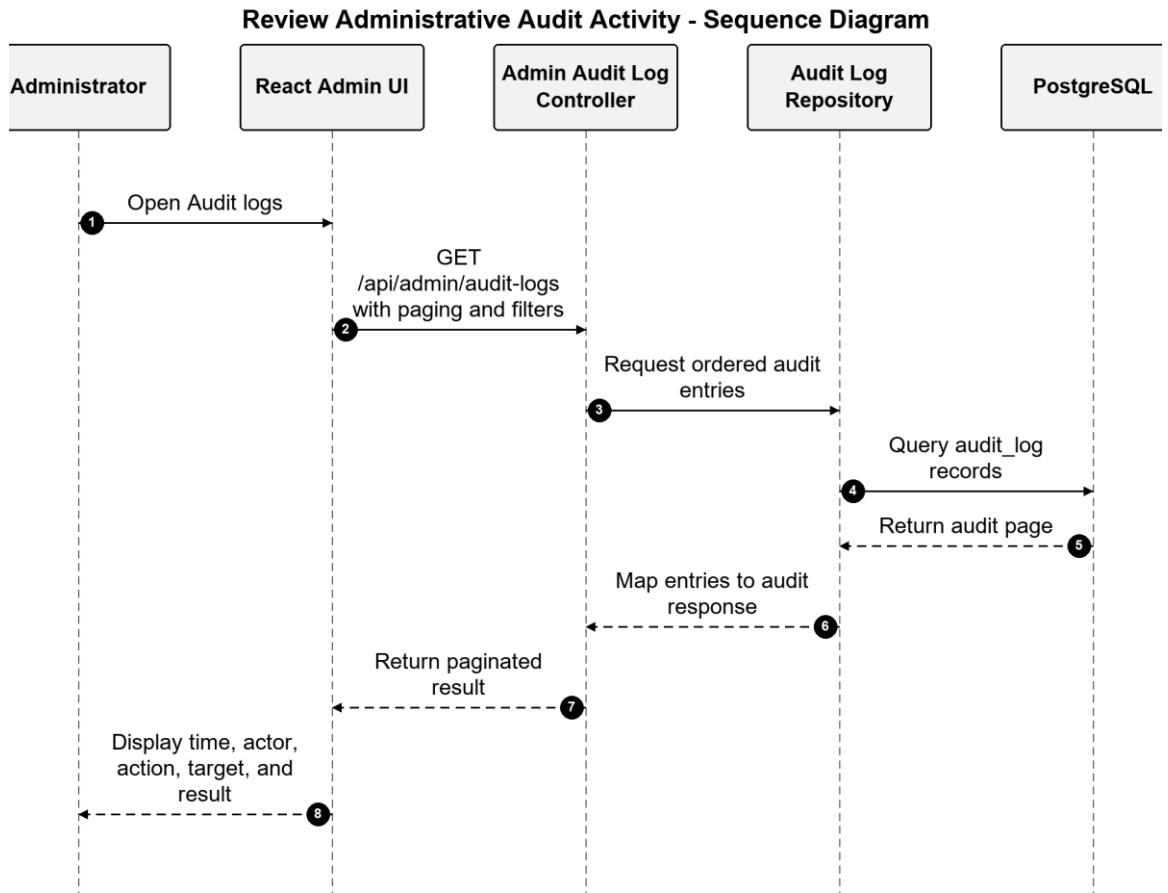


Figure 3.22. Sequence diagram for reviewing administrative audit activity

## 3.5 Security, Failure, and Consistency Design

### 3.5.1 Authentication and Authorization

CareerFit uses stateless bearer authentication and combines role checks with service-level ownership rules. Public access is limited to explicitly permitted operations, while Candidate, Recruiter, Administrator, and Automation workflows require the appropriate authenticated context.

Security headers, CORS restrictions, and HTTPS deployment reduce browser and transport risks but do not replace ownership checks or production controls. Development access to API documentation and metrics must be restricted in production.

### 3.5.2 Failure Handling

CV processing represents long-running work with statuses rather than keeping a browser request open until every score completes. Extraction or validation failure records a failure state and reason. Matching tasks start after commit so background work cannot read uncommitted rows. Report moderation uses transactions and row locking to update the target, report counter, report resolutions, and audit state together. A banned Job is excluded from active flows, while a banned CV can no longer remain the default CV.

Email can operate through SMTP or a no-op implementation. Notification delivery outcomes are recorded as sent, skipped, or failed, and policy guards enforce deduplication, quota, cooldown, and timing rules. Email failure must not roll back an already valid matching result. Conversely, application creation and status changes use transactions and uniqueness constraints because partial persistence would change recruitment state.

### 3.5.3 Identified Design Risks

*Table 3.9. Design risks and required treatment*

| <b>Risk</b>                                   | <b>Current control</b>                                                          | <b>Remaining treatment</b>                                                  |
|-----------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Unauthorized cross-account access             | URL roles and service ownership/visibility checks                               | Keep negative integration tests for identifier-based operations             |
| Duplicate applications, Matchings, or reports | Service checks plus unique constraints/indexes                                  | Convert database conflicts into stable API errors                           |
| False or abusive content reports              | Fixed reasons, one pending report per reporter-target, and Administrator review | Add rate limits, evidence rules, moderator assignment, and abuse monitoring |
| Incorrect moderation action                   | Case detail, explicit Ban/Dismiss, transactions, and audit events               | Add appeal, restoration, and dual-review policy for production              |
| Replayed email action                         | Hashed token, pending/redeemed state, expiry, and POST execution                | Retain replay, expiry, scanner, and rate-limit tests                        |
| Scheduler/configuration drift                 | Scheduler timing reads application properties                                   | Maintain configuration and schedule-boundary tests                          |

| <b>Risk</b>                         | <b>Current control</b>                       | <b>Remaining treatment</b>                                            |
|-------------------------------------|----------------------------------------------|-----------------------------------------------------------------------|
| Sensitive data in logs              | Structured audit metadata and DTO boundaries | Review retention, redaction, access, and exported evidence            |
| Lexical scoring bias or omission    | Reasons, validation, feedback, and HITL      | Evaluate representative data; never treat score as hiring probability |
| Local file-storage loss or exposure | Configured path and Docker volume            | Define backup, encryption, malware scanning, and retention            |

### 3.6 Deployment Architecture

The development deployment uses Docker Compose for PostgreSQL and optionally the backend. PostgreSQL 16 exposes container port 5432 as host port 5433 by default. A backend process running directly on the host connects to localhost:5433; the backend container connects through the Compose network to postgres:5432. Confusing these two addresses is a common configuration failure.

The optional backend container exposes port 8080, waits for PostgreSQL health, and mounts persistent CV storage. The React/Vite frontend runs separately during development. Environment variables provide database, JWT, URL/CORS, mail, OCR, and storage configuration; the backend image includes the project's Vietnamese/English Tesseract setup.

Actuator exposes health and Prometheus metrics according to the current security configuration. PostgreSQL health checks gate backend container startup. This architecture is appropriate for reproducible local development and demonstration, but it is not by itself a production topology. A production design would additionally require managed secrets, TLS termination, restricted management endpoints, database backup and recovery, protected object storage, centralized logs, monitoring alerts, and explicit scaling and retention policies.

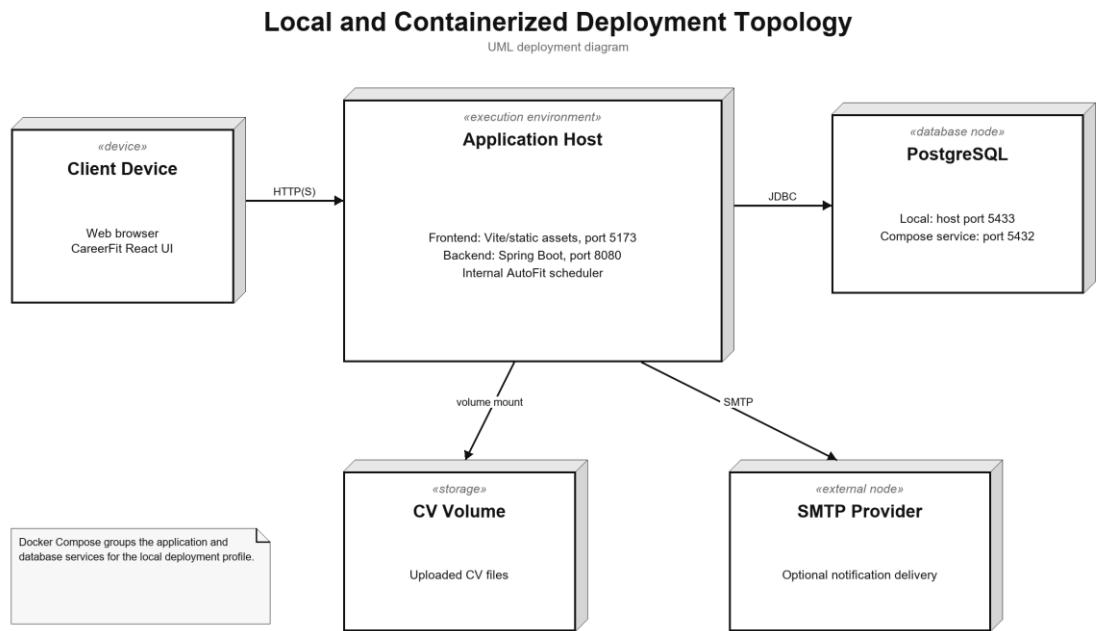


Figure 3.23. Local and containerized deployment topology

### 3.7 Implementation Overview

CareerFit consists of a Java 21/Spring Boot 3.2.5 backend and a React 18/TypeScript frontend built with Vite. PostgreSQL, Flyway, and Docker Compose provide persistence and the reproducible local runtime. Domain packages share the modular-monolith process described in Section 3.1.

Table 3.10. Main implementation technologies

| Layer                    | Technology                     | Implementation purpose                                                              |
|--------------------------|--------------------------------|-------------------------------------------------------------------------------------|
| Backend language/runtime | Java 21                        | Domain logic, API, background processing, and integrations                          |
| Application framework    | Spring Boot 3.2.5              | Dependency injection, web MVC, configuration, scheduling, and application lifecycle |
| Security                 | Spring Security and JWT 0.12.5 | Stateless bearer authentication, role authorization, and JWT handling               |



| Layer                | Technology                                                | Implementation purpose                                                               |
|----------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------|
| Persistence          | Spring Data JPA, PostgreSQL 16, Flyway                    | Entity persistence, queries, constraints, indexes, and versioned migrations          |
| Document processing  | PDFBox 3.0.2, Apache POI, Tesseract CLI                   | PDF/DOCX extraction and OCR fallback for images or scanned PDFs                      |
| API documentation    | springdoc-openapi 2.5.0                                   | OpenAPI documents and Swagger UI                                                     |
| Monitoring           | Spring Actuator, Micrometer Prometheus registry           | Health and HTTP/application metrics                                                  |
| Frontend             | React 18.3, TypeScript 5.9, React Router 7, React Query 5 | Role-based pages, typed API integration, navigation, and server-state queries        |
| Visualization and UI | Recharts and Lucide React                                 | Charts and interface icons                                                           |
| Build and test       | Maven, npm/Vite, JUnit, Testcontainers, Playwright        | Compilation, automated backend tests, integration environment, and browser E2E tests |

The backend package root is `com.careerfit.backend`. Domain packages cover authentication, career data, recruitment, Matching and Recommendation, Feedback, Automation and Notification, Reporting and Administration, Audit, and Settings. Shared configuration, security, validation, exceptions, and text utilities remain cross-cutting.

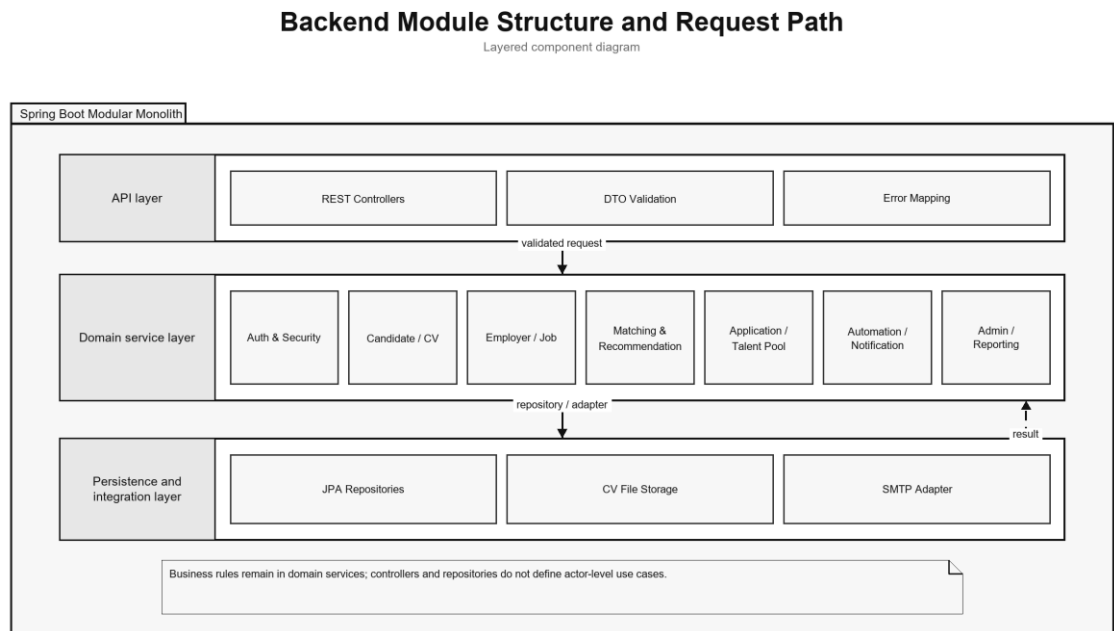


Figure 3.24. Backend module structure and request path

## 3.8 Authentication and Authorization Implementation

### 3.8.1 Login and JWT Processing

AuthController exposes registration, email/password login, and current-account operations. AuthService checks account state and credentials and delegates JWT creation to JwtService. Passwords are stored with BCryptPasswordEncoder. A successful login returns an access token and a user DTO with the ID, email, full name, role, and verification state. Passwordless login was removed; signed one-time tokens remain only for explicit email actions.

Each authenticated request passes through JwtAuthenticationFilter, a OncePerRequestFilter. The filter requires the Bearer prefix when an Authorization header is present, validates token structure and signature, extracts subject and role, rejects roles outside Candidate, Recruiter, and Administrator, and reloads the account from the database. Reloading prevents a cryptographically valid token from continuing to authorize a deleted or suspended account. The filter then creates a Spring Security authentication object with the corresponding ROLE\_\* authority.

UserIdResolutionFilter runs after JWT authentication. It resolves the persisted account ID from the authenticated email and stores the UUID as the userId request attribute. Controllers can therefore pass a stable identifier to services without accepting a user ID supplied by the browser. The implementation currently

performs a user lookup in both filters; this is straightforward but creates duplicate database reads that could be consolidated later.

### **3.8.2 URL Rules and Ownership Checks**

SecurityConfig uses a stateless session policy. Public access is limited to selected authentication operations, public Job/employer/analytics GET routes, similar-job retrieval, health/metrics/OpenAPI routes, and email-action redemption. Candidate and Recruiter report endpoints require authentication and are checked again by role and ownership in ContentReportService. Administrative report queue, detail, ban, and dismiss endpoints require the Administrator role.

Role checks are necessary but insufficient for resources addressed by UUID. ApplicationService, for example, verifies that a requested CV belongs to the authenticated Candidate and that a Recruiter owns the Job before listing applicants, changing status, or inviting a candidate. The same pattern is required across identifier-based services. Errors are returned through shared security and application error writers so that 401 unauthenticated and 403 forbidden responses remain distinguishable.

The configuration disables CSRF because bearer tokens are used instead of a server-side browser session. CORS origins are loaded from application configuration. Frame denial, Content Security Policy, and HSTS headers are configured. HSTS is effective only when the application is served through HTTPS, so the local HTTP runtime does not itself demonstrate transport security.

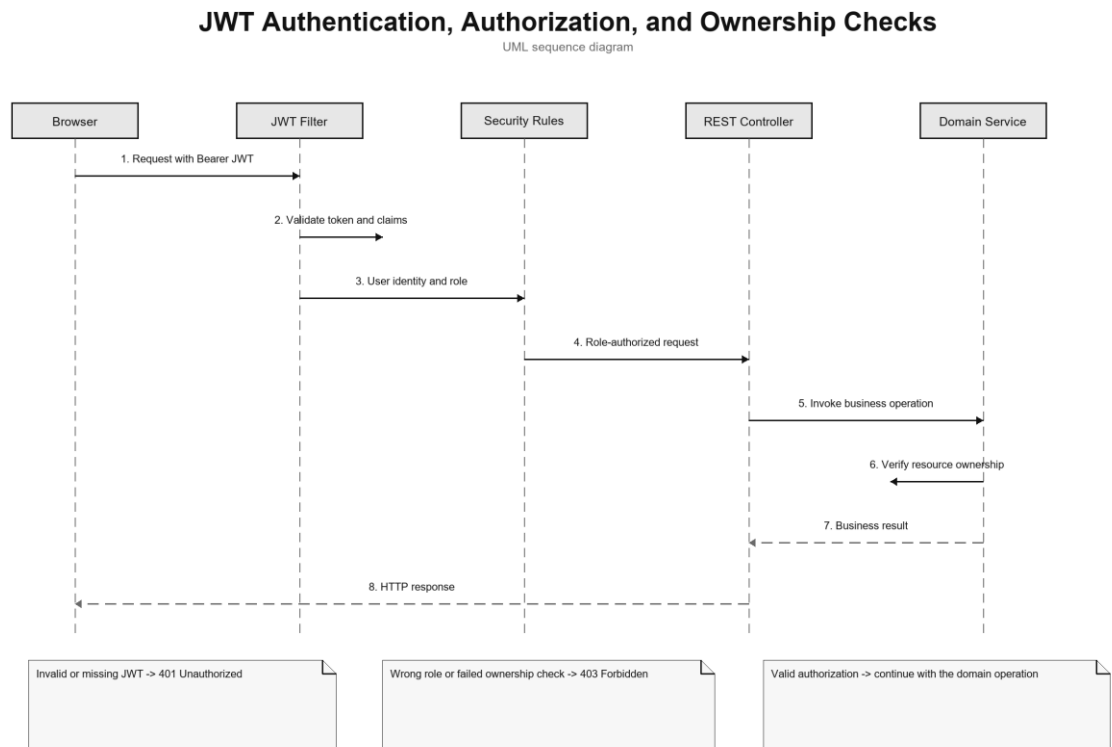


Figure 3.25. JWT authentication, role authorization, and ownership-check sequence

## 3.9 CV Ingestion and Validation

### 3.9.1 Upload and Manual Creation

CvController provides upload, manual creation, manual drafts, CV listing, details, processing status, review read/update, review confirmation, default selection, and deletion. CvIngestionService coordinates the workflow. Uploaded content is extracted and converted into reviewable sections. Manual entry can be saved as a DRAFT. The Candidate can correct sections and confirm them before vectorization and Job scoring start.

Table 3.11. CV processing states

| State      | Meaning                                               | Typical transition                         |
|------------|-------------------------------------------------------|--------------------------------------------|
| DRAFT      | Manual content is saved but not submitted for scoring | Draft edit → review/confirm                |
| UPLOADED   | Uploaded metadata and source are accepted             | Upload → validation/extraction             |
| VALIDATING | File, content, and extraction checks are running      | Valid source → review; hard error → failed |

| State           | Meaning                                                  | Typical transition                              |
|-----------------|----------------------------------------------------------|-------------------------------------------------|
| REVIEW_REQUIRED | Extracted/manual sections are ready for Candidate review | Edit sections → confirm                         |
| PROCESSING      | Confirmed content is normalized and vectorized           | Vector saved → asynchronous scoring             |
| SCORING_DONE    | The confirmed CV vector is available for matching        | Use cards, feedback, applications, and AutoFit  |
| FAILED          | Processing cannot continue                               | Failure reason is stored and audited            |
| BANNED          | An Administrator acted on a report case                  | Removed as default and excluded from normal use |

### 3.9.2 File Validation and Extraction

PdfExtractionService accepts PDF, PNG, JPG/JPEG, and DOCX. It checks extension, MIME type, and leading magic bytes. PDFBox extracts embedded PDF text, Apache POI extracts DOCX text, and images use OCR. Scanned pages are preprocessed before Tesseract runs, and OCR cleanup removes common noise before the text is split into review sections. The default OCR languages are vie+eng, with configured page and time limits.

Text shorter than 50 characters after extraction and cleanup is a hard failure. Text from 50 to 199 characters is accepted with a sparse-content warning. Encrypted PDFs are rejected, image dimensions are limited, and temporary OCR files are deleted. The review step also lets the Candidate fix extraction mistakes before the CV is used for scoring. Production use would still need malware scanning and stronger isolation.

QualityValidationService produces structured signals for questionable CV or JD content, such as missing sections, seniority/experience conflicts, salary/content conflicts, and invalid deadlines. Blocking errors are separated from warnings so users can fix the data instead of losing their work.

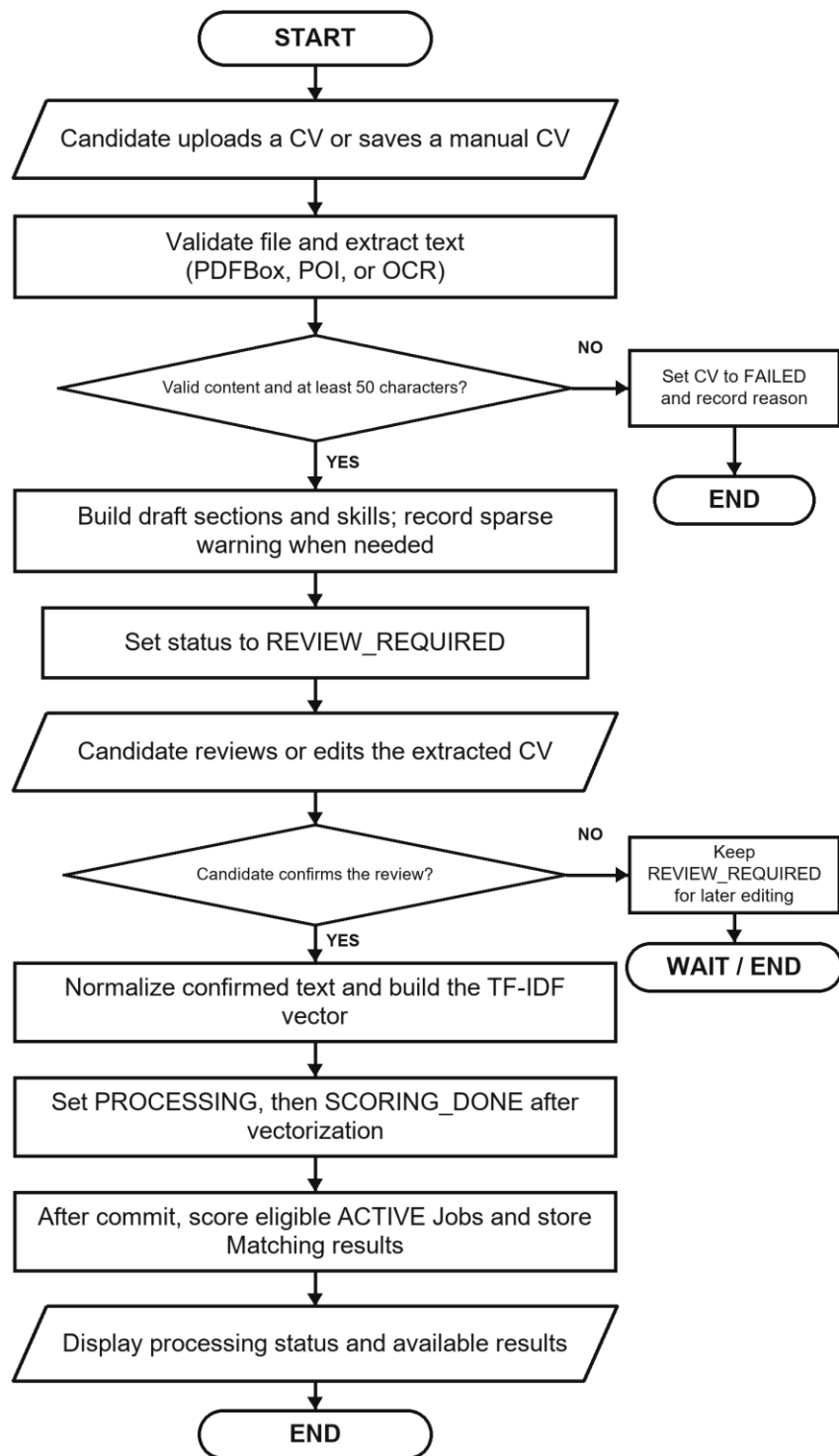


Figure 3.26. CV ingestion, review, confirmation, and matching flowchart

## 3.10 Text Normalization and TF-IDF Implementation

### 3.10.1 Text Normalization

TextNormalizationService removes HTML tags, replaces characters outside its English/Vietnamese alphanumeric pattern, lowercases text, splits on whitespace, removes tokens shorter than two characters, and filters a language-specific stop-word set. Language detection is a lightweight heuristic: text containing more than five Vietnamese diacritic characters is classified as Vietnamese; otherwise it is treated as English.

This implementation is deterministic and easy to inspect, but it is not a linguistic word segmenter. Vietnamese multi-syllable skills can be split into independent tokens, and punctuation-containing technology names such as C++, C#, .NET, or Node.js may lose distinguishing characters. These limitations are important when interpreting matching errors and motivate a future domain-aware tokenizer or controlled alias dictionary.

### 3.10.2 Static Corpus and Vector Construction

TfIdfService builds its IDF map once at application startup from a static seed corpus of representative IT terms. The corpus groups programming languages, frameworks, databases, cloud and DevOps technologies, architecture, testing, security, data/ML, collaboration, seniority, roles, and common Vietnamese IT words. A static corpus prevents scores from shifting whenever a user uploads one CV or adds one Job.

For a token list, term frequency is the token count divided by the total number of tokens. The implemented smoothed IDF is:

$$idf(t) = \log(1 + N / (1 + df(t))).$$

An unknown term receives  $\log(1 + N)$ , which treats it as rare and informative. The TF-IDF vector is persisted as JSON and reused during scoring. Cosine similarity iterates over the smaller vector for the dot product, computes both Euclidean magnitudes, and returns zero when either vector is empty.

The unknown-term policy has a practical trade-off. It preserves project-specific technology terms absent from the seed corpus, but a rare misspelling can receive the same high IDF treatment. Input validation, alias normalization, corpus versioning, and evaluation on realistic data are therefore necessary before treating the score as stable beyond the academic environment.

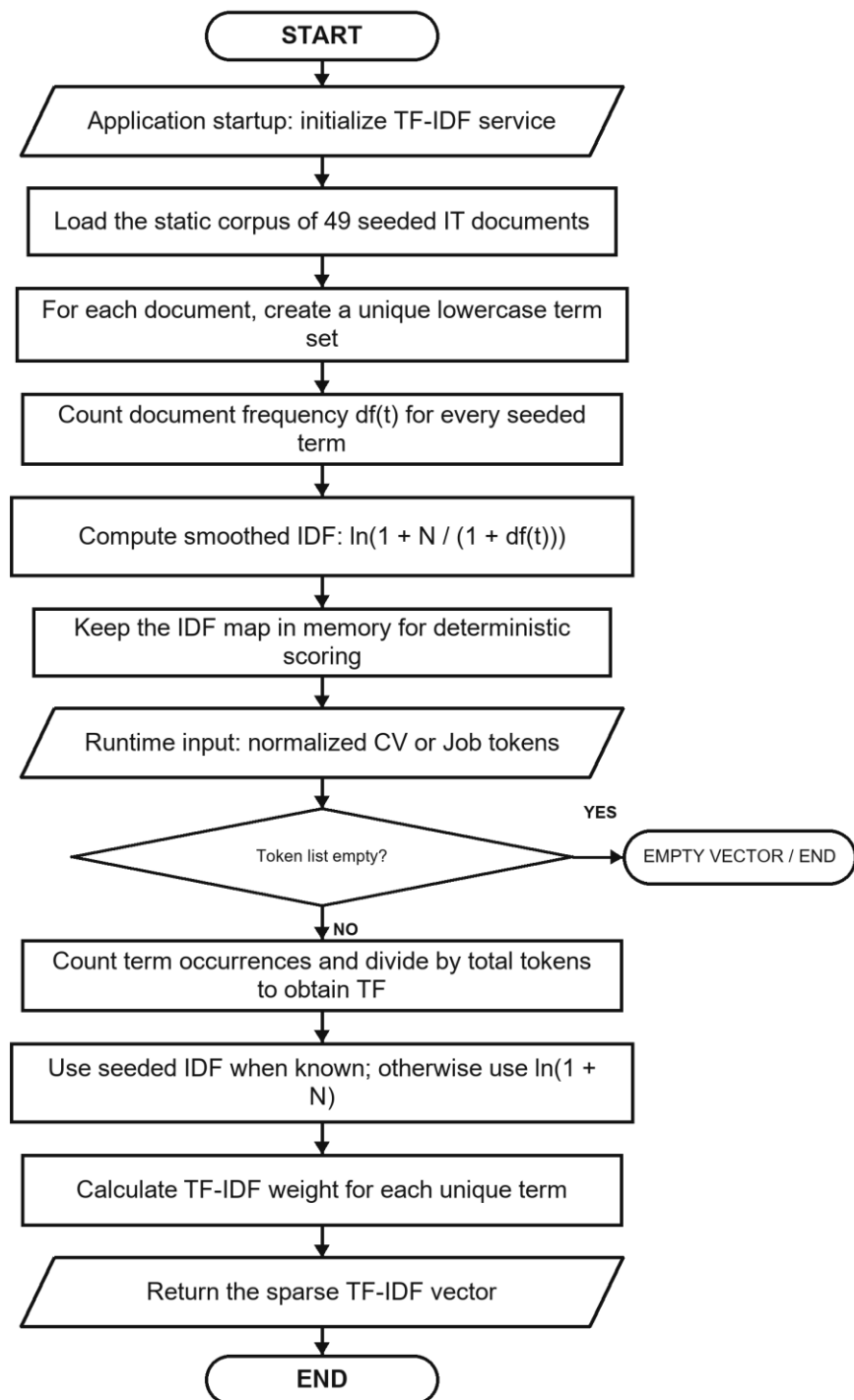


Figure 3.27. Seed-corpus initialization and TF-IDF construction flowchart



## 3.11 Matching and Recommendation Implementation

### 3.11.1 Scoring Service

ScoringService loads the CV vector from extractedTermsJson. For the Job, it prefers learnedProfileVectorJson when a non-empty learned vector exists; otherwise it uses the original tfidfVectorJson. Cosine similarity becomes the raw score, which is multiplied by 100 and rounded to two decimal places. The raw score is stored with six decimal places.

Table 3.12. Implemented score interpretation

| Condition                                                                                             | Stored/displayed result                       |
|-------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Direct score below 70                                                                                 | LOW label                                     |
| Direct score from 70 to below 90                                                                      | MEDIUM label                                  |
| Direct score at least 90                                                                              | HIGH label; Potential is not added            |
| Potential score at least 62 with skill compatibility $\geq 0.50$ and family compatibility $\geq 0.55$ | Potential guard continues                     |
| Core target and transferable career evidence exist, with no severe seniority gap                      | isPotential=true and an explanation is stored |
| Direct score below 20 and skill compatibility below 0.70                                              | Potential is rejected as weak evidence        |

The direct-match label uses the configured medium boundary (70) and high boundary (90), producing LOW, MEDIUM, or HIGH. Potential is a separate boolean and does not replace that label. The current Potential model is loaded from a versioned JSON knowledge base and is evaluated separately from cosine similarity.

Direct-match reasons use the highest-weight Job terms that also appear in the CV, with the Job domain added when available. Potential uses normalized aliases, direct skill-transfer links, role-family transfer, shared foundations, and seniority compatibility. Its explanation can list transferable skills and shared foundations. This remains a rule-based explanation, not a learned claim that the Candidate will succeed.

### **3.11.2 Matching Orchestration and Persistence**

After the Candidate confirms the CV review, vectorization is committed and `AfterCommitExecutor` submits `MatchingService.scoreAllJobsForCv`. The service reloads the CV, scores eligible active Jobs, and creates or updates the unique Matching row. Reasons and Potential metadata are stored as JSON. The same post-commit boundary is used when a published Job is scored against existing confirmed CVs.

A Recruiter first completes a company profile, then can save a Job as DRAFT, check JD quality, and publish it with the required deadline. Published Jobs can be marked urgent and are scored after commit. `MatchingQueryService` supports Candidate and Recruiter views with server-side filters, pagination, sorting, label/Potential filters, and clear empty-state metadata. Portfolio details remain protected by the Candidate's visibility setting and application state.

### **3.11.3 Recommendation Service**

`RecommendationService` produces Candidate Job recommendations from Profile/preferences and available Matching context, while similar-Job retrieval remains public. Results are bounded to eligible visible Jobs and may return a limited-data or empty state when suitable information is unavailable. Recommendation stays separate from Application: viewing or skipping a result changes no recruitment state, and creation requires an explicit request or a separately enabled `AutoFit` policy.

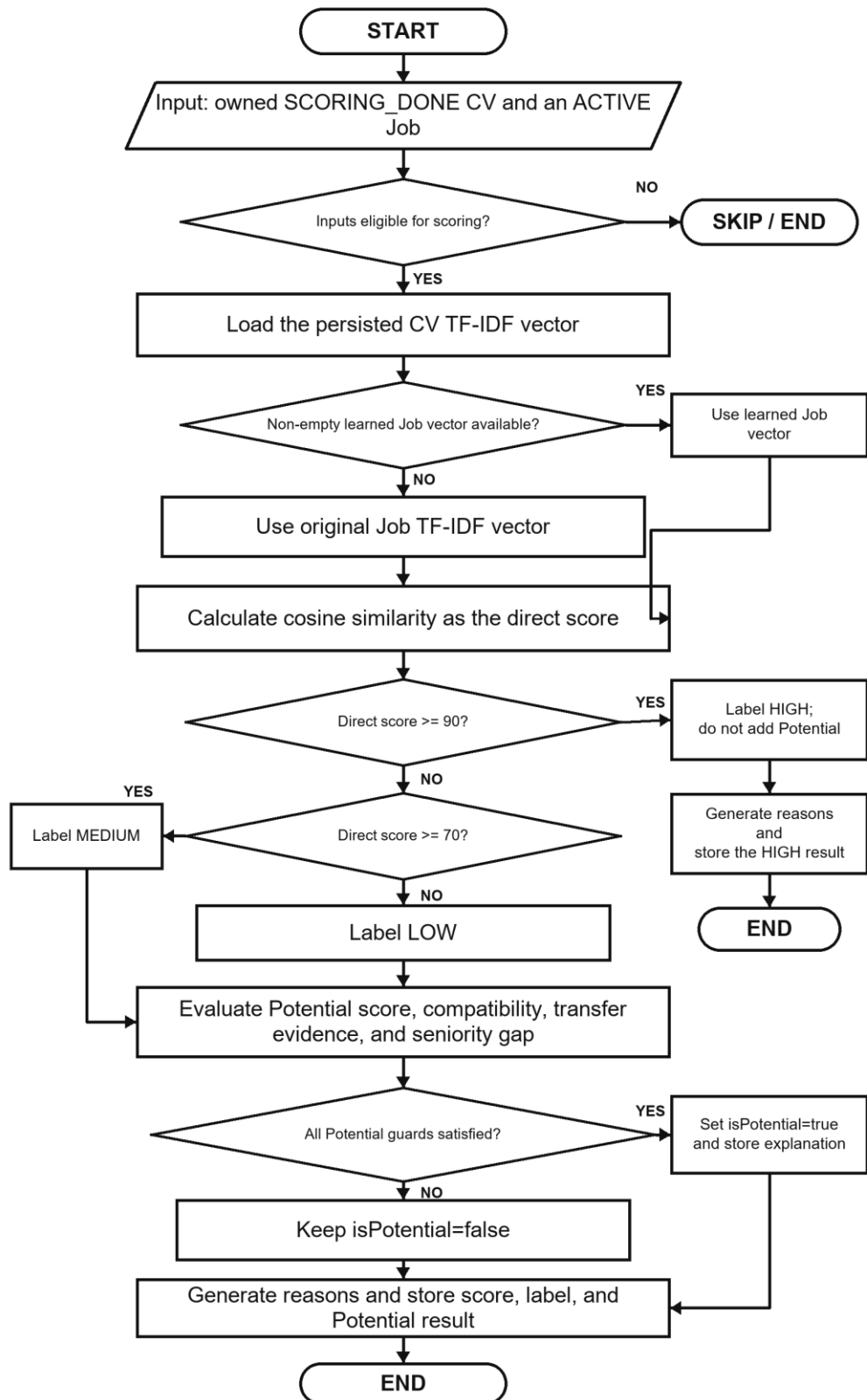


Figure 3.28. CV-Job matching and Potential assessment flowchart

### 3.12 Feedback Learning with Rocchio

FeedbackService.submitFeedback resolves the Matching and authenticated Candidate, verifies that the Candidate owns the associated CV, and then upserts feedback by (matching\_id, actor\_id). Concurrent uniqueness conflicts are resolved by reloading and updating the existing row. Each event stores actor role, feedback type, and source channel and creates an audit entry. NOT\_INTERESTED is stored but does not trigger Rocchio because it can represent preference rather than technical relevance. GOOD\_MATCH, POTENTIAL, and BAD\_MATCH trigger post-commit Job-vector learning.

RocchioService locks the Job for update and always loads the original Job TF-IDF vector as  $q$ . Positive examples are CV vectors attached to GOOD\_MATCH or POTENTIAL feedback; negative examples come from BAD\_MATCH. The service computes term-wise centroids and applies:

$$q_{new} = 1.0q + 0.75 \text{ positive\_centroid} - 0.15 \text{ negative\_centroid}.$$

Negative weights are removed rather than persisted. The learned vector is stored in learnedProfileVectorJson. Every Matching associated with the Job is then marked needsRecompute=true. AutomationScheduler.recomputeStaleMatchings rescans these rows every 30 minutes and clears the marker only after successful scoring.

Using the original base vector and the complete feedback history makes recomputation repeatable for the same data and parameters. FeedbackService starts learning after the transaction commits, so Rocchio reads the saved feedback instead of racing the transaction that created it. Job locking prevents two updates from writing separate learned vectors at the same time, and integration tests cover this post-commit behavior.

Table 3.13. Feedback handling

| Feedback       | Persisted | Rocchio classification | Immediate learning trigger |
|----------------|-----------|------------------------|----------------------------|
| GOOD_MATCH     | Yes       | Positive               | Yes                        |
| POTENTIAL      | Yes       | Positive               | Yes                        |
| BAD_MATCH      | Yes       | Negative               | Yes                        |
| NOT_INTERESTED | Yes       | Neither                | No                         |

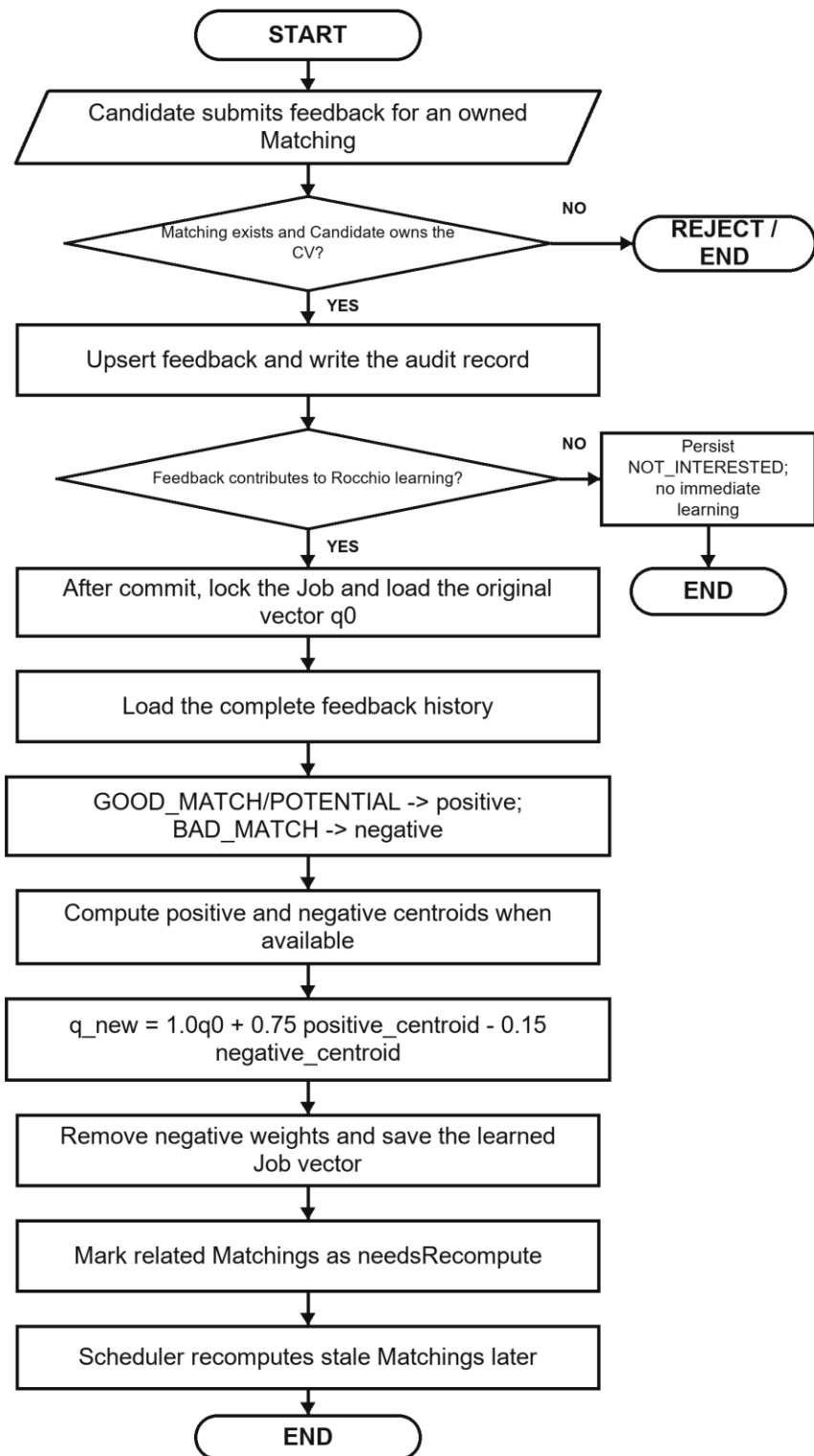


Figure 3.29. Rocchio feedback-learning and recomputation flowchart

### 3.13 Application and Recruiter Workflow

`ApplicationService.submit` resolves the authenticated Candidate, requires an ACTIVE Job, rejects an existing Candidate–Job Application, and uses the requested owned CV or owned default CV. The current service does not enforce `applicationDeadline` or require a manually selected CV to be `SCORING_DONE`. If a Matching exists, it is attached as score context. The accepted Application is made available to the Candidate and owning Recruiter.

A Candidate views owned applications through a paginated API and can filter them by status. A Recruiter can list applicants only for an owned Job, update valid states, invite a Candidate, and report a visible CV in the context of an owned Job. The Talent Pool adds Job-based Candidate discovery, a Potential-only view, and CV bookmarks. A Candidate can report an active Job from the Job interface.

Response DTOs combine application, Candidate, CV, Matching, Job, and bookmark details without exposing persistence entities. Portfolio fields are returned only when the Candidate has enabled visibility and the application state permits it. Server-side pagination, sorting, and filters keep larger Job and Candidate catalogues manageable.

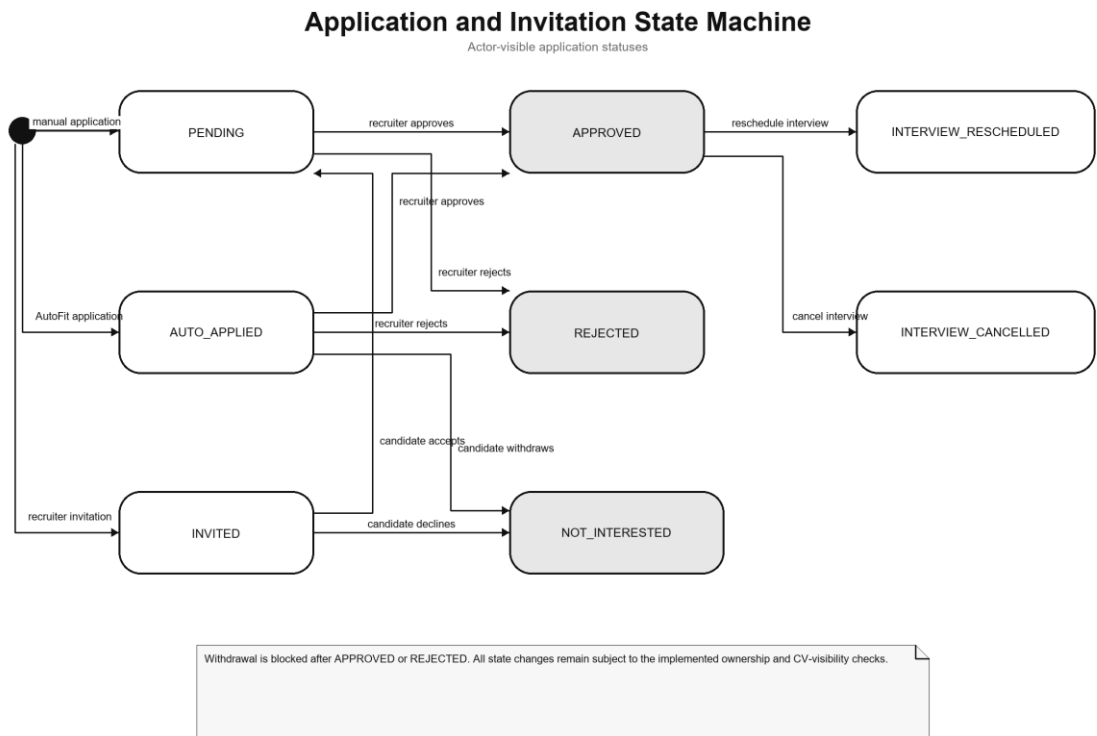


Figure 3.30. Application and invitation UML state machine

## 3.14 AutoFit and Background Processing

### 3.14.1 Async Executor and Scheduler

`AsyncConfig` creates a bounded `ThreadPoolTaskExecutor`. `AfterCommitExecutor` submits CV and Job matching only after the related transaction commits. Mail sending and Rocchio learning can also leave the request thread, while persisted CV, Matching, email-action, and delivery states make completion and failures observable.

`AutomationScheduler` coordinates matching recomputation, digests, high-match checks, auto-apply, token cleanup, and reminders using system-configured schedules. Per-item error handling prevents one failed record from stopping a full scan. Scheduled execution is internal CareerFit behavior and is not modeled as an external UML actor.

### **3.14.2 Auto-Apply**

AutoApplyService.runForPolicy resolves the policy owner and Candidate, then requires an owned default CV in SCORING\_DONE status. It selects Matchings for ACTIVE Jobs at or above the configured threshold, skips an existing Candidate–Job Application, and creates at most three AUTO\_APPLIED Applications per run. The current execution does not use applicationDeadline, notification quota, cooldown, quiet hours, or notification preferences as application-eligibility rules.

Auto-apply and notification delivery use related but separate checks. Application creation requires consent and application-policy conditions, while email delivery also checks timing, quota, cooldown, category, and deduplication. Keeping the two decisions separate prevents a notification setting from silently becoming permission to submit an application.



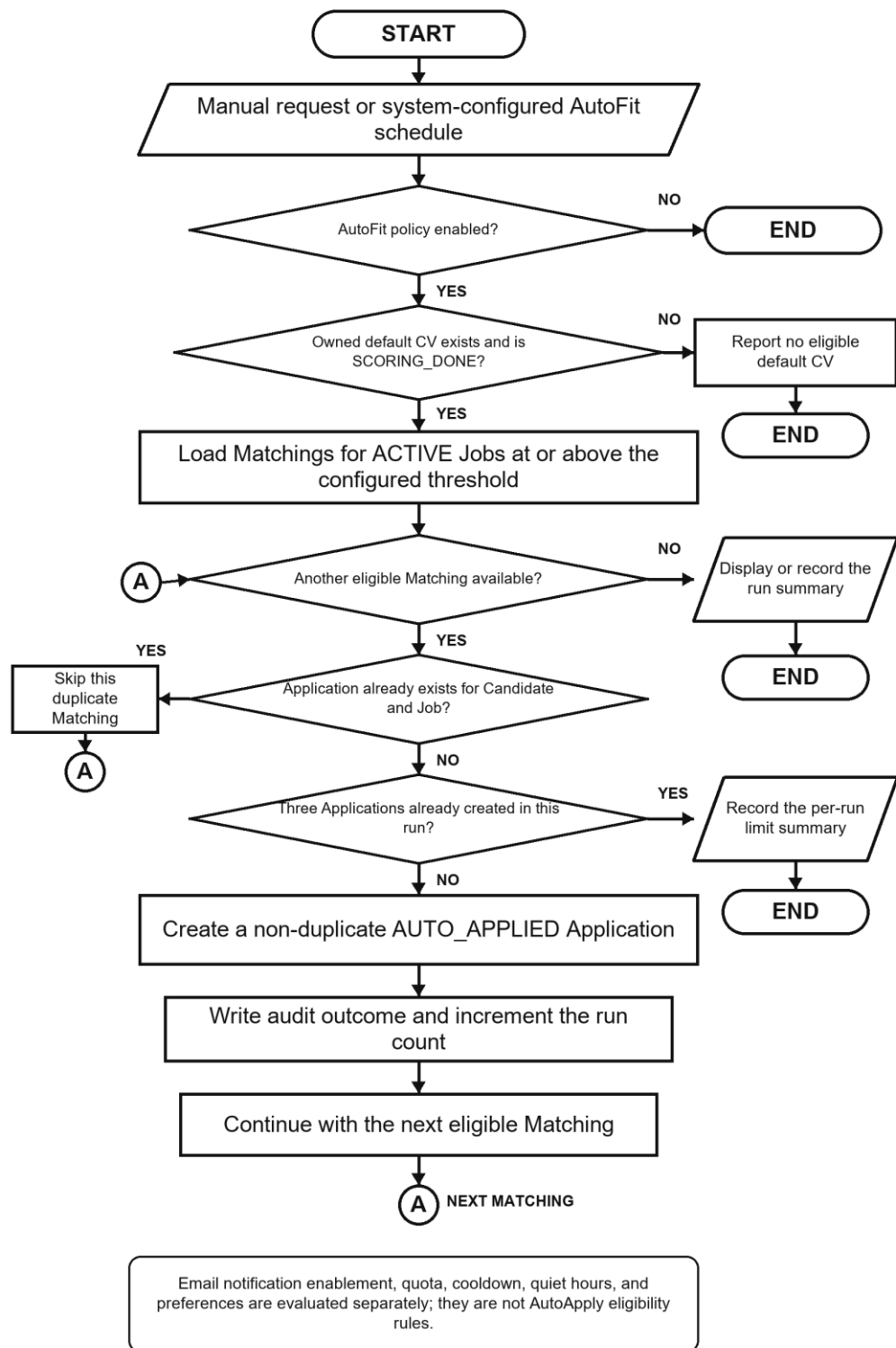


Figure 3.31. AutoFit eligibility and application decision flowchart

### **3.14.3 Notification Guard and Delivery**

NotificationPolicyGuard evaluates email delivery separately from AutoFit application eligibility. It applies email enablement, email-type preferences, deduplication, daily quota, cooldown, and quiet hours, and records sent, skipped, or failed delivery outcomes. SMTP is used when enabled, while NoOpMailService supports local development without claiming production delivery evidence.

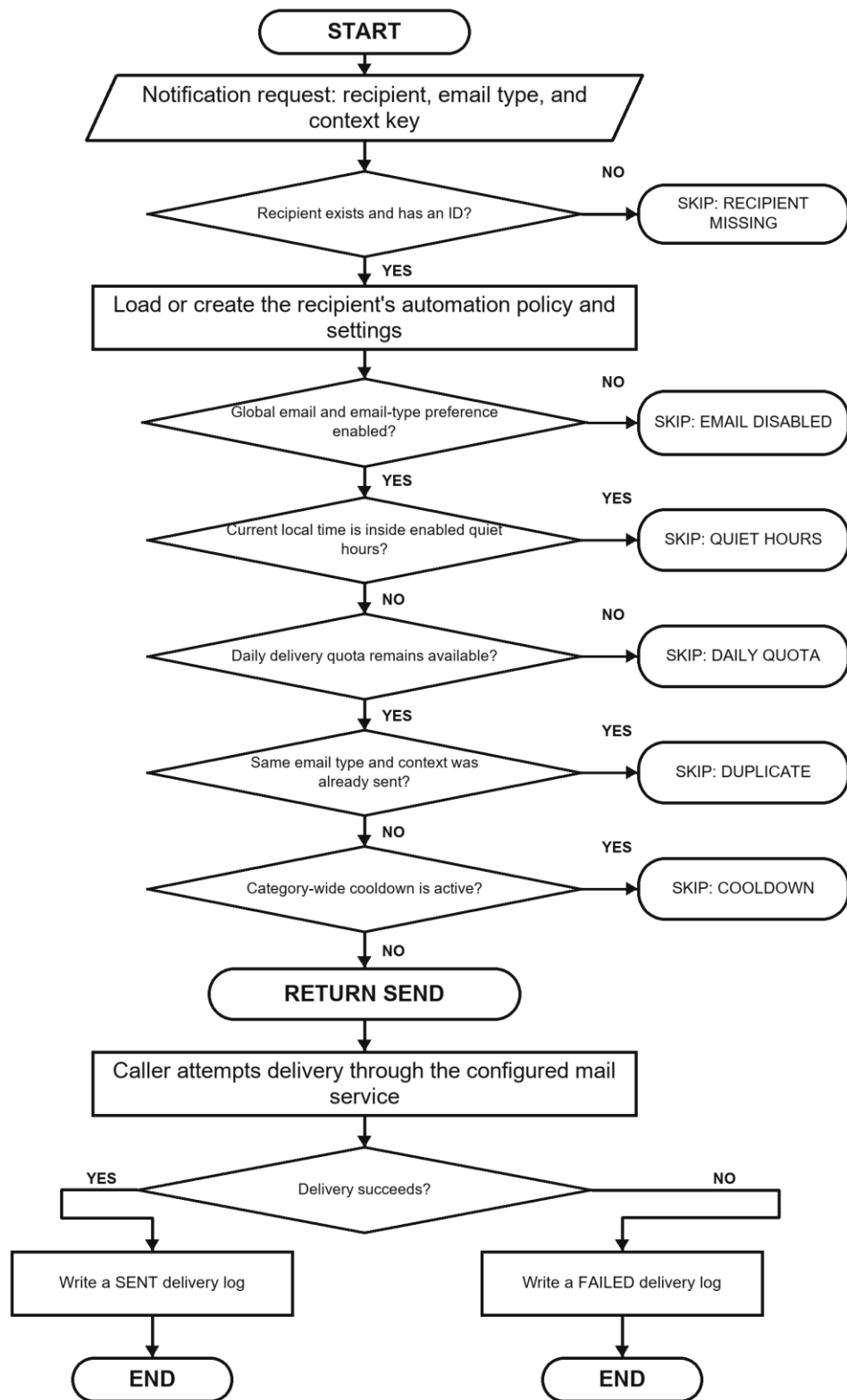


Figure 3.32. Notification policy evaluation and delivery outcome flowchart

### **3.15 Email Action and Audit Implementation**

EmailActionService generates a 32-character token derived from UUID text and stores the related email-action state with the recipient, optional Matching or Application, action type, and expiry. The current successful actionable-email scope covers Matching Feedback and Recruiter invitation responses. Although VIEW\_JOB and UNSUBSCRIBE\_DIGEST action types exist, the current VIEW\_JOB branch does not redirect and the unsubscribe branch does not update the Candidate policy, so they are not documented as successful UC-07 behavior. Tokens remain valid for 72 hours.

EmailActionController exposes a public confirmation and redemption flow because possession of the high-entropy token is the credential. GET validates the token and displays a non-mutating confirmation page. POST validates it again, performs a supported action, marks a successful action REDEEMED, and records EXPIRED when an expired action is submitted for confirmation. Invalid or already redeemed actions do not change the referenced business state.

This implementation removes the earlier state-changing GET and raw-token persistence findings. The remaining production concerns are rate limiting, deployment-specific link origin, secret rotation, mail-delivery monitoring, clearer user-facing error messages, and protected audit review.

Audit records are written directly through AuditLogRepository in major services. Entries can contain actor type/ID, action, target, result, source channel, and JSON metadata. Examples include CV failure, application submission, invitation, feedback, auto-apply, JOB\_REPORTED, CV\_REPORTED, report dismissal, and banning a Job or CV. Direct repository calls are simple, but a centralized audit facade could better standardize metadata, redaction, and failure behavior.

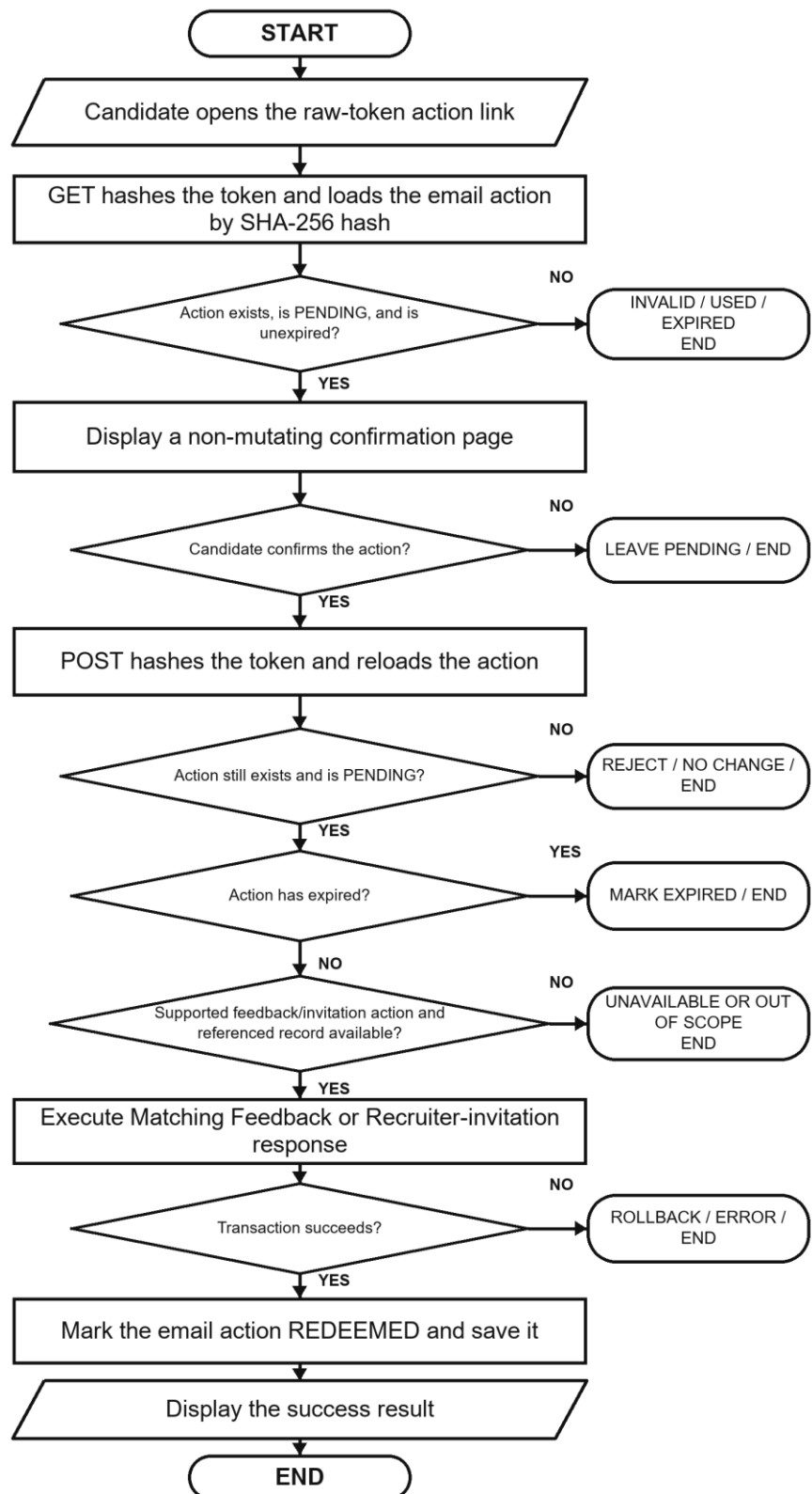


Figure 3.33. Actionable-email confirmation and redemption flowchart

### 3.16 Persistence and API Implementation

Flyway V1–V25 defines the schema, and Hibernate validates the mappings. V25 adds content\_report, moderation indexes and counters, the unique pending-report rule, and BANNED Job/CV states.

Database constraints are treated as correctness boundaries rather than only documentation. Unique indexes protect email identity, default CV, Matching, Application, imported Job hash, and duplicate pending reports by the same reporter and target. Check constraints restrict roles, labels, lifecycle states, report target/reason/status, source channels, and salary modes. Version columns and explicit row locks help protect mutable core entities.

Controllers return validated DTOs in shared API envelopes. Service exceptions distinguish not found, forbidden, bad request, and conflict outcomes; springdoc publishes OpenAPI. Catalogue endpoints use pagination or bounded queries instead of unbounded payloads.

*Table 3.14. Representative API-to-service mapping*

| <b>API operation</b>                                       | <b>Controller/service responsibility</b>           | <b>Persistence effect</b>                        |
|------------------------------------------------------------|----------------------------------------------------|--------------------------------------------------|
| POST /api/cv/upload; GET/PATCH /api/cv/{id}/review         | Extract content and expose/edit review sections    | CV, review data, quality signals, audit          |
| POST /api/cv/{id}/review/confirm                           | Confirm owned CV and start matching                | CV vector, Matchings, audit                      |
| GET /api/matches/me/cards; POST /api/matches/{id}/feedback | Query Candidate cards and submit feedback          | Feedback, audit, learned Job vector, stale flags |
| POST /api/jobs/drafts; POST /api/jobs/{id}/publish         | Validate company/JD/deadline and manage owned Job  | Job, vectors, Matchings, audit                   |
| PUT /api/recruiter/talent/.../bookmark                     | Validate Job ownership and save Talent Pool choice | Recruiter CV bookmark                            |
| PATCH /api/automation/policy                               | Update the account's automation policy             | Automation policy and version                    |

| API operation                                                | Controller/service responsibility                                                              | Persistence effect                               |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------|
| POST /api/reports/jobs/{id}; POST /api/reports/cvs/{id}      | Validate role, target state, Job ownership/CV visibility, reason, and duplicate pending report | ContentReport, pending report count, audit       |
| GET /api/admin/reports; POST .../{type}/{id}/ban or /dismiss | Load report cases and resolve all pending reports for a target                                 | Target status/counter, report resolutions, audit |

### 3.17 Frontend Integration

Section 3.4 presents selected role-based interfaces and their actor-visible sequences. At the integration level, App.tsx defines public, Candidate, Recruiter, and Administrator routes. Protected routes check the loaded account role, while session restoration calls /api/auth/me and clears invalid state. Candidate Job reporting, Recruiter CV reporting, and Administrator Report moderation remain protected by role, ownership, and target-visibility checks.

Frontend/src/lib/api.ts centralizes HTTP access. It reads VITE\_API\_BASE\_URL with /api as the default, attaches the bearer token from sessionStorage, selects JSON headers except for multipart FormData, unwraps the shared response envelope, and throws a normalized error for unsuccessful responses. DTO mapping functions convert backend shapes and labels into UI models. React Query hooks manage loading, refetching, caching, polling, and error state for server data.

The frontend stores the access token and account summary in sessionStorage, limiting persistence to the current browser tab. This still exposes the token to any successful cross-site scripting attack. The Content Security Policy reduces some exposure, but production hardening should evaluate short-lived tokens, refresh rotation, and HttpOnly secure-cookie alternatives according to the deployed architecture.

API-driven pages use server-side filtering and pagination and display real errors instead of mock Job data. Reporting uses fixed reasons, optional comments, pending-state badges, and an Administrator moderation queue. Detailed interfaces appear in Section 3.4 and their workflows are evaluated in Chapter 4.

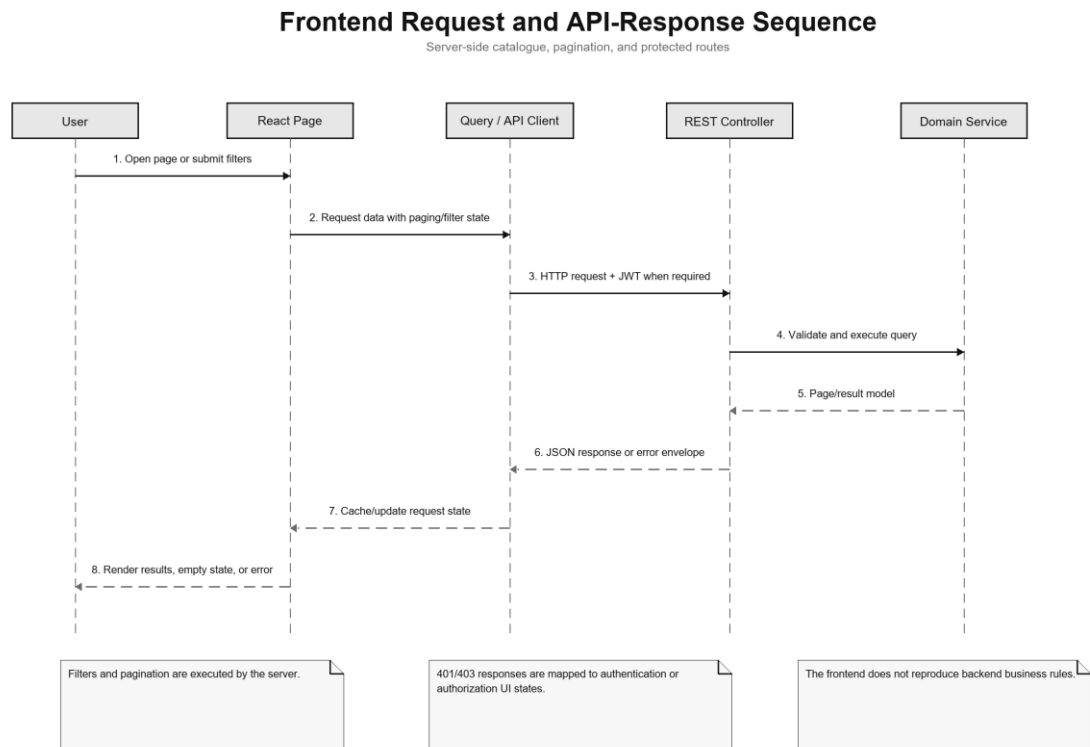


Figure 3.34. Frontend request and API-response sequence

### 3.18 Deployment and Observability Implementation

The implementation follows the deployment topology in Section 3.6. Environment variables select host or container database addresses and configure JWT, CORS, application URL, mail, OCR, and CV storage. The Vite development server calls /api or a configured backend base URL.

Actuator exposes health and Prometheus endpoints. Micrometer records HTTP request metrics with the application name tag and latency histograms. Console logs include the timestamp, thread, level, request ID when available, logger, and message. These features provide basic monitoring data, but an available endpoint does not by itself prove that the monitoring system works. Chapter 4 must separately check reachability, access rules, Prometheus collection, and dashboard or alert evidence.

Maven builds the backend and can execute JUnit, Spring Security, and Testcontainers tests. The frontend build runs TypeScript checking before Vite bundling. Playwright configurations support local and production-oriented E2E runs. Build and test commands, versions, environment, failures, and generated artifacts must be recorded when Chapter 4 results are finalized.



### 3.19 Implementation Limitations

Table 3.15. Verified implementation limitations

| Area               | Current limitation                                                                                                                                                           | Consequence or required improvement                                                                                                       |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Content moderation | Ban/Dismiss exists, but appeal and restoration do not                                                                                                                        | Add moderator workflow, appeal, evidence retention, and abuse-rate controls                                                               |
| Email actions      | Hashed token and confirm-then-POST flow                                                                                                                                      | Add rate limiting, delivery monitoring, and deployment origin controls                                                                    |
| CV/OCR             | Preprocessing and cleanup are heuristic                                                                                                                                      | Scanned layouts and low-quality images still need user correction                                                                         |
| Text processing    | Simple tokenization remains the direct-score baseline                                                                                                                        | Technology spelling and Vietnamese phrases may lose information                                                                           |
| Potential model    | Aliases and transfer weights are manually defined                                                                                                                            | Validate rules with recruiter-labeled data and review drift                                                                               |
| Automation         | Auto-application and notification policy are implemented through separate execution paths; Application deadline is stored but not enforced by manual or automatic submission | Add deadline enforcement and focused tests; keep notification quota, cooldown, and quiet hours separate from auto-application eligibility |
| Frontend session   | JWT and account summary use sessionStorage                                                                                                                                   | Evaluate refresh/cookie architecture and XSS controls                                                                                     |
| File storage       | Local path or Docker volume                                                                                                                                                  | Add malware scanning, encryption, backup, retention, and object storage                                                                   |

| Area  | Current limitation                         | Consequence or required improvement                                       |
|-------|--------------------------------------------|---------------------------------------------------------------------------|
| Audit | Several services write audit rows directly | Centralize redaction, request context, schema rules, and integrity policy |

These limitations are included because the purpose of an implementation chapter is to explain the system that exists, not an idealized version. They also provide concrete hypotheses and negative cases for Chapter 4.

### 3.20 Chapter Summary

This chapter presented the CareerFit architecture, modules, data model, functional design, security and consistency design, deployment topology, and implementation in Spring Boot, React, PostgreSQL, and Flyway. It covered JWT security, CV review, direct matching, the Potential assessment, feedback learning, applications, Talent Pool, AutoFit, email actions, Job/CV reporting, administrator moderation, frontend integration, and monitoring. Chapter 4 evaluates the refreshed system.

## CHAPTER 4. TESTING AND EVALUATION

### 4.1 Evaluation Objectives

The evaluation addressed four questions: (1) whether lexical scoring with Rocchio moves holdout items in the intended direction after feedback; (2) whether the backend modules, migrations, and selected contracts pass automated tests; (3) whether the main Guest, Candidate, Recruiter, and Administrator workflows execute through the integrated browser application; and (4) what the local environment shows about authorization, runtime health, monitoring, and API latency. Algorithm behavior, software correctness, browser workflows, security observations, and runtime evidence were assessed separately because success in one area does not establish success in the others. In particular, neither a controlled ranking improvement nor a passing software suite establishes production readiness or real-world hiring effectiveness.

### 4.2 Evaluation Environment

*Table 4.1. Evaluation environment*

| Component                  | Observed version or configuration                                      |
|----------------------------|------------------------------------------------------------------------|
| Host operating environment | Windows, PowerShell                                                    |
| Java                       | 21.0.1                                                                 |
| Backend framework          | Spring Boot 3.2.5                                                      |
| Maven                      | Repository Maven Wrapper; clean verify                                 |
| Node.js / npm              | Bundled Node runtime with repository npm scripts                       |
| Frontend build             | Vite 6.4.3, React 18.3, TypeScript 5.9, ESLint                         |
| Docker / Compose           | Docker Desktop with Compose                                            |
| Test database              | PostgreSQL 16 through Testcontainers; local Compose PostgreSQL for E2E |
| Database migrations        | Flyway V1–V25                                                          |
| Browser E2E                | Playwright 1.61 using installed desktop Chrome                         |
| Integrated backend         | Compose backend on port 8080                                           |
| Integrated frontend        | Vite development server on 127.0.0.1:5173                              |

Backend integration tests used isolated PostgreSQL 16 Testcontainers with Flyway migrations, while browser tests exercised the local Docker-based CareerFit

stack through desktop Chrome. The controlled algorithm dataset was evaluated separately from the mutable local database used for E2E and latency observations. Table 4.1 records the relevant versions and configurations; these values describe the evaluated environment rather than a production deployment.

This separation prevents seeded local runtime data from being interpreted as algorithm ground truth.

## 4.3 Dataset and Evaluation Design

### 4.3.1 Controlled Algorithm Dataset

AlgorithmEvaluatorTest used a controlled synthetic dataset containing 50 Jobs, 100 CVs, 300 training pairs, and 300 holdout pairs. Training feedback and holdout evaluation were separated: feedback from a training CV updated the Job vector, while metrics were calculated from different holdout pairs. The scenario intentionally placed a shared latent skill in related training and holdout CVs so that the test could determine whether Rocchio moved a relevant unseen item upward.

Before feedback, the system ranked CVs with the original lexical Job vectors. Training-pair feedback then changed the learned vectors and triggered Matching recomputation; evaluation compared the resulting holdout order rather than reusing training pairs as the success cases.

This design evaluates a specific causal behavior of the feedback loop. It does not reproduce organic recruiter judgments, demographic variation, adversarial CVs, changing market language, or naturally unbalanced applicant pools. The designed relationship can produce a large improvement; therefore, the result must not be interpreted as expected performance on real recruitment data.

### 4.3.2 Local Runtime Data

The integrated environment used Flyway seed data, imported Jobs, and records created by E2E tests. This data supports complete workflows and small local runtime observations but has no independent relevance labels. It was therefore not used to calculate ranking accuracy, fairness, or hiring quality.

## 4.4 Backend Automated Testing

The complete backend suite was executed with the Maven Wrapper using clean verify, including PostgreSQL Testcontainers and Flyway migrations.

*Table 4.2. Backend automated test results*

| Measure     | Result |
|-------------|--------|
| Test suites | 35     |
| Tests       | 141    |

| Measure                        | Result                            |
|--------------------------------|-----------------------------------|
| Failures                       | 0                                 |
| Errors                         | 0                                 |
| Skipped                        | 0                                 |
| Aggregated Surefire suite time | 164.516 s                         |
| Maven lifecycle                | clean verify and JAR build passed |

The 35 suites cover authentication, CV and Job lifecycles, Matching, Recommendation, Feedback, AutoFit, Applications, Talent Pool, analytics, reporting, moderation, and the controlled algorithm experiment.

#### 4.5 Controlled Algorithm Results

The benchmark applied Rocchio with  $\alpha=1.0$ ,  $\beta=0.75$ , and  $\gamma=0.15$ . Coverage remained 1.0 before and after feedback.

*Table 4.3. Baseline and Rocchio results on the synthetic holdout scenario*

| Metric      | Baseline | After Rocchio | Delta     |
|-------------|----------|---------------|-----------|
| Precision@5 | 0.012000 | 0.172000      | +0.160000 |
| Recall@5    | 0.060000 | 0.860000      | +0.800000 |
| nDCG@3      | 0.030000 | 0.830000      | +0.800000 |
| nDCG@5      | 0.037737 | 0.837737      | +0.800000 |
| nDCG@10     | 0.050424 | 0.850424      | +0.800000 |
| MRR         | 0.058755 | 0.842665      | +0.783910 |
| HitRate@5   | 0.060000 | 0.860000      | +0.800000 |
| Coverage    | 1.000000 | 1.000000      | 0         |

Position-sensitive metrics improved strongly after the designed feedback. nDCG@5 increased from approximately 0.038 to 0.838, while Recall@5 and HitRate@5 increased from 0.06 to 0.86. Precision@5 reached 0.172, so many top-five items were still not labeled relevant. The feedback loop therefore moved related holdout CVs upward without making every result relevant.

### Controlled benchmark: baseline versus Rocchio

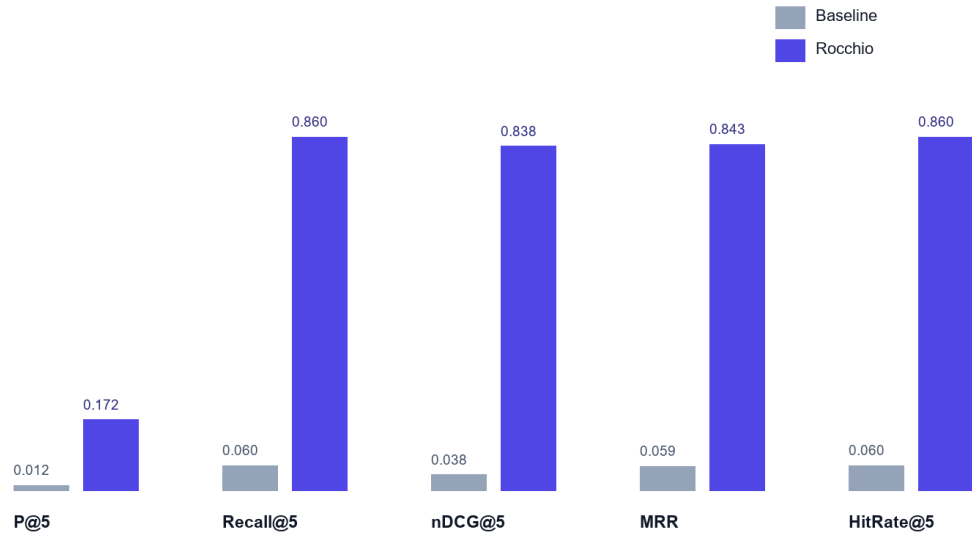


Figure 4.1. Baseline and Rocchio benchmark metrics at  $K = 5$

The large delta is expected because the dataset contains a designed latent relationship. It supports the implemented Rocchio behavior but does not estimate real recruitment outcomes.

Coverage remaining 1.0 indicates that improvement was not obtained by excluding difficult queries or shrinking the evaluated result set. Repeated runs with fixed data and parameters reproduced the same metrics.

#### 4.6 Concurrency Observation in Benchmark Runs

The final isolated benchmark completed without the background consistency conflict observed in earlier runs. Feedback learning begins after transaction commit and prior Matching state is cleared before recomputation, which made the evaluated run stable and deterministic. This does not establish production concurrency safety; concurrent-load recovery, retry, timeout, and conflict behavior remain unevaluated.

#### 4.7 Frontend Build Evaluation

TypeScript checking, ESLint, the Vite production build, and bundle validation all passed. The build transformed 2,362 modules, and its largest JavaScript chunk remained below Vite's warning threshold. These results confirm build correctness for the evaluated source; they do not constitute browser performance or user-experience measurement.

#### 4.8 Browser End-to-End Evaluation

Playwright exercised the integrated frontend, backend, PostgreSQL database, and desktop Chrome. All 49 tests passed.

Table 4.4. Integrated Chrome workflow results

| Scenario group            | Main verification                                                                                | Result |
|---------------------------|--------------------------------------------------------------------------------------------------|--------|
| Guest and authentication  | Public catalogue/detail, login, session restoration, and role routes                             | Passed |
| Candidate CV and matching | Draft, upload/review/confirm, matching, filters, and feedback                                    | Passed |
| Candidate recruitment     | Urgent Jobs, apply/withdraw, settings, AutoFit, and active-Job report                            | Passed |
| Recruiter workflow        | Company, Job draft/publish, applicants, Talent Pool, bookmark, invitation, and visible-CV report | Passed |
| Administration            | Suspend/reactivate, audit, report queue/detail, and ban action                                   | Passed |
| Resilience and interface  | API errors, navigation, account state, status labels, and dark mode                              | Passed |

The 49 tests cover representative workflows across all four user contexts, including reporting and moderation. They provide automated workflow evidence rather than independent participant UAT; Firefox and WebKit were outside the evaluated scope.

#### 4.9 Security-Oriented API Checks

Selected API checks covered public access, missing or invalid authentication, valid Candidate authentication, and denial of an Administrator route to a Candidate.

Table 4.5. API authorization observations

| Request                         | Expected | Observed | Result |
|---------------------------------|----------|----------|--------|
| Public Job search without token | 200      | 200      | Passed |

| <b>Request</b>                                             | <b>Expected</b> | <b>Observed</b> | <b>Result</b> |
|------------------------------------------------------------|-----------------|-----------------|---------------|
| Candidate CV endpoint without token                        | 401             | 401             | Passed        |
| Current-account endpoint with invalid Basic scheme         | 401             | 401             | Passed        |
| Current-account endpoint with valid Candidate bearer token | 200             | 200             | Passed        |
| Administrator dashboard with Candidate bearer token        | 403             | 403             | Passed        |

The observed 200, 401, and 403 responses support the tested authentication and role boundaries. These are authorization spot checks, not a penetration test, and do not cover every ownership, abuse, malware, rate-limit, or secret-management risk.

#### 4.10 Runtime Health, Monitoring, and Local Latency

*Table 4.6. Runtime observations*

| <b>Endpoint or check</b>        | <b>Observation</b> |
|---------------------------------|--------------------|
| /api/jobs/search?page=0&size=20 | 200                |
| /actuator/health                | 200, UP            |
| /actuator/health/liveness       | 200, UP            |
| /actuator/health/readiness      | 200, UP            |
| /actuator/prometheus            | 200                |

The core Job API, aggregate health, liveness, readiness, and Prometheus endpoints returned the observations shown in Table 4.6. Mail health was excluded when mail delivery was disabled, so an intentionally unavailable provider did not make the local aggregate status misleading.

Thirty sequential warm localhost requests to Job search produced a mean of 61.79 ms, p50 of 55.20 ms, p95 of 85.11 ms, minimum of 44.99 ms, and maximum of 99.32 ms. The small single-client sample contains no concurrent load, production network, repeated batches, or resource monitoring. It is a local consistency observation, not a throughput, capacity, or production-performance benchmark.



## 4.11 Evaluation Summary by Research Question

Table 4.7. Answers supported by current evidence

| Question                                                       | Evidence-supported answer                                                                                                             |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Does Rocchio change holdout ranking in the intended direction? | Yes for the controlled synthetic dataset; the same large metric delta was reproduced.                                                 |
| Is the backend automated suite passing?                        | Yes. All 141 tests across 35 suites passed, and clean verify built the JAR.                                                           |
| Do the main browser workflows execute?                         | Yes. All 49 integrated Chrome tests passed; Firefox/WebKit and participant UAT remain incomplete.                                     |
| Does report moderation work in the tested contracts?           | Yes for Candidate Job report, Recruiter visible-CV report, and Administrator Job ban; production abuse handling is not evaluated.     |
| Are selected authorization boundaries enforced?                | Observed public, authentication, role, ownership, and report-visibility checks passed; this is not comprehensive security validation. |
| Is the runtime healthy?                                        | Yes in the evaluated local profile: health groups and the core API returned HTTP 200.                                                 |
| Is performance production-ready?                               | Not evaluated. Only a small sequential localhost latency sample was collected.                                                        |

## 4.12 Threats to Validity

### 4.12.1 Construct Validity

The synthetic benchmark measures planted retrieval relationships, not hiring success, fairness, or recruiter satisfaction. Matching metrics cannot determine whether a person should be employed. Scripted E2E tests measure workflow execution rather than usability, explanation quality, or user trust. The selected measures therefore support technical behavior only.

### 4.12.2 Internal Validity

Asynchronous learning and persisted Matching state could affect repeatability. The benchmark starts learning after commit, clears previous Matching state, and checks background outcomes. The final run also checked logs for background

failures. Browser tests use explicit conditions and clean up created Jobs, although shared records and scheduler timing can still affect reruns.

The browser suite also avoided relying only on the first seeded Job and removed the Job created by the Recruiter workflow, reducing dependence on mutable catalogue order.

#### **4.12.3 External Validity**

The controlled data lacks organic feedback, demographic diversity, adversarial behavior, and production class imbalance. Runtime evidence comes from one Windows workstation, local PostgreSQL, Docker Desktop, and desktop Chrome. No independent participant UAT was conducted. Results cannot be generalized to other browsers, cloud deployment, enterprise load, or real users.

#### **4.12.4 Reproducibility**

The Maven Wrapper, Flyway migrations, Testcontainers, package lock, controlled dataset, result artifacts, and recorded commands support reproduction. Exact reproduction is nevertheless limited because the evaluated worktree and E2E database were not frozen as a clean release. A final reproducible release should preserve a clean source archive together with the corresponding evidence package.

### **4.13 Chapter Summary**

The evaluation confirmed controlled Rocchio behavior and the tested software workflows. It supports a functioning academic prototype within the evaluated environment, not production readiness, hiring effectiveness, or production-scale performance.

## **PART 3. CONCLUSION**

### **1. Summary of Results**

CareerFit IT AutoPilot integrates public Job discovery, role-based workflows, reviewed CV processing, separate Matching and Recommendation, Rocchio feedback, controlled AutoFit, actionable email, moderation, and audit records. Ranking evidence remains separate from application and policy decisions so that users retain visible control points.

Evaluation passed 141 backend tests across 35 suites and all 49 integrated Chrome tests. In the controlled synthetic benchmark, Rocchio moved related holdout CVs upward, including an  $nDCG@5$  increase from approximately 0.038 to 0.838. This evidence supports an academic prototype, not production readiness or real-world hiring effectiveness.

### **2. Achievement of Thesis Objectives**

#### **2.1 Role-Based Recruitment Platform**

The prototype supports public Job exploration and distinct Candidate, Recruiter, and Administrator workflows. Integrated Chrome tests cover representative role paths, including reporting and moderation; complete route coverage, other browsers, and independent usability evaluation remain outside scope.

#### **2.2 CV and Job-Description Processing**

The system supports uploaded and manual CVs, file validation, extraction, configured OCR fallback, review and confirmation, and structured Job validation. Tests support these workflows; hardened storage, OCR availability, and broader document variation remain unresolved.

#### **2.3 Matching and Recommendation**

CareerFit provides an interpretable TF-IDF/cosine Matching baseline, reasons, a separate Potential assessment, and profile-oriented Recommendation. Tests support the scoring and contracts, but lexical matching remains limited for synonyms, career context, and multilingual terms; scores are not hiring probabilities.

#### **2.4 Feedback Learning**

Typed feedback updates learned Job vectors through Rocchio and triggers Matching recomputation. The controlled training/holdout benchmark confirms the intended adaptation, but not ranking quality or concurrency safety under organic production feedback.

## 2.5 Policy-Driven Automation and Email Action

AutoFit checks consent, CV, Job, threshold, duplicates, and per-run limits before creating Applications. Actionable email uses hashed expiring tokens and confirmation before execution. These controls support bounded automation, while rate limiting, provider testing, and deployment hardening remain incomplete.

## 2.6 Auditability and Evaluation

Audit records cover major Application, Feedback, Automation, moderation, and administrative events. Evaluation supports the prototype scope, while complete audit coverage and production observability remain unverified.

*Table Con.1. Objective assessment*

| Objective                          | Assessment                                        | Primary evidence                                                                        |
|------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------|
| Role-based web platform            | Achieved for prototype scope                      | API contracts and 49 integrated Chrome tests                                            |
| Reviewed CV/JD ingestion           | Achieved for supported formats and configured OCR | Draft/review/confirm, extraction, quality, Job, and integration tests                   |
| Explainable matching and Potential | Achieved as rule-based baseline                   | TF-IDF/scoring tests and skill-transfer reasons                                         |
| Rocchio feedback adaptation        | Achieved in controlled test                       | Synthetic benchmark, after-commit execution, and clean reports                          |
| AutoFit and communication          | Achieved as configurable prototype                | Policies, scheduler, reminders, notifications, and signed email actions                 |
| Recruiter operational workflow     | Achieved for prototype scope                      | Company, Job, applicants, Talent Pool, bookmark, and invitation tests                   |
| Content reporting and moderation   | Achieved for the implemented basic flow           | ContentReportServiceTest and three browser contracts                                    |
| Production readiness               | Not demonstrated                                  | Local health passed; scale, real users, appeals, and production security remain missing |

### **3. Discussion**

#### **3.1 Value of an Interpretable Baseline**

The lexical baseline exposes tokens, weights, shared terms, and score construction, supporting explanation, debugging, and reproducible comparison. Its value is transparency rather than semantic completeness. Future semantic or hybrid methods should therefore be compared with this baseline rather than assumed superior.

#### **3.2 Human-in-the-Loop as System Structure**

Users validate data, inspect reasons, choose actions, configure or pause automation, provide feedback, and review records. These control points make automation reviewable, but human involvement alone does not remove bias or unsafe thresholds; representative evaluation, appeals, access control, and audit review remain necessary.

#### **3.3 Meaning of the Rocchio Improvement**

The large synthetic improvement follows from the designed latent relationship between training and holdout CVs. It confirms reranking in the intended direction under fixed conditions, not a comparable improvement from organic feedback, which may be sparse, inconsistent, delayed, or influenced by nontechnical factors.

#### **3.4 Test Results versus Background Errors**

Asynchronous workflows require checking background logs and side effects in addition to foreground assertions. The corrected benchmark no longer reproduced the observed conflict, but production concurrency and recovery remain unevaluated.

#### **3.5 Product Positioning**

CareerFit is an IT-focused, explainable, policy-controlled academic prototype integrating separate Matching and Recommendation, feedback learning, controlled actions, moderation, and auditability. It neither replaces professional judgment nor establishes superiority over commercial platforms.

### **4. Limitations**

#### **4.1 Data and Algorithm Limitations**

The synthetic dataset lacks organic language, recruiter disagreement, demographic analysis, adversarial CVs, and changing terminology. TF-IDF uses a static corpus and simple tokenization, while Potential assessment relies on a manually maintained skill-transfer knowledge base.

## **4.2 Evaluation Limitations**

Automated tests do not establish complete coverage or user acceptance. Evaluation used desktop Chrome and sequential local latency requests; concurrency, capacity, accessibility, other browsers, real moderation, fairness, and hiring outcomes remain unevaluated.

## **4.3 Security and Privacy Limitations**

Email-action controls reduce selected risks, but frontend tokens remain in sessionStorage, and local CV storage lacks demonstrated malware scanning, encryption, retention, and recovery. Rate limiting, privacy governance, and production security assessment remain necessary.

## **4.4 Reliability and Maintainability Limitations**

Audit generation is not fully centralized, production recovery is unverified, and moderation lacks appeal and restoration workflows. The evaluated dirty worktree also prevents the commit hash from identifying an exact release.

## **5. Future Work**

Future work should prioritize:

1. building frozen, independently labeled datasets with temporal holdouts, reviewer agreement, hard negatives, and fairness analysis;
2. comparing lexical, semantic, and hybrid retrieval while preserving inspectable reasons;
3. strengthening sessions, rate limits, malware scanning, encrypted storage, retention, recovery, and email-provider testing;
4. evaluating concurrency, failure recovery, accessibility, other browsers, load, and user acceptance; and
5. completing moderation appeals, centralized audit redaction, protected object storage, and reproducible release archives.

These priorities address verified limitations and are not current features.

## **6. Closing Remarks**

CareerFit demonstrates an auditable Human-in-the-Loop recruitment workflow using interpretable ranking, feedback learning, and controlled automation. It provides a tested academic prototype and reproducible baseline while keeping final recruitment decisions and real-world effectiveness outside the established evidence.

## REFERENCES

- [1] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975, doi: 10.1145/361219.361220.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] C. Buck and P. Koehn, “Quick and reliable document alignment via TF/IDF-weighted cosine distance,” in *Proceedings of the First Conference on Machine Translation*, vol. 2, Berlin, Germany, 2016, pp. 672–678, doi: 10.18653/v1/W16-2365.
- [4] J. J. Rocchio, “Relevance feedback in information retrieval,” in *The SMART Retrieval System: Experiments in Automatic Document Processing*, G. Salton, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1971, pp. 313–323.
- [5] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.
- [6] N. Drushchak and M. Romanyshyn, “Introducing the Djinni Recruitment Dataset: A corpus of anonymized CVs and job postings,” in *Proceedings of the Third Ukrainian Natural Language Processing Workshop*, Torino, Italy, 2024, pp. 8–13, doi: 10.63317/2dcvy45ws6yb.
- [7] X. Yu, R. Xu, C. Xue, J. Zhang, X. Ma, and Z. Yu, “ConFit v2: Improving resume-job matching using hypothetical resume embedding and runner-up hard-negative mining,” in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 12775–12790, doi: 10.18653/v1/2025.findings-acl.661.
- [8] S. Vaishampayan et al., “Human and LLM-based resume matching: An observational study,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, NM, USA, 2025, pp. 4823–4838, doi: 10.18653/v1/2025.findings-naacl.270.
- [9] E. Tabassi, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.
- [10] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in *Proceedings of the*

- 2020 Conference on Fairness, Accountability, and Transparency, Barcelona, Spain, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.
- [11] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.
- [12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.
- [13] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2006, doi: 10.6028/NIST.SP.800-92.
- [14] A. Colas, J. Araki, Z. Zhou, B. Wang, and Z. Feng, “Knowledge-grounded natural language recommendation explanation,” in Proceedings of the 6th BlackboxNLP Workshop, Singapore, 2023, pp. 1–15, doi: 10.18653/v1/2023.blackboxnlp-1.1.
- [15] ITviec, “ITviec | Top IT Jobs for You,” [Online]. Available: <https://itviec.com/>. [Accessed: Aug. 12, 2026].
- [16] CareerViet, “Fast Recruitment & Job Search at CareerViet.vn,” [Online]. Available: <https://careerviet.vn/en>. [Accessed: Aug. 12, 2026].
- [17] TopDev, “TopDev - Việc Làm Lương Cao Hàng Đầu,” [Online]. Available: <https://topdev.vn/>. [Accessed: Aug. 12, 2026].
- [18] TopCV Vietnam, “TopCV Pro - Nơi kết nối Ứng viên chất & Doanh nghiệp hàng đầu,” [Online]. Available: <https://www.topcv.vn/pro>. [Accessed: Aug. 12, 2026].
- [19] CareerViet, “Tăng Tỷ Lệ Ứng Tuyển Thành Công Với AI Matching - Cách Biến CV Của Bạn Thành 'Ứng Viên Sáng Giá',” CareerViet Talent Community, [Online]. Available: <https://careerviet.vn/en/talentcommunity/tang-ti-le-ung-tuyen-thanh-cong-voi-ai-matching-cach-bien-cv-cua-ban-thanh-ung-vien-sang-gia.35A52D7D.html>. [Accessed: Aug. 12, 2026].



## APPENDICES

### Appendix A. Requirement-Test Traceability Matrix

*Table App.A.1. Requirement–test traceability for the approved Use Cases*

| Use Case | Business goal                                 | Verified workflow or test scope                                                           |
|----------|-----------------------------------------------|-------------------------------------------------------------------------------------------|
| UC-01    | Manage Career Profile                         | Candidate Profile, Portfolio, CV review and confirmation, default CV, and deletion        |
| UC-02    | Explore Jobs                                  | Public Job catalogue, search, detail, employer information, urgent Jobs, and similar Jobs |
| UC-03    | Manage Job Applications                       | Application submission, history, withdrawal, and invitation response                      |
| UC-04    | Provide Matching Feedback                     | Feedback submission and learned-ranking behavior                                          |
| UC-05    | Review Personalized Career Insights           | CV–Job Matching, profile-based recommendation, and Candidate analytics                    |
| UC-06    | Manage AutoFit                                | AutomationPolicy configuration, manual execution, and AutoApply eligibility               |
| UC-07    | Respond Through Actionable Email              | Confirmation, feedback or invitation redemption, replay rejection, and expiry handling    |
| UC-08    | Manage Employer Profile and Job Postings      | Employer Profile and Job lifecycle operations                                             |
| UC-09    | Review and Process Applicants                 | Applicant review and Application decision changes                                         |
| UC-10    | Manage Talent Pool and Invitations            | Bookmark, Talent Pool, invitation, and invitation withdrawal                              |
| UC-11    | Review Recruitment Analytics                  | Recruiter analytics and supported recruitment summaries                                   |
| UC-12    | Report Suspicious Recruitment Content         | Candidate Job-reporting and Recruiter visible-CV-reporting paths                          |
| UC-13    | Administer Platform Access and Job Visibility | Administrative User access and Job visibility controls                                    |
| UC-14    | Review and Resolve Content Reports            | Report review, dismissal, and supported moderation actions                                |

## Appendix B. Selected API Contracts

*Table App.B.1. Selected CareerFit API contracts*

| Method | Endpoint                                     | Business purpose                                                 |
|--------|----------------------------------------------|------------------------------------------------------------------|
| POST   | /api/auth/login                              | Authenticate a User and issue the supported signed JWT response  |
| GET    | /api/jobs/search                             | Explore the public Job catalogue using supported search criteria |
| POST   | /api/cv/upload                               | Upload a Candidate CV for validation and review                  |
| GET    | /api/cv/{cvId}/review                        | View the extracted CV review draft                               |
| PATCH  | /api/cv/{cvId}/review                        | Confirm or correct the supported CV review data                  |
| POST   | /api/matches/{matchingId}/feedback           | Submit Candidate feedback for a Matching result                  |
| POST   | /api/applications                            | Submit a Job Application                                         |
| POST   | /api/reports/jobs/{jobId}                    | Report suspicious Job content                                    |
| POST   | /api/reports/cvs/{cvId}                      | Report a visible Candidate CV                                    |
| GET    | /api/admin/reports                           | Review submitted content reports                                 |
| POST   | /api/admin/reports/{type}/{targetId}/ban     | Apply the supported moderation action to reported content        |
| POST   | /api/admin/reports/{type}/{targetId}/dismiss | Dismiss a reviewed content report                                |
| PATCH  | /api/automation/policy                       | Save Candidate AutoFit configuration                             |
| GET    | /actuator/health                             | Read the configured aggregate runtime health status              |

Except for intentionally public contracts, protected routes require a signed JWT together with backend role and ownership checks. The table describes actor-visible contracts rather than controller, repository, transaction, or persistence details.

## Appendix C. Full Data Dictionary

This appendix documents the final CareerFit PostgreSQL schema after Flyway V1-V25. It contains 24 active tables and 293 persisted columns. The earlier email\_token table is excluded because migration V16 removes it. Keys and constraints are reported from the running migrated schema; descriptions summarize the observable role of each stored value.

### C.1 Physical Table Catalogue

*Table App.C.1. CareerFit physical table catalogue*

| No. | Table name        | Data group                               | Purpose                                                                                        |
|-----|-------------------|------------------------------------------|------------------------------------------------------------------------------------------------|
| 1   | analytics_event   | Reporting, Analytics, and Audit          | Records product and recruitment events used by the implemented analytics views.                |
| 2   | application       | Employer, Job, Matching, and Recruitment | Stores a Candidate's application or Recruiter invitation state for a Job.                      |
| 3   | audit_log         | Reporting, Analytics, and Audit          | Stores append-oriented evidence for important user, system, moderation, and automated actions. |
| 4   | automation_policy | Automation, Email, and Notification      | Stores one account-level AutoFit, digest, email, and notification policy.                      |
| 5   | candidate         | Identity, Candidate, CV, and Portfolio   | Stores the Candidate profile, preferences, experience, and compatibility AutoFit fields.       |

| No. | Table name                  | Data group                               | Purpose                                                                                     |
|-----|-----------------------------|------------------------------------------|---------------------------------------------------------------------------------------------|
| 6   | candidate_portfolio_link    | Identity, Candidate, CV, and Portfolio   | Stores external portfolio links owned by a Candidate.                                       |
| 7   | candidate_portfolio_project | Identity, Candidate, CV, and Portfolio   | Stores structured project entries in a Candidate portfolio.                                 |
| 8   | content_report              | Reporting, Analytics, and Audit          | Stores Candidate or Recruiter reports about a Job or visible CV and the moderation outcome. |
| 9   | cv                          | Identity, Candidate, CV, and Portfolio   | Stores CV metadata, extracted and reviewed content, processing state, and reporting state.  |
| 10  | email_action                | Automation, Email, and Notification      | Tracks creation and delivery lifecycle metadata for an outbound actionable email.           |
| 11  | email_action_token          | Automation, Email, and Notification      | Stores the hash and lifecycle of a one-time actionable-email token.                         |
| 12  | employer_profile            | Employer, Job, Matching, and Recruitment | Stores the Recruiter's company profile and public employer information.                     |
| 13  | feedback                    | Employer, Job, Matching, and Recruitment | Stores Candidate or Recruiter judgments about a CV-Job Matching result.                     |

| No. | Table name                 | Data group                               | Purpose                                                                                              |
|-----|----------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------|
| 14  | job                        | Employer, Job, Matching, and Recruitment | Stores the Job Description, structured recruitment fields, vectors, publication state, and counters. |
| 15  | job_market_snapshot        | Reporting, Analytics, and Audit          | Stores date-based aggregate Job-market counts and distributions.                                     |
| 16  | job_trend_snapshot         | Reporting, Analytics, and Audit          | Stores date-based view and application counters for an individual Job.                               |
| 17  | matching                   | Employer, Job, Matching, and Recruitment | Stores one scored relationship between a CV and Job, including labels, reasons, and Potential data.  |
| 18  | notification_delivery_log  | Automation, Email, and Notification      | Records notification-policy outcomes used for quota, deduplication, cooldown, and audit evidence.    |
| 19  | notification_job           | Automation, Email, and Notification      | Stores queued notification work and its retry lifecycle.                                             |
| 20  | recommendation_interaction | Employer, Job, Matching, and Recruitment | Stores Candidate interactions with personalized Job recommendations.                                 |

| No. | Table name            | Data group                               | Purpose                                                                                   |
|-----|-----------------------|------------------------------------------|-------------------------------------------------------------------------------------------|
| 21  | recruiter_cv_bookmark | Employer, Job, Matching, and Recruitment | Stores a Recruiter's saved Candidate/CV for a specific owned Job.                         |
| 22  | skills                | Identity, Candidate, CV, and Portfolio   | Stores the searchable and normalized skill catalogue used by profile and Job inputs.      |
| 23  | user_account          | Identity, Candidate, CV, and Portfolio   | Stores authentication identity, role, activation state, verification state, and language. |
| 24  | user_settings         | Identity, Candidate, CV, and Portfolio   | Stores supporting account settings that do not require their own domain table.            |

## C.2 Field-Level Data Dictionary

### C.2.1 Identity, Candidate, CV, and Portfolio

#### 1.1.1 user\_account

Stores authentication identity, role, activation state, verification state, and language.

*Table App.C.2. Data dictionary for user\_account*

| No. | Column        | PostgreSQL type           | Null | Key | Default / constraint                    | Description                                                    |
|-----|---------------|---------------------------|------|-----|-----------------------------------------|----------------------------------------------------------------|
| 1   | id            | uuid                      | No   | PK  | Default:<br>gen_random_uuid()           | Unique identifier for this record.                             |
| 2   | email         | character<br>varying(255) | No   | UQ  | -                                       | Normalized email address used as the account login identifier. |
| 3   | password_hash | character<br>varying(255) | Yes  | -   | -                                       | BCrypt password hash; the original password is not stored.     |
| 4   | role          | character<br>varying(20)  | No   | -   | Allowed: CANDIDATE,<br>RECRUITER, ADMIN | Role used by authorization and role-specific workflows.        |
| 5   | full_name     | character<br>varying(255) | Yes  | -   | -                                       | Display name of the user.                                      |
| 6   | is_active     | boolean                   | No   | -   | Default: true                           | Indicates whether active applies to the record.                |

| No. | Column             | PostgreSQL type          | Null | Key | Default / constraint | Description                                                 |
|-----|--------------------|--------------------------|------|-----|----------------------|-------------------------------------------------------------|
| 7   | email_verified     | boolean                  | No   | -   | Default: false       | Indicates whether the account email has been verified.      |
| 8   | preferred_language | character varying(10)    | No   | -   | Default: 'vi'        | Preferred interface or content language code.               |
| 9   | created_at         | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was created.                      |
| 10  | updated_at         | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was last updated.                 |
| 11  | version            | bigint                   | No   | -   | Default: 0           | Optimistic-lock version used to detect conflicting updates. |



### 1.1.2 user\_settings

Stores supporting account settings that do not require their own domain table.

*Table App.C.3. Data dictionary for user\_settings*

| No. | Column     | PostgreSQL type             | Null | Key                                          | Default / constraint          | Description                                         |
|-----|------------|-----------------------------|------|----------------------------------------------|-------------------------------|-----------------------------------------------------|
| 1   | id         | uuid                        | No   | PK                                           | Default:<br>gen_random_uuid() | Unique identifier for this record.                  |
| 2   | user_id    | uuid                        | No   | FK -<br>><br>user_<br>acco<br>unt.id<br>; UQ | -                             | References the related user_account record.         |
| 3   | settings   | jsonb                       | No   | -                                            | Default: '{}'                 | JSON object containing supporting account settings. |
| 4   | created_at | timestamp with<br>time zone | No   | -                                            | Default: now()                | Timestamp when the record was created.              |
| 5   | updated_at | timestamp with<br>time zone | No   | -                                            | Default: now()                | Timestamp when the record was last updated.         |

### 1.1.3 candidate

Stores the Candidate profile, preferences, experience, and compatibility AutoFit fields.

*Table App.C.4. Data dictionary for candidate*

| No. | Column        | PostgreSQL type           | Null | Key                                          | Default / constraint          | Description                                 |
|-----|---------------|---------------------------|------|----------------------------------------------|-------------------------------|---------------------------------------------|
| 1   | id            | uuid                      | No   | PK                                           | Default:<br>gen_random_uuid() | Unique identifier for this record.          |
| 2   | user_id       | uuid                      | No   | FK -<br>><br>user_<br>acco<br>unt.id<br>; UQ | -                             | References the related user_account record. |
| 3   | phone         | character<br>varying(30)  | Yes  | -                                            | -                             | Stores the phone value for this record.     |
| 4   | location      | character<br>varying(255) | Yes  | -                                            | -                             | Stores the location value for this record.  |
| 5   | desired_title | character<br>varying(255) | Yes  | -                                            | -                             | Job title preferred by the Candidate.       |

| No. | Column                  | PostgreSQL type       | Null | Key | Default / constraint            | Description                                                            |
|-----|-------------------------|-----------------------|------|-----|---------------------------------|------------------------------------------------------------------------|
| 6   | desired_seniority       | character varying(50) | Yes  | -   | -                               | Seniority level preferred by the Candidate.                            |
| 7   | desired_skills          | jsonb                 | Yes  | -   | -                               | JSON list of skills preferred by the Candidate.                        |
| 8   | desired_work_model      | character varying(50) | Yes  | -   | -                               | Preferred onsite, hybrid, or remote work model.                        |
| 9   | desired_salary_min      | numeric(15,2)         | Yes  | -   | Minimum must not exceed maximum | Lower bound of the Candidate's preferred salary range.                 |
| 10  | desired_salary_max      | numeric(15,2)         | Yes  | -   | Minimum must not exceed maximum | Upper bound of the Candidate's preferred salary range.                 |
| 11  | desired_salary_currency | character varying(10) | Yes  | -   | -                               | Currency used for the Candidate's preferred salary range.              |
| 12  | years_of_experience     | integer               | Yes  | -   | Must be non-negative            | Candidate's declared total years of professional experience.           |
| 13  | about_me                | text                  | Yes  | -   | -                               | Candidate's profile introduction.                                      |
| 14  | auto_apply_enabled      | boolean               | No   | -   | Default: false                  | Indicates whether AutoFit auto-application is enabled for the account. |

| No. | Column                   | PostgreSQL type             | Null | Key | Default / constraint        | Description                                                              |
|-----|--------------------------|-----------------------------|------|-----|-----------------------------|--------------------------------------------------------------------------|
| 15  | auto_apply_thres<br>hold | numeric(5,2)                | No   | -   | Default: 90.00; Range 0-100 | Minimum normalized score configured for AutoFit application eligibility. |
| 16  | preferred_language       | character<br>varying(10)    | No   | -   | Default: 'vi'               | Preferred interface or content language code.                            |
| 17  | created_at               | timestamp with<br>time zone | No   | -   | Default: now()              | Timestamp when the record was created.                                   |
| 18  | updated_at               | timestamp with<br>time zone | No   | -   | Default: now()              | Timestamp when the record was last updated.                              |
| 19  | avatar_url               | character<br>varying(500)   | Yes  | -   | -                           | Location of the Candidate avatar image.                                  |
| 20  | version                  | bigint                      | No   | -   | Default: 0                  | Optimistic-lock version used to detect conflicting updates.              |

#### 1.1.4 candidate\_portfolio\_link

Stores external portfolio links owned by a Candidate.

Table App.C.5. Data dictionary for candidate\_portfolio\_link

| No. | Column       | PostgreSQL type             | Null | Key                               | Default / constraint          | Description                                 |
|-----|--------------|-----------------------------|------|-----------------------------------|-------------------------------|---------------------------------------------|
| 1   | id           | uuid                        | No   | PK                                | Default:<br>gen_random_uuid() | Unique identifier for this record.          |
| 2   | candidate_id | uuid                        | No   | FK -<br>><br>candi<br>date.i<br>d | -                             | References the related candidate record.    |
| 3   | type         | character<br>varying(50)    | Yes  | -                                 | -                             | Stores the type value for this record.      |
| 4   | url          | character<br>varying(500)   | No   | -                                 | -                             | Stores the url value for this record.       |
| 5   | created_at   | timestamp with<br>time zone | No   | -                                 | Default: now()                | Timestamp when the record was created.      |
| 6   | updated_at   | timestamp with<br>time zone | No   | -                                 | Default: now()                | Timestamp when the record was last updated. |

### 1.1.5 candidate\_portfolio\_project

Stores structured project entries in a Candidate portfolio.

Table App.C.6. Data dictionary for candidate\_portfolio\_project

| No. | Column       | PostgreSQL type           | Null | Key                               | Default / constraint          | Description                                             |
|-----|--------------|---------------------------|------|-----------------------------------|-------------------------------|---------------------------------------------------------|
| 1   | id           | uuid                      | No   | PK                                | Default:<br>gen_random_uuid() | Unique identifier for this record.                      |
| 2   | candidate_id | uuid                      | No   | FK -<br>><br>candi<br>date.i<br>d | -                             | References the related candidate record.                |
| 3   | name         | character<br>varying(255) | No   | -                                 | -                             | Stores the name value for this record.                  |
| 4   | role         | character<br>varying(255) | Yes  | -                                 | -                             | Role used by authorization and role-specific workflows. |
| 5   | summary      | text                      | Yes  | -                                 | -                             | Stores the summary value for this record.               |
| 6   | tech_stack   | jsonb                     | Yes  | -                                 | -                             | Stores the tech stack value for this record.            |

| No. | Column      | PostgreSQL type             | Null | Key | Default / constraint | Description                                    |
|-----|-------------|-----------------------------|------|-----|----------------------|------------------------------------------------|
| 7   | project_url | character<br>varying(500)   | Yes  | -   | -                    | URL associated with the project.               |
| 8   | impact      | text                        | Yes  | -   | -                    | Stores the impact value for this record.       |
| 9   | created_at  | timestamp with<br>time zone | No   | -   | Default: now()       | Timestamp when the record was created.         |
| 10  | updated_at  | timestamp with<br>time zone | No   | -   | Default: now()       | Timestamp when the record was last<br>updated. |

### 1.1.6 cv

Stores CV metadata, extracted and reviewed content, processing state, and reporting state.

*Table App.C.7. Data dictionary for cv*

| No. | Column       | PostgreSQL type           | Null | Key                               | Default / constraint          | Description                                                 |
|-----|--------------|---------------------------|------|-----------------------------------|-------------------------------|-------------------------------------------------------------|
| 1   | id           | uuid                      | No   | PK                                | Default:<br>gen_random_uuid() | Unique identifier for this record.                          |
| 2   | candidate_id | uuid                      | No   | FK -<br>><br>candi<br>date.i<br>d | -                             | References the related candidate record.                    |
| 3   | display_name | character<br>varying(255) | No   | -                                 | -                             | User-facing name of the CV.                                 |
| 4   | source       | character<br>varying(20)  | No   | -                                 | Allowed: UPLOAD,<br>MANUAL    | Channel or origin from which the record was created.        |
| 5   | is_default   | boolean                   | No   | -                                 | Default: false                | Indicates whether default applies to the record.            |
| 6   | raw_text     | text                      | Yes  | -                                 | -                             | Current CV text used by the review and processing workflow. |



| No. | Column          | PostgreSQL type           | Null | Key | Default / constraint                                                                                                                   | Description                                                     |
|-----|-----------------|---------------------------|------|-----|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 7   | parsed_summary  | text                      | Yes  | -   | -                                                                                                                                      | Summary derived from the reviewed CV content.                   |
| 8   | top_skills      | jsonb                     | Yes  | -   | -                                                                                                                                      | JSON list of the main skills extracted or confirmed for the CV. |
| 9   | extracted_terms | jsonb                     | Yes  | -   | -                                                                                                                                      | JSON representation of normalized terms used by scoring.        |
| 10  | language        | character<br>varying(10)  | Yes  | -   | -                                                                                                                                      | Stores the language value for this record.                      |
| 11  | status          | character<br>varying(30)  | No   | -   | Default: 'UPLOADED';<br>Allowed: UPLOADED,<br>VALIDATING,<br>REVIEW_REQUIRED,<br>DRAFT, PROCESSING,<br>SCORING_DONE,<br>FAILED, BANNED | Current lifecycle state of the cv record.                       |
| 12  | file_path       | character<br>varying(500) | Yes  | -   | -                                                                                                                                      | Protected storage path of the uploaded CV file.                 |

| No. | Column             | PostgreSQL type          | Null | Key | Default / constraint | Description                                                         |
|-----|--------------------|--------------------------|------|-----|----------------------|---------------------------------------------------------------------|
| 13  | file_original_name | character varying(255)   | Yes  | -   | -                    | Original client filename of the uploaded CV.                        |
| 14  | failure_reason     | character varying(500)   | Yes  | -   | -                    | Actor-visible reason recorded when CV processing fails.             |
| 15  | last_scored_at     | timestamp with time zone | Yes  | -   | -                    | Timestamp of the most recent completed CV scoring operation.        |
| 16  | created_at         | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was created.                              |
| 17  | updated_at         | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was last updated.                         |
| 18  | version            | bigint                   | No   | -   | Default: 0           | Optimistic-lock version used to detect conflicting updates.         |
| 19  | original_raw_text  | text                     | Yes  | -   | -                    | Original extracted or manually entered CV text before review edits. |

| No. | Column               | PostgreSQL type | Null | Key | Default / constraint | Description                                                           |
|-----|----------------------|-----------------|------|-----|----------------------|-----------------------------------------------------------------------|
| 20  | review_sections      | jsonb           | Yes  | -   | -                    | JSON object containing editable CV review sections.                   |
| 21  | review_issues        | jsonb           | Yes  | -   | -                    | JSON list of validation warnings or issues presented during review.   |
| 22  | pending_report_count | integer         | No   | -   | Default: 0           | Number of unresolved reports currently associated with the Job or CV. |

### 1.1.7 skills

Stores the searchable and normalized skill catalogue used by profile and Job inputs.

*Table App.C.8. Data dictionary for skills*

| No. | Column          | PostgreSQL type           | Null | Key | Default / constraint             | Description                                           |
|-----|-----------------|---------------------------|------|-----|----------------------------------|-------------------------------------------------------|
| 1   | id              | uuid                      | No   | PK  | Default:<br>gen_random_uuid()    | Unique identifier for this record.                    |
| 2   | canonical_key   | character<br>varying(100) | No   | UQ  | -                                | Stable canonical identifier of a skill.               |
| 3   | name            | character<br>varying(100) | No   | -   | -                                | Stores the name value for this record.                |
| 4   | normalized_name | character<br>varying(100) | No   | UQ  | -                                | Normalized skill name used for uniqueness and search. |
| 5   | search_text     | text                      | No   | -   | -                                | Searchable text assembled for skill lookup.           |
| 6   | category        | character<br>varying(50)  | No   | -   | -                                | Stores the category value for this record.            |
| 7   | popularity      | integer                   | No   | -   | Default: 0; Must be non-negative | Observed usage count used for skill ordering.         |

| No. | Column     | PostgreSQL type          | Null | Key | Default / constraint | Description                                     |
|-----|------------|--------------------------|------|-----|----------------------|-------------------------------------------------|
| 8   | is_active  | boolean                  | No   | -   | Default: true        | Indicates whether active applies to the record. |
| 9   | created_at | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was created.          |
| 10  | updated_at | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was last updated.     |

## C.2.2 Employer, Job, Matching, and Recruitment

### 1.1.8 *employer\_profile*

Stores the Recruiter's company profile and public employer information.

*Table App.C.9. Data dictionary for employer\_profile*

| No. | Column       | PostgreSQL type           | Null | Key                                  | Default / constraint          | Description                                             |
|-----|--------------|---------------------------|------|--------------------------------------|-------------------------------|---------------------------------------------------------|
| 1   | id           | uuid                      | No   | PK                                   | Default:<br>gen_random_uuid() | Unique identifier for this record.                      |
| 2   | recruiter_id | uuid                      | No   | FK -<br>><br>user_<br>acco<br>unt.id | -                             | References the related user_account record.             |
| 3   | company_name | character<br>varying(255) | No   | -                                    | -                             | Public company name shown with the Employer Profile.    |
| 4   | slug         | character<br>varying(255) | No   | UQ                                   | -                             | Unique URL-friendly identifier of the Employer Profile. |
| 5   | logo_url     | character<br>varying(500) | Yes  | -                                    | -                             | Location of the company logo image.                     |

| No. | Column       | PostgreSQL type           | Null | Key | Default / constraint | Description                                                    |
|-----|--------------|---------------------------|------|-----|----------------------|----------------------------------------------------------------|
| 6   | cover_url    | character<br>varying(500) | Yes  | -   | -                    | Location of the company cover image.                           |
| 7   | summary      | text                      | Yes  | -   | -                    | Stores the summary value for this record.                      |
| 8   | description  | text                      | Yes  | -   | -                    | Stores the description value for this record.                  |
| 9   | industry     | character<br>varying(100) | Yes  | -   | -                    | Stores the industry value for this record.                     |
| 10  | company_size | character<br>varying(50)  | Yes  | -   | -                    | Declared size category of the employer.                        |
| 11  | location     | character<br>varying(255) | Yes  | -   | -                    | Stores the location value for this record.                     |
| 12  | website_url  | character<br>varying(500) | Yes  | -   | -                    | URL associated with the website.                               |
| 13  | benefits     | jsonb                     | Yes  | -   | -                    | JSON list of employer benefits displayed to users.             |
| 14  | is_featured  | boolean                   | No   | -   | Default: false       | Indicates whether the employer is highlighted in public views. |

| No. | Column     | PostgreSQL type          | Null | Key | Default / constraint | Description                                                 |
|-----|------------|--------------------------|------|-----|----------------------|-------------------------------------------------------------|
| 15  | created_at | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was created.                      |
| 16  | updated_at | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was last updated.                 |
| 17  | version    | bigint                   | No   | -   | Default: 0           | Optimistic-lock version used to detect conflicting updates. |



### 1.1.9 job

Stores the Job Description, structured recruitment fields, vectors, publication state, and counters.

*Table App.C.10. Data dictionary for job*

| No. | Column          | PostgreSQL type           | Null | Key                                  | Default / constraint          | Description                                                    |
|-----|-----------------|---------------------------|------|--------------------------------------|-------------------------------|----------------------------------------------------------------|
| 1   | id              | uuid                      | No   | PK                                   | Default:<br>gen_random_uuid() | Unique identifier for this record.                             |
| 2   | recruiter_id    | uuid                      | No   | FK -<br>><br>user_<br>acco<br>unt.id | -                             | References the related user_account record.                    |
| 3   | title           | character<br>varying(255) | No   | -                                    | -                             | Stores the title value for this record.                        |
| 4   | company         | character<br>varying(255) | No   | -                                    | -                             | Stores the company value for this record.                      |
| 5   | original_text   | text                      | No   | -                                    | -                             | Original Job Description text used for display and processing. |
| 6   | required_skills | jsonb                     | Yes  | -                                    | -                             | JSON list of skills required by the Job.                       |

| No. | Column              | PostgreSQL type        | Null | Key | Default / constraint                                                                  | Description                                                      |
|-----|---------------------|------------------------|------|-----|---------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 7   | nice_to_have_skills | jsonb                  | Yes  | -   | -                                                                                     | JSON list of preferred but non-mandatory skills.                 |
| 8   | seniority_level     | character varying(50)  | Yes  | -   | -                                                                                     | Seniority level requested by the Job.                            |
| 9   | employment_type     | character varying(50)  | Yes  | -   | -                                                                                     | Employment arrangement associated with the Job.                  |
| 10  | location            | character varying(255) | Yes  | -   | -                                                                                     | Stores the location value for this record.                       |
| 11  | remote_type         | character varying(50)  | Yes  | -   | -                                                                                     | Onsite, hybrid, or remote arrangement of the Job.                |
| 12  | domain              | character varying(100) | Yes  | -   | -                                                                                     | IT domain or specialization assigned to the Job.                 |
| 13  | salary_mode         | character varying(20)  | No   | -   | Salary fields must match salary mode; Allowed: NEGOTIABLE, RANGE, UP_TO, FROM, HIDDEN | Mode controlling how salary information is stored and displayed. |

| No. | Column                 | PostgreSQL type        | Null | Key | Default / constraint                                                                        | Description                                                    |
|-----|------------------------|------------------------|------|-----|---------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| 14  | salary_min             | numeric(15,2)          | Yes  | -   | Salary fields must match salary mode; Must be non-negative; Minimum must not exceed maximum | Lower bound of the Job salary range when applicable.           |
| 15  | salary_max             | numeric(15,2)          | Yes  | -   | Salary fields must match salary mode; Must be non-negative; Minimum must not exceed maximum | Upper bound of the Job salary range when applicable.           |
| 16  | salary_currency        | character varying(10)  | Yes  | -   | -                                                                                           | Currency of the Job salary values.                             |
| 17  | salary_type            | character varying(20)  | Yes  | -   | Allowed: MONTHLY, HOURLY, YEARLY                                                            | Time basis of salary values, such as monthly or yearly.        |
| 18  | salary_is_visible      | boolean                | No   | -   | Default: true                                                                               | Indicates whether salary information is visible to users.      |
| 19  | salary_display_text    | character varying(255) | Yes  | -   | -                                                                                           | Preformatted salary text used by the interface when available. |
| 20  | learned_profile_vector | jsonb                  | Yes  | -   | -                                                                                           | JSON sparse Job vector updated from relevance feedback.        |

| No. | Column          | PostgreSQL type          | Null | Key | Default / constraint                                                                  | Description                                               |
|-----|-----------------|--------------------------|------|-----|---------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 21  | tfidf_vector    | jsonb                    | Yes  | -   | -                                                                                     | JSON sparse TF-IDF vector built from the Job Description. |
| 22  | language        | character varying(10)    | No   | -   | Default: 'vi'                                                                         | Stores the language value for this record.                |
| 23  | status          | character varying(20)    | No   | -   | Default: 'ACTIVE';<br>Allowed: ACTIVE, CLOSED, DRAFT, PAUSED, HIDDEN_BY_ADMIN, BANNED | Current lifecycle state of the job record.                |
| 24  | created_at      | timestamp with time zone | No   | -   | Default: now()                                                                        | Timestamp when the record was created.                    |
| 25  | updated_at      | timestamp with time zone | No   | -   | Default: now()                                                                        | Timestamp when the record was last updated.               |
| 26  | deadline        | date                     | Yes  | -   | -                                                                                     | Legacy or date-only Job application deadline.             |
| 27  | applicant_count | integer                  | No   | -   | Default: 0; Must be non-negative                                                      | Stored number of applicants associated with the Job.      |

| No. | Column            | PostgreSQL type          | Null | Key | Default / constraint             | Description                                                          |
|-----|-------------------|--------------------------|------|-----|----------------------------------|----------------------------------------------------------------------|
| 28  | view_count        | integer                  | No   | -   | Default: 0; Must be non-negative | Stored number of Job-detail views.                                   |
| 29  | version           | bigint                   | No   | -   | Default: 0                       | Optimistic-lock version used to detect conflicting updates.          |
| 30  | source_platform   | character varying(50)    | Yes  | -   | -                                | External platform from which the Job was imported, when applicable.  |
| 31  | source_url        | character varying(1000)  | Yes  | -   | -                                | Original URL of an imported Job.                                     |
| 32  | scraped_at        | timestamp with time zone | Yes  | -   | -                                | Timestamp when an external Job record was collected.                 |
| 33  | external_hash     | character varying(64)    | Yes  | -   | -                                | Stable hash used to detect duplicate imported Jobs.                  |
| 34  | application_count | integer                  | No   | -   | Default: 0                       | Stored application count used by Job popularity and analytics flows. |

| No. | Column                | PostgreSQL type          | Null | Key | Default / constraint | Description                                                           |
|-----|-----------------------|--------------------------|------|-----|----------------------|-----------------------------------------------------------------------|
| 35  | is_urgent             | boolean                  | No   | -   | Default: false       | Indicates that the Recruiter marked the Job for urgent hiring.        |
| 36  | application_dead_line | timestamp with time zone | Yes  | -   | -                    | Timestamp after which the Job no longer accepts applications.         |
| 37  | pending_report_count  | integer                  | No   | -   | Default: 0           | Number of unresolved reports currently associated with the Job or CV. |

### 1.1.10 matching

Stores one scored relationship between a CV and Job, including labels, reasons, and Potential data.

*Table App.C.11. Data dictionary for matching*

| No. | Column           | PostgreSQL type | Null | Key                         | Default / constraint          | Description                                                        |
|-----|------------------|-----------------|------|-----------------------------|-------------------------------|--------------------------------------------------------------------|
| 1   | id               | uuid            | No   | PK                          | Default:<br>gen_random_uuid() | Unique identifier for this record.                                 |
| 2   | cv_id            | uuid            | No   | FK -<br>><br>cv.id;<br>UQ   | -                             | References the related cv record.                                  |
| 3   | job_id           | uuid            | No   | FK -<br>><br>job.id<br>; UQ | -                             | References the related job record.                                 |
| 4   | raw_score        | numeric(10,6)   | No   | -                           | Range 0-1                     | Cosine-similarity score in the implemented zero-to-one range.      |
| 5   | normalized_score | numeric(5,2)    | No   | -                           | Range 0-100                   | User-facing Matching score normalized to zero through one hundred. |

| No. | Column           | PostgreSQL type          | Null | Key | Default / constraint                  | Description                                                   |
|-----|------------------|--------------------------|------|-----|---------------------------------------|---------------------------------------------------------------|
| 6   | label            | character varying(20)    | No   | -   | Allowed: LOW, MEDIUM, HIGH, POTENTIAL | Implemented LOW, MEDIUM, HIGH, or POTENTIAL Matching label.   |
| 7   | is_potential     | boolean                  | No   | -   | Default: false                        | Indicates that the separate Potential assessment applies.     |
| 8   | match_reasons    | jsonb                    | Yes  | -   | -                                     | JSON reasons explaining the direct CV-Job score.              |
| 9   | potential_reason | jsonb                    | Yes  | -   | -                                     | JSON explanation produced by the Potential assessment.        |
| 10  | needs_recompute  | boolean                  | No   | -   | Default: false                        | Indicates that the persisted Matching should be recalculated. |
| 11  | created_at       | timestamp with time zone | No   | -   | Default: now()                        | Timestamp when the record was created.                        |



| No. | Column          | PostgreSQL type          | Null | Key | Default / constraint                            | Description                                                     |
|-----|-----------------|--------------------------|------|-----|-------------------------------------------------|-----------------------------------------------------------------|
| 12  | updated_at      | timestamp with time zone | No   | -   | Default: now()                                  | Timestamp when the record was last updated.                     |
| 13  | recruiter_label | character varying(30)    | Yes  | -   | Allowed: APPROVED, POTENTIAL, REJECTED, PENDING | Recruiter decision label attached to the Matching when present. |
| 14  | version         | bigint                   | No   | -   | Default: 0                                      | Optimistic-lock version used to detect conflicting updates.     |

### ***1.1.11 feedback***

Stores Candidate or Recruiter judgments about a CV-Job Matching result.

*Table App.C.12. Data dictionary for feedback*

| No. | Column      | PostgreSQL type          | Null | Key                                 | Default / constraint             | Description                                           |
|-----|-------------|--------------------------|------|-------------------------------------|----------------------------------|-------------------------------------------------------|
| 1   | id          | uuid                     | No   | PK                                  | Default:<br>gen_random_uuid()    | Unique identifier for this record.                    |
| 2   | matching_id | uuid                     | No   | FK -<br>><br>matching.id;<br>UQ     | -                                | References the related matching record.               |
| 3   | actor_id    | uuid                     | No   | FK -<br>><br>user_account.id;<br>UQ | -                                | References the related user_account record.           |
| 4   | actor_role  | character<br>varying(20) | No   | -                                   | Allowed: CANDIDATE,<br>RECRUITER | Role of the user that produced the event or feedback. |

| No. | Column         | PostgreSQL type          | Null | Key | Default / constraint                                      | Description                                          |
|-----|----------------|--------------------------|------|-----|-----------------------------------------------------------|------------------------------------------------------|
| 5   | feedback_type  | character varying(30)    | No   | -   | Allowed: GOOD_MATCH, POTENTIAL, BAD_MATCH, NOT_INTERESTED | Business judgment submitted for the Matching.        |
| 6   | source_channel | character varying(20)    | No   | -   | Default: 'WEB'; Allowed: WEB, EMAIL, DIGEST, AUTOPILOT    | Stores the source channel value for this record.     |
| 7   | metadata       | jsonb                    | Yes  | -   | -                                                         | Structured JSON metadata associated with the record. |
| 8   | created_at     | timestamp with time zone | No   | -   | Default: now()                                            | Timestamp when the record was created.               |

### 1.1.12 application

Stores a Candidate's application or Recruiter invitation state for a Job.

*Table App.C.13. Data dictionary for application*

| No. | Column       | PostgreSQL type | Null | Key                              | Default / constraint          | Description                              |
|-----|--------------|-----------------|------|----------------------------------|-------------------------------|------------------------------------------|
| 1   | id           | uuid            | No   | PK                               | Default:<br>gen_random_uuid() | Unique identifier for this record.       |
| 2   | candidate_id | uuid            | No   | FK -<br>><br>candidate.id;<br>UQ | -                             | References the related candidate record. |
| 3   | job_id       | uuid            | No   | FK -<br>><br>job.id;<br>UQ       | -                             | References the related job record.       |

| No. | Column          | PostgreSQL type             | Null | Key                      | Default / constraint                                                                                           | Description                                                                          |
|-----|-----------------|-----------------------------|------|--------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 4   | matching_id     | uuid                        | Yes  | FK -<br>><br>matching.id | -                                                                                                              | References the related matching record.                                              |
| 5   | cv_id           | uuid                        | Yes  | FK -<br>><br>cv.id       | -                                                                                                              | References the related cv record.                                                    |
| 6   | status          | character<br>varying(30)    | No   | -                        | Default: 'PENDING';<br>Allowed: PENDING,<br>AUTO_APPLIED,<br>APPROVED, REJECTED,<br>INVITED,<br>NOT_INTERESTED | Current lifecycle state of the application record.                                   |
| 7   | is_auto_applied | boolean                     | No   | -                        | Default: false                                                                                                 | Indicates that the Application was created by AutoFit rather than manual submission. |
| 8   | applied_at      | timestamp with<br>time zone | No   | -                        | Default: now()                                                                                                 | Timestamp when the Application was created.                                          |

| No. | Column          | PostgreSQL type          | Null | Key | Default / constraint | Description                                                        |
|-----|-----------------|--------------------------|------|-----|----------------------|--------------------------------------------------------------------|
| 9   | updated_at      | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was last updated.                        |
| 10  | cover_letter    | text                     | Yes  | -   | -                    | Optional cover letter submitted with the Application.              |
| 11  | recruiter_notes | text                     | Yes  | -   | -                    | Recruiter notes associated with the Application decision workflow. |
| 12  | version         | bigint                   | No   | -   | Default: 0           | Optimistic-lock version used to detect conflicting updates.        |

### 1.1.13 recommendation\_interaction

Stores Candidate interactions with personalized Job recommendations.

Table App.C.14. Data dictionary for recommendation\_interaction

| No. | Column       | PostgreSQL type          | Null | Key                               | Default / constraint                                                               | Description                                              |
|-----|--------------|--------------------------|------|-----------------------------------|------------------------------------------------------------------------------------|----------------------------------------------------------|
| 1   | id           | uuid                     | No   | PK                                | Default:<br>gen_random_uuid()                                                      | Unique identifier for this record.                       |
| 2   | candidate_id | uuid                     | No   | FK -<br>><br>candi<br>date.i<br>d | -                                                                                  | References the related candidate record.                 |
| 3   | job_id       | uuid                     | No   | FK -<br>><br>job.id               | -                                                                                  | References the related job record.                       |
| 4   | action       | character<br>varying(30) | No   | -                                 | Allowed: VIEWED,<br>SKIPPED, APPLIED,<br>SAVED,<br>NOT_INTERESTED,<br>SHOW_SIMILAR | Candidate interaction recorded for a<br>recommended Job. |

| No. | Column     | PostgreSQL type             | Null | Key | Default / constraint                                         | Description                                             |
|-----|------------|-----------------------------|------|-----|--------------------------------------------------------------|---------------------------------------------------------|
| 5   | source     | character<br>varying(20)    | No   | -   | Default: 'WEB'; Allowed:<br>WEB, EMAIL, DIGEST,<br>AUTOPILOT | Channel or origin from which the record<br>was created. |
| 6   | metadata   | jsonb                       | Yes  | -   | -                                                            | Structured JSON metadata associated<br>with the record. |
| 7   | created_at | timestamp with<br>time zone | No   | -   | Default: now()                                               | Timestamp when the record was created.                  |



### 1.1.14 recruiter\_cv\_bookmark

Stores a Recruiter's saved Candidate/CV for a specific owned Job.

*Table App.C.15. Data dictionary for recruiter\_cv\_bookmark*

| No. | Column | PostgreSQL type | Null | Key                         | Default / constraint          | Description                        |
|-----|--------|-----------------|------|-----------------------------|-------------------------------|------------------------------------|
| 1   | id     | uuid            | No   | PK                          | Default:<br>gen_random_uuid() | Unique identifier for this record. |
| 2   | job_id | uuid            | No   | FK -<br>><br>job.id<br>; UQ | -                             | References the related job record. |

| No. | Column       | PostgreSQL type             | Null | Key                                      | Default / constraint | Description                              |
|-----|--------------|-----------------------------|------|------------------------------------------|----------------------|------------------------------------------|
| 3   | candidate_id | uuid                        | No   | FK -<br>><br>candi<br>date.i<br>d;<br>UQ | -                    | References the related candidate record. |
| 4   | cv_id        | uuid                        | No   | FK -<br>><br>cv.id                       | -                    | References the related cv record.        |
| 5   | created_at   | timestamp with<br>time zone | No   | -                                        | Default: now()       | Timestamp when the record was created.   |

### C.2.3 Automation, Email, and Notification

#### 1.1.15 automation\_policy

Stores one account-level AutoFit, digest, email, and notification policy.

Table App.C.16. Data dictionary for automation\_policy

| No. | Column               | PostgreSQL type | Null | Key                                          | Default / constraint          | Description                                                              |
|-----|----------------------|-----------------|------|----------------------------------------------|-------------------------------|--------------------------------------------------------------------------|
| 1   | id                   | uuid            | No   | PK                                           | Default:<br>gen_random_uuid() | Unique identifier for this record.                                       |
| 2   | user_id              | uuid            | No   | FK -<br>><br>user_<br>acco<br>unt.id<br>; UQ | -                             | References the related user_account record.                              |
| 3   | auto_apply_enabled   | boolean         | No   | -                                            | Default: false                | Indicates whether AutoFit auto-application is enabled for the account.   |
| 4   | auto_apply_threshold | numeric(5,2)    | No   | -                                            | Default: 90.00; Range 0-100   | Minimum normalized score configured for AutoFit application eligibility. |
| 5   | auto_invite_enabled  | boolean         | No   | -                                            | Default: false                | Stored policy flag for automated invitation support.                     |

| No. | Column                   | PostgreSQL type        | Null | Key | Default / constraint                                       | Description                                                 |
|-----|--------------------------|------------------------|------|-----|------------------------------------------------------------|-------------------------------------------------------------|
| 6   | daily_digest_enabled     | boolean                | No   | -   | Default: true                                              | Indicates whether the daily digest is enabled.              |
| 7   | daily_digest_time        | time without time zone | No   | -   | Default: '08:00:00'::time without time zone                | Local time at which the digest is intended to run.          |
| 8   | user_timezone            | character varying(50)  | No   | -   | Default: 'Asia/Ho_Chi_Minh'                                | Time zone used to interpret digest and quiet-hour settings. |
| 9   | quiet_hours_enabled      | boolean                | No   | -   | Default: false                                             | Indicates whether notification quiet hours are active.      |
| 10  | quiet_hours_start        | time without time zone | No   | -   | Default: '22:00:00'::time without time zone                | Local start time of the configured quiet-hours interval.    |
| 11  | quiet_hours_end          | time without time zone | No   | -   | Default: '07:00:00'::time without time zone                | Local end time of the configured quiet-hours interval.      |
| 12  | job_scan_enabled         | boolean                | No   | -   | Default: false                                             | Indicates whether scheduled Job scanning is enabled.        |
| 13  | job_scan_frequency_hours | integer                | No   | -   | Default: 1; Frequency must be positive; delay non-negative | Configured interval between scheduled Job scans.            |

| No. | Column                         | PostgreSQL type          | Null | Key | Default / constraint                                        | Description                                                               |
|-----|--------------------------------|--------------------------|------|-----|-------------------------------------------------------------|---------------------------------------------------------------------------|
| 14  | high_match_email_enabled       | boolean                  | No   | -   | Default: false                                              | Indicates whether high-match email notifications are enabled.             |
| 15  | high_match_threshold           | numeric(5,2)             | No   | -   | Default: 90.00; Range 0-100                                 | Minimum score used by the high-match email policy.                        |
| 16  | max_email_per_day              | integer                  | No   | -   | Default: 5; Must be non-negative                            | Maximum number of policy-controlled emails allowed per day.               |
| 17  | notification_cool_down_hours   | integer                  | No   | -   | Default: 24; Must be non-negative                           | Minimum cooldown between related notification categories.                 |
| 18  | replacement_after_skip_enabled | boolean                  | No   | -   | Default: false                                              | Indicates whether a replacement recommendation may follow a skipped item. |
| 19  | replacement_delay_minutes      | integer                  | No   | -   | Default: 45; Frequency must be positive; delay non-negative | Delay before a configured replacement notification is eligible.           |
| 20  | email_action_enabled           | boolean                  | No   | -   | Default: true                                               | Indicates whether actionable email links are enabled.                     |
| 21  | created_at                     | timestamp with time zone | No   | -   | Default: now()                                              | Timestamp when the record was created.                                    |

| No. | Column                      | PostgreSQL type          | Null | Key | Default / constraint | Description                                                 |
|-----|-----------------------------|--------------------------|------|-----|----------------------|-------------------------------------------------------------|
| 22  | updated_at                  | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was last updated.                 |
| 23  | version                     | bigint                   | No   | -   | Default: 0           | Optimistic-lock version used to detect conflicting updates. |
| 24  | email_notifications_enabled | boolean                  | No   | -   | Default: true        | Global account switch for email notifications.              |
| 25  | paused_until                | timestamp with time zone | Yes  | -   | -                    | Timestamp until which account-level automation is paused.   |

### 1.1.16 email\_action

Tracks creation and delivery lifecycle metadata for an outbound actionable email.

Table App.C.17. Data dictionary for email\_action

| No. | Column            | PostgreSQL type           | Null | Key                                  | Default / constraint          | Description                                                    |
|-----|-------------------|---------------------------|------|--------------------------------------|-------------------------------|----------------------------------------------------------------|
| 1   | id                | uuid                      | No   | PK                                   | Default:<br>gen_random_uuid() | Unique identifier for this record.                             |
| 2   | recipient_user_id | uuid                      | No   | FK -<br>><br>user_<br>acco<br>unt.id | -                             | References the related user_account record.                    |
| 3   | action_type       | character<br>varying(50)  | No   | -                                    | -                             | Business or system action recorded by this row.                |
| 4   | target_type       | character<br>varying(50)  | Yes  | -                                    | -                             | Type of Job, CV, or other target referenced by the record.     |
| 5   | target_id         | uuid                      | Yes  | -                                    | -                             | Identifier of the polymorphic target referenced by the record. |
| 6   | subject           | character<br>varying(500) | Yes  | -                                    | -                             | Subject line of the outbound actionable email.                 |

| No. | Column        | PostgreSQL type             | Null | Key | Default / constraint                                                                                    | Description                                                |
|-----|---------------|-----------------------------|------|-----|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 7   | template_name | character<br>varying(100)   | Yes  | -   | -                                                                                                       | Template identifier used to construct the outbound email.  |
| 8   | status        | character<br>varying(20)    | No   | -   | Default: 'CREATED';<br>Allowed: CREATED,<br>SENT, OPENED,<br>CONFIRMED,<br>REJECTED, EXPIRED,<br>FAILED | Current lifecycle state of the email action record.        |
| 9   | created_at    | timestamp with<br>time zone | No   | -   | Default: now()                                                                                          | Timestamp when the record was created.                     |
| 10  | sent_at       | timestamp with<br>time zone | Yes  | -   | -                                                                                                       | Timestamp when the email was marked as sent.               |
| 11  | opened_at     | timestamp with<br>time zone | Yes  | -   | -                                                                                                       | Timestamp when the email lifecycle recorded an open event. |
| 12  | executed_at   | timestamp with<br>time zone | Yes  | -   | -                                                                                                       | Timestamp when the referenced email action completed.      |



### 1.1.17 email\_action\_token

Stores the hash and lifecycle of a one-time actionable-email token.

Table App.C.18. Data dictionary for email\_action\_token

| No. | Column       | PostgreSQL type          | Null | Key                                  | Default / constraint          | Description                                      |
|-----|--------------|--------------------------|------|--------------------------------------|-------------------------------|--------------------------------------------------|
| 1   | id           | uuid                     | No   | PK                                   | Default:<br>gen_random_uuid() | Unique identifier for this record.               |
| 2   | token_hash   | character<br>varying(64) | No   | UQ                                   | -                             | SHA-256 hash of the one-time email action token. |
| 3   | recipient_id | uuid                     | No   | FK -<br>><br>user_<br>acco<br>unt.id | -                             | References the related user_account record.      |
| 4   | matching_id  | uuid                     | Yes  | FK -<br>><br>matc<br>hing.i<br>d     | -                             | References the related matching record.          |

| No. | Column      | PostgreSQL type             | Null | Key | Default / constraint                                                                                                                                | Description                                                  |
|-----|-------------|-----------------------------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 5   | action_type | character<br>varying(50)    | No   | -   | Allowed: GOOD_MATCH,<br>POTENTIAL,<br>BAD_MATCH,<br>NOT_INTERESTED,<br>VIEW_JOB,<br>UNSUBSCRIBE_DIGEST,<br>INVITATION_ACCEPT,<br>INVITATION_DECLINE | Business or system action recorded by<br>this row.           |
| 6   | status      | character<br>varying(20)    | No   | -   | Default: 'PENDING';<br>Allowed: PENDING,<br>REDEEMED, EXPIRED                                                                                       | Current lifecycle state of the email action<br>token record. |
| 7   | expires_at  | timestamp with<br>time zone | No   | -   | -                                                                                                                                                   | Timestamp after which the email action<br>token is rejected. |

| No. | Column         | PostgreSQL type          | Null | Key                   | Default / constraint | Description                                                      |
|-----|----------------|--------------------------|------|-----------------------|----------------------|------------------------------------------------------------------|
| 8   | redeemed_at    | timestamp with time zone | Yes  | -                     | -                    | Timestamp when the email action token was successfully redeemed. |
| 9   | created_at     | timestamp with time zone | No   | -                     | Default: now()       | Timestamp when the record was created.                           |
| 10  | application_id | uuid                     | Yes  | FK - > application.id | -                    | References the related application record.                       |

### 1.1.18 notification\_job

Stores queued notification work and its retry lifecycle.

Table App.C.19. Data dictionary for notification\_job

| No. | Column      | PostgreSQL type          | Null | Key | Default / constraint                                                               | Description                                                    |
|-----|-------------|--------------------------|------|-----|------------------------------------------------------------------------------------|----------------------------------------------------------------|
| 1   | id          | uuid                     | No   | PK  | Default:<br>gen_random_uuid()                                                      | Unique identifier for this record.                             |
| 2   | job_type    | character<br>varying(50) | No   | -   | -                                                                                  | Type of notification work stored in the queue.                 |
| 3   | payload     | jsonb                    | No   | -   | -                                                                                  | JSON payload required to execute the queued notification work. |
| 4   | status      | character<br>varying(20) | No   | -   | Default: 'PENDING';<br>Allowed: PENDING,<br>PROCESSING, DONE,<br>FAILED, CANCELLED | Current lifecycle state of the notification job record.        |
| 5   | retry_count | integer                  | No   | -   | Default: 0                                                                         | Number of processing retries already attempted.                |

| No. | Column        | PostgreSQL type          | Null | Key | Default / constraint | Description                                                |
|-----|---------------|--------------------------|------|-----|----------------------|------------------------------------------------------------|
| 6   | next_retry_at | timestamp with time zone | Yes  | -   | -                    | Timestamp when the failed notification job may be retried. |
| 7   | created_at    | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was created.                     |
| 8   | updated_at    | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was last updated.                |

### 1.1.19 notification\_delivery\_log

Records notification-policy outcomes used for quota, deduplication, cooldown, and audit evidence.

Table App.C.20. Data dictionary for notification\_delivery\_log

| No. | Column            | PostgreSQL type           | Null | Key                                  | Default / constraint | Description                                                      |
|-----|-------------------|---------------------------|------|--------------------------------------|----------------------|------------------------------------------------------------------|
| 1   | id                | uuid                      | No   | PK                                   | -                    | Unique identifier for this record.                               |
| 2   | recipient_user_id | uuid                      | No   | FK -<br>><br>user_<br>acco<br>unt.id | -                    | References the related user_account record.                      |
| 3   | email_type        | character<br>varying(80)  | No   | -                                    | -                    | Notification or email category evaluated by the delivery policy. |
| 4   | context_key       | character<br>varying(120) | Yes  | -                                    | -                    | Optional business context used for notification deduplication.   |

| No. | Column     | PostgreSQL type             | Null | Key | Default / constraint | Description                                                         |
|-----|------------|-----------------------------|------|-----|----------------------|---------------------------------------------------------------------|
| 5   | status     | character<br>varying(20)    | No   | -   | -                    | Current lifecycle state of the notification<br>delivery log record. |
| 6   | reason     | character<br>varying(255)   | Yes  | -   | -                    | Business reason associated with the<br>report or delivery outcome.  |
| 7   | created_at | timestamp with<br>time zone | No   | -   | Default: now()       | Timestamp when the record was created.                              |

## C.2.4 Reporting, Analytics, and Audit

### 1.1.20 *content\_report*

Stores Candidate or Recruiter reports about a Job or visible CV and the moderation outcome.

*Table App.C.21. Data dictionary for content\_report*

| No. | Column      | PostgreSQL type          | Null | Key                                  | Default / constraint | Description                                                    |
|-----|-------------|--------------------------|------|--------------------------------------|----------------------|----------------------------------------------------------------|
| 1   | id          | uuid                     | No   | PK                                   | -                    | Unique identifier for this record.                             |
| 2   | reporter_id | uuid                     | No   | FK -<br>><br>user_<br>acco<br>unt.id | -                    | References the related user_account record.                    |
| 3   | target_type | character<br>varying(10) | No   | -                                    | Allowed: JOB, CV     | Type of Job, CV, or other target referenced by the record.     |
| 4   | target_id   | uuid                     | No   | -                                    | -                    | Identifier of the polymorphic target referenced by the record. |



| No. | Column      | PostgreSQL type            | Null | Key                              | Default / constraint                                                                                                                                         | Description                                                     |
|-----|-------------|----------------------------|------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| 5   | reason      | character<br>varying(40)   | No   | -                                | Allowed:<br>IMPERSONATION,<br>FRAUD_SCAM,<br>FALSE_INFORMATION,<br>INAPPROPRIATE_CONTENT,<br>DISCRIMINATION_HARASSMENT,<br>PRIVACY_VIOLATION,<br>SPAM, OTHER | Business reason associated with the report or delivery outcome. |
| 6   | comment     | character<br>varying(1000) | Yes  | -                                | -                                                                                                                                                            | Optional explanatory comment supplied by the reporter.          |
| 7   | status      | character<br>varying(20)   | No   | -                                | Default: 'PENDING';<br>Allowed: PENDING,<br>DISMISSED, ACTIONED                                                                                              | Current lifecycle state of the content report record.           |
| 8   | resolved_by | uuid                       | Yes  | FK -<br>><br>user_<br>account.id | -                                                                                                                                                            | References the related user_account record.                     |

| No. | Column          | PostgreSQL type             | Null | Key | Default / constraint | Description                                          |
|-----|-----------------|-----------------------------|------|-----|----------------------|------------------------------------------------------|
| 9   | resolution_note | character<br>varying(500)   | Yes  | -   | -                    | Administrator note explaining the report resolution. |
| 10  | created_at      | timestamp with<br>time zone | No   | -   | Default: now()       | Timestamp when the record was created.               |
| 11  | resolved_at     | timestamp with<br>time zone | Yes  | -   | -                    | Timestamp when the content report was resolved.      |

### 1.1.21 *analytics\_event*

Records product and recruitment events used by the implemented analytics views.

*Table App.C.22. Data dictionary for analytics\_event*

| No. | Column        | PostgreSQL type          | Null | Key                                  | Default / constraint | Description                                                            |
|-----|---------------|--------------------------|------|--------------------------------------|----------------------|------------------------------------------------------------------------|
| 1   | id            | uuid                     | No   | PK                                   | -                    | Unique identifier for this record.                                     |
| 2   | actor_user_id | uuid                     | Yes  | FK -<br>><br>user_<br>acco<br>unt.id | -                    | References the related user_account record.                            |
| 3   | actor_role    | character<br>varying(30) | Yes  | -                                    | -                    | Role of the user that produced the event or feedback.                  |
| 4   | event_type    | character<br>varying(60) | No   | -                                    | -                    | Analytics event category.                                              |
| 5   | subject_type  | character<br>varying(60) | Yes  | -                                    | -                    | Type of business object associated with the analytics event.           |
| 6   | subject_id    | uuid                     | Yes  | -                                    | -                    | Identifier of the business object associated with the analytics event. |

| No. | Column      | PostgreSQL type          | Null | Key | Default / constraint | Description                                          |
|-----|-------------|--------------------------|------|-----|----------------------|------------------------------------------------------|
| 7   | metadata    | jsonb                    | Yes  | -   | -                    | Structured JSON metadata associated with the record. |
| 8   | occurred_at | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the analytics event occurred.         |
| 9   | created_at  | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was created.               |

### 1.1.22 *audit\_log*

Stores append-oriented evidence for important user, system, moderation, and automated actions.

*Table App.C.23. Data dictionary for audit\_log*

| No. | Column      | PostgreSQL type           | Null | Key | Default / constraint                 | Description                                                         |
|-----|-------------|---------------------------|------|-----|--------------------------------------|---------------------------------------------------------------------|
| 1   | id          | uuid                      | No   | PK  | Default:<br>gen_random_uuid()        | Unique identifier for this record.                                  |
| 2   | actor_type  | character<br>varying(20)  | No   | -   | Allowed: USER, SYSTEM                | Distinguishes user-generated and system-generated audit actions.    |
| 3   | actor_id    | uuid                      | Yes  | -   | -                                    | Stores the actor id value for this record.                          |
| 4   | action_type | character<br>varying(100) | No   | -   | -                                    | Business or system action recorded by this row.                     |
| 5   | target_type | character<br>varying(50)  | Yes  | -   | -                                    | Type of Job, CV, or other target referenced by the record.          |
| 6   | target_id   | uuid                      | Yes  | -   | -                                    | Identifier of the polymorphic target referenced by the record.      |
| 7   | result      | character<br>varying(20)  | Yes  | -   | Allowed: SUCCESS,<br>FAILURE, DENIED | Recorded success, failure, or denial outcome of the audited action. |

| No. | Column         | PostgreSQL type             | Null | Key | Default / constraint                                 | Description                                                                    |
|-----|----------------|-----------------------------|------|-----|------------------------------------------------------|--------------------------------------------------------------------------------|
| 8   | source_channel | character<br>varying(20)    | Yes  | -   | Allowed: WEB, EMAIL,<br>DIGEST, AUTOPILOT,<br>SYSTEM | Stores the source channel value for this<br>record.                            |
| 9   | ip_address     | character<br>varying(45)    | Yes  | -   | -                                                    | Request IP address retained as audit<br>metadata when available.               |
| 10  | user_agent     | character<br>varying(500)   | Yes  | -   | -                                                    | Browser or client identification retained<br>as audit metadata when available. |
| 11  | metadata       | jsonb                       | Yes  | -   | -                                                    | Structured JSON metadata associated<br>with the record.                        |
| 12  | created_at     | timestamp with<br>time zone | No   | -   | Default: now()                                       | Timestamp when the record was created.                                         |

### 1.1.23 job\_trend\_snapshot

Stores date-based view and application counters for an individual Job.

Table App.C.24. Data dictionary for job\_trend\_snapshot

| No. | Column            | PostgreSQL type          | Null | Key                         | Default / constraint             | Description                                                          |
|-----|-------------------|--------------------------|------|-----------------------------|----------------------------------|----------------------------------------------------------------------|
| 1   | id                | uuid                     | No   | PK                          | Default:<br>gen_random_uuid()    | Unique identifier for this record.                                   |
| 2   | job_id            | uuid                     | No   | FK -<br>><br>job.id<br>; UQ | -                                | References the related job record.                                   |
| 3   | snapshot_date     | date                     | No   | UQ                          | -                                | Date represented by the aggregate or per-Job snapshot.               |
| 4   | view_count        | integer                  | No   | -                           | Default: 0; Must be non-negative | Stored number of Job-detail views.                                   |
| 5   | application_count | integer                  | No   | -                           | Default: 0; Must be non-negative | Stored application count used by Job popularity and analytics flows. |
| 6   | created_at        | timestamp with time zone | No   | -                           | Default: now()                   | Timestamp when the record was created.                               |

### 1.1.24 job\_market\_snapshot

Stores date-based aggregate Job-market counts and distributions.

Table App.C.25. Data dictionary for job\_market\_snapshot

| No. | Column            | PostgreSQL type | Null | Key | Default / constraint             | Description                                            |
|-----|-------------------|-----------------|------|-----|----------------------------------|--------------------------------------------------------|
| 1   | id                | uuid            | No   | PK  | Default:<br>gen_random_uuid()    | Unique identifier for this record.                     |
| 2   | snapshot_date     | date            | No   | UQ  | -                                | Date represented by the aggregate or per-Job snapshot. |
| 3   | total_posted_jobs | integer         | No   | -   | Default: 0; Must be non-negative | Total number of posted Jobs counted for the snapshot.  |
| 4   | active_jobs       | integer         | No   | -   | Default: 0; Must be non-negative | Number of active Jobs counted for the snapshot.        |
| 5   | new_jobs          | integer         | No   | -   | Default: 0; Must be non-negative | Number of newly posted Jobs counted for the snapshot.  |
| 6   | employer_count    | integer         | No   | -   | Default: 0; Must be non-negative | Number of employers represented in the snapshot.       |



| No. | Column                 | PostgreSQL type          | Null | Key | Default / constraint | Description                                   |
|-----|------------------------|--------------------------|------|-----|----------------------|-----------------------------------------------|
| 7   | distribution_by_role   | jsonb                    | Yes  | -   | -                    | JSON distribution of Jobs by role category.   |
| 8   | distribution_by_salary | jsonb                    | Yes  | -   | -                    | JSON distribution of Jobs by salary category. |
| 9   | created_at             | timestamp with time zone | No   | -   | Default: now()       | Timestamp when the record was created.        |

## **Appendix D. Evaluation Summary**

The final evidence package contains the 141-test backend reports, controlled algorithm benchmark, frontend type/lint/build results, 49-test Chrome output, runtime health evidence, the updated report, and evaluation/result.json. Chapter 4 reports what this evidence supports and what it does not support.

## **Appendix E. UAT and Demonstration Script**

### **Preparation**

1. Start PostgreSQL, the backend, and the frontend; verify aggregate health and the public Job API; and prepare separate Candidate, Recruiter, and Administrator accounts.
2. Record the current Git commit, working-tree status, configuration profile, database source, and evaluation artifact locations.
3. Use clearly marked test records and note the cleanup action for every Job, CV, application, invitation, or policy created during the demonstration.

### **Execution**

1. As a Guest, search for an IT Job, apply filters, and open Job and employer details.
2. As a Candidate, sign in, upload or draft a CV, review and confirm it, inspect matches, apply or withdraw, submit feedback, and report one designated active test Job with a clear reason.
3. As a Recruiter, complete the company profile, publish a test Job, inspect applicants and Talent Pool, bookmark and invite a Candidate, update application status, and report one visible test CV using the owned Job context.
4. Configure Candidate AutoFit enablement and threshold, verify pause/resume, then use Run Now for a controlled check. Review notification quota, cooldown, quiet hours, and email preferences separately as supporting settings.
5. As an Administrator, inspect users, audit records, and the Report moderation tab; open the designated test report case; dismiss it or ban only disposable test content; then verify status, counters, and audit evidence.
6. Show Actuator health, the archived backend and Chrome test results, the benchmark artifact, and the final cleanup state.

### **Acceptance and cleanup**

1. Record each expected and observed result, preserve any failure evidence, and do not convert an unverified step into a passing claim.
2. Delete or deactivate demonstration data, restore account state, and confirm that background logs contain no unhandled exception.
3. Archive the final evidence with a clean commit or a complete source-state package.

## **Appendix F. Local Deployment Instructions**

### **Prerequisites**

- Install Java 21, Node.js with npm, Docker Desktop with Docker Compose, and Git.
- Provide environment-specific database, JWT, CORS, application URL, mail, OCR, and storage configuration. Do not commit production secrets.

### **Startup procedure**

1. From the repository root, start PostgreSQL with ``docker compose up -d postgres`` and wait for the container health check to pass.
2. From ``Backend/careerfit-backend``, run ``mvnw.cmd spring-boot:run``. The host backend connects to Compose PostgreSQL at ``localhost:5433``.
3. From ``Frontend``, run ``npm install`` when dependencies are not present, followed by ``npm run dev``. The local Vite application is available at ``http://localhost:5173``.

### **Verification**

1. Open ``http://localhost:8080/actuator/health`` and verify the expected aggregate, liveness, and readiness responses for the selected profile.
2. Open the frontend, verify public Job search, sign in with a designated test account, and confirm authenticated API access.
3. If OCR, mail, or another optional dependency is disabled, record the limitation instead of treating the missing external service as a verified production integration.

### **Shutdown and cleanup**

1. Stop the frontend and backend processes, then use ``docker compose down`` to stop the local containers. Remove volumes only when test data loss is intended.

## **Appendix G. Role-Based User Guide**

### **Guest**

Open the Jobs page, search by title, skill, or company, select filters, and open a Job card to view its public details. Authentication is required before application or personalized scoring.

### **Candidate**

Sign in, open My CVs, upload a PDF/DOCX/image or save a manual draft. Review and confirm the sections before scoring. Then choose the default CV, inspect match and Potential reasons, manage applications, provide feedback, configure AutoFit, and use Report on an active Job only when there is a real issue.

### **Recruiter**

Sign in to the Recruiter workspace, complete the company profile, and create or publish Jobs. Use applicants and Talent Pool to review visible CVs, bookmark or invite a Candidate, and update valid application states. A visible CV can be reported only with an owned Job that gives the Recruiter access to it.

### **Administrator**

Use the administrative workspace to inspect users, Jobs, audit records, notifications, email actions, and Report moderation. Review the report reasons and target details before choosing Dismiss or Ban. Banning is a state-changing action and should be tested only on disposable records.